

COPYCAT

שם פרויקט : COPYCAT

שם תלמיד : נופר .ל. עמיעז

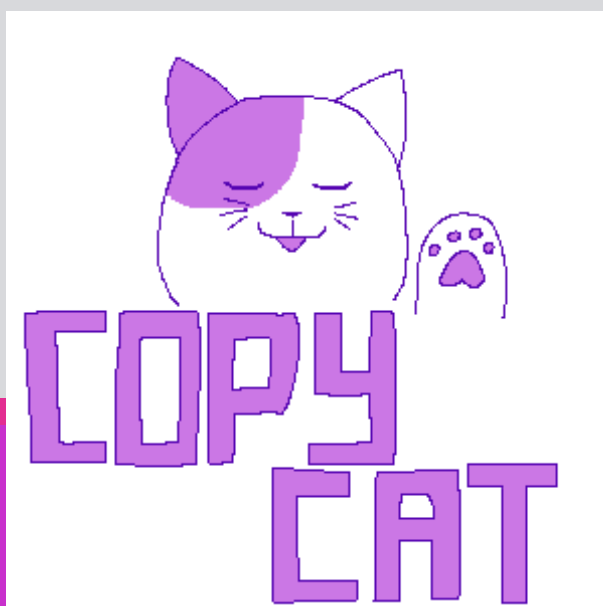
ת.ז תלמיד : 216787903

שם בית ספר : תיכון אלון רמת השרון

שם המנחה : עידן פלדברג

שם החלופה : הגנת סייבר

תאריך הגשה : 25.5.24



תוכן עניינים

3.....	מבוא
3.....	ייזום
4.....	פירוט תיאור המערכת
5.....	תכנון
6.....	ניהול סיכונים
7.....	תיאור תחום הידע – פרק מילולי
7.....	יכולות המערכת
10.....	מבנה
10.....	תיאור הארכיטקטורה
10.....	תיאור הטכנולוגיה הרלוונטית
11.....	תיאור זרימת המידע במערכת
12.....	תיאור האלגוריתמים המרכזיים במערכת
15.....	תיאור סביבת הפיתוח
16.....	תיאור פרוטוקול התקשורת
18.....	תיאור מסכי המערכת
22.....	תיאור מבני הנתונים
24.....	סקירת חולשות ואיומים
26.....	מימוש הפרויקט
26.....	חלק א
36.....	חלק ב
45.....	חלק ג
49.....	מדריך למשתמש
49.....	עץ קבצים
50.....	התקנת המערכת
51.....	משתמשי המערכת
59.....	רפלקציה
60.....	ביבליוגרפיה
61.....	נספחים
61.....	הקובץ copyCatClient.py
63.....	הקובץ copyCatClient_comm.py
68.....	הקובץ copyCatGUI.py
115.....	הקובץ clientSideConstants.py
118.....	הקובץ request_type.py
120.....	הקובץ copyCatServer.py
122.....	הקובץ copyCatServer_comm.py
131.....	הקובץ copyCatHandle.py

175..... **הקובץ serverSideConstants**

מבוא

ייזום

- הפרויקט COPYCAT הוא תוכנה להגנה מפני כופרה. הפרויקט יאפשר למשתמש לגבות את כול המידע שעל המחשב שלו לענן, המשתמש גם יוכל לפתוח את התוכנה ולהסתכל על הגיבוי ועל הקבצים שבו. המשתמש יצטרך להוכיח את זהותו לפני שהוא יוכל לצפות בגיבוי. הגיבוי יישמר באותו הארגון שבו המידע נמצא במחשב של המשתמש – הקבצים והתיקיות יהיו באותן התיקיות שהם נמצאים בהם על מחשב הלקוח. הפרויקט יוודא את זהות הלקוח באימות דו שלבי (שאלה שהושמה על ידי המשתמש בהפעלה הראשונית של הפרויקט – something you know ובאמצעות שליחת אימייל – something you have). התוכנה תשמור מספר גיבויים שונים (לפי בחירת המשתמש שיוכל להעלות גיבוי של המידע שעל מחשבו בכול זמן) והמשתמש יוכל להוריד כול אחד מהם למחשבו ולפתוח קבצים מהם לפי בחירתו. בחרתי את הפרויקט הזה מכיוון שתכנות בענן סקרן אותי. אני גם צופה שהמימוש של האחסון בענן יהיה החלק המאתגר ביותר בפרויקט.
- הלקוח יהיה מי שרוצה ליצור גיבוי מאובטח של המידע שלא נמצא על המחשב שלו ככה שהוא לא יוכל להיות קורבן של סחיטה באמצעות איום מחיקת המידע שלו. או לקוח שרוצה גיבוי בטוח של המידע שלו למקרה ובו המחשב קורס או שהמידע נפגע מכול סיבה אחרת. הלקוח יוכל לשחזר מידע מכול גיבוי שהוא העלה.
- מטרת הפרויקט היא לייצר גיבוי בטוח של המידע של המשתמש בענן וגם לאפשר למשתמש גישה למידע השמור בענן (לצפייה) בלי שהוא יצטרך להוריד אותו למחשב שלו ובלי שתוקף יוכל לגשת למידע השמור בענן ולעשות את אותו הדבר.
- הבעיה – ניסיונות סחיטה וסכנות שונות לבטיחות מידע המשתמש. תועלת התוכנית – אחסון המידע במקום נפרד מהמחשב אך מוגש למשתמש ומוגן מתוקפים. המערכת תיתן שירות של גיבוי מידע המשתמש, הורדת הגיבוי למחשב המשתמש ויכולת לצפות בקבצים השמורים בענן.
- השוואה עם יישומים קיימים : idrive – קווי דימיון : הוא מגבה את כול המחשב ולא רק קבצים הבדלים : הוא שומר עד 30 גיבויים שונים, מאפשר לשחזר קבצים מזמנים שונים, מאפשר שימוש מכמה מכשירים.

פירוט תיאור המערכת

- המערכת שיצרתי מגנה על המידע של המשתמש ושומרת גיבויים למקרה הצורך. לכל משתמש יש דלי של מידע שבו מאוחסנים הגיבויים שלו. בכל גיבוי יש את כל המידע של המשתמש ברגע העדכון לפי הסדר שבו המשתמש שמר אותו על המחשב שלו. כשהמשתמש מאבד את המידע, או רוצה להסתכל על מידע ישן הוא יכול למצוא גיבויים מתאריכים שונים, הוא יכול להוריד את כל הקבצים ולהסתכל עליהם.
- במערכת שלי יש רק סוג משתמש אחד, שהוא ה-USER. היכולות שלו:
 - **העלאת גיבוי**: המשתמש יכול להעלות גיבויים לשרת דרך הממשק.
 - **הורדת גיבוי**: המשתמש יכול להוריד את הקבצים שהוא העלה.
 - **צפייה בפרטי הדלי**: המשתמש יכול לראות כמה גיבויים יש בדלי שלו, מתי הם הועלו לענן וכמה בתים של מידע יש בהם.
 - **צפייה בפרטי הגיבוי**: המשתמש יכול לראות רשימה של הקבצים השמורים בגיבוי מסוים, הוא יכול לראות איפה הקבצים שמורים בענן (כדאי שאני אשנה את זה).
 - **פתיחה של קובץ ספציפי**: המשתמש יכול לבחור קובץ ולפתוח אותו במחשב שלו, כשהמשתמש עושה את זה הקובץ לא נשמר במחשב שלו. אלא בקובץ זמני שנמחק עם סגירת הקובץ. לא ניתן להוריד רק קובץ אחד מהענן.

פירוט הבדיקות:

הבדיקות בנויות מכמה שלבים.

השלב הראשון – בדיקת התוכנה כשהלקוח והשרת רצים על אותו המחשב.

השלב הזה הוא הבדיקה הבסיסית ביותר של הקוד. בו אני מנסה להריץ אותו בלי לבדוק יוצאים מן הכלל או לנסות להכשיל אותו. בשלב זה דיבאגתי ברמה הבסיסית ביותר כך שהתוכנה לא תעבוד כרצוי ולא תקרוס. בבדיקה זו מסד הנתונים לא היה מוצפן, מכיוון שהייתי צריכה לבדוק אם התוכנה הכניסה את הערכים הנכונים. לכן הייתי צריכה שהמידע יהיה גלוי. מאותה הסיבה גם ההודעות שעברו בין השרת ללקוח לא היו מוצפנות.

השלב השני – בדיקת exceptions

בבדיקה זו אני אנסה למצוא את הפרצות בתוכנה וכשלונות של מקרים שלא התכוננתי אליהם (לדוגמא: שהשרת לא יקבל מידע מהמשתמש כשהוא מצפה לו ועוד). בדיקת ההצפנה, בה אני אבדוק האם הלקוח והשרת מצליחים לפענח זה את זה. זה יהיה השלב של חיזוק אבטחת הפרויקט והאלמנטים המגננטיים של הפרויקט.

שלב שלישי – בדיקה סופית

בדיקה זו תהיה בדיקה של התמודדות הפרויקט עם כמה לקוחות מכמה מחשבים. זו תהיה הבדיקה הסופית כדי לוודא שלא פספסתי שום דבר לפני הגשת הפרויקט.

תכנון

בשלב הראשון, תכננתי לבצע את הפעולות הבאות:

- **יצירת שרת** : אבנה את השרת שינהל את הבקשות מהלקוח, ינהל את הגיבויים ואת ה-DB ויבטיח שמירת מידע בצורה מאובטחת.
- **יצירת לקוח** : אפתח את החלק של המערכת שיתקשר עם המשתמש, יאפשר לו להעלות ולהוריד קבצים, ולנהל את הגיבויים בצורה נוחה.
- **יצירת GUI** : אבנה את הממשק הגרפי (GUI) שיהיה אינטואיטיבי וקל לשימוש, ויאפשר למשתמש לבצע את כל הפעולות הנדרשות.
- **בניית DB לפרויקט** : אבנה את מסד הנתונים שיאחסן את המידע של המשתמשים, של הגיבויים שלהם ושל המידע שלהם לאימות הזהות שלהם.
- **כתיבת גרסה ראשונית לספר הפרויקט**

בשלב הראשון של התוכנית שלי רציתי לסיים את כל הדברים הללו במקביל עד להגשת הפרויקט הראשוני.

בשלב השני

- הצפנה : אני אוסיף הצפנה לתקשורת בין השרת ללקוח ואצפין את מאגרי המידע.
 - בדיקות : אוודא שהפרויקט שלי עובד ואתקן אותו בהתאם.
 - עיצוב : אוסיף לוגיים ותמונות כדי שהפרויקט יראה יפה למשתמש.
- בשלב השלישי, אני אסיים לכתוב את ספר הפרויקט ואצלם את הסרטון.

ניהול סיכונים

ביצועים תחת עומס – לוודא שהמערכת פועלת בצורה מהירה ויעילה גם כאשר ישנם הרבה משתמשים בו זמנית. למשל אם שני משתמשים ינסו להעלות גיבויים במקביל, זה יכול להיות ליצור עומס על השרת ולכן זה יהיה אתגר בכתיבת השרת ובניהול שלו.

אבטחת מידע – להבטיח שהמידע של המשתמשים נשאר מאובטח, כולל הצפנה ושמירה על פרטיות הנתונים. לוודא שהמערכת מאובטחת נגד התקפות נפוצות. לדוגמה לוודא שכאשר המשתמש מכניס את הנתונים שלו למערכת, לא ניתן לבצע INJECTION SQL. אתגר נוסף, היה ללמוד הצפנה לצורך אבטחת המידע. בנוסף, כדי לשמור על פרטיות המשתמש חשוב להצפין גם את מסד המידע וגם את התקשורת בין הלקוח לשרת כדי למנוע האזנה של צד שלישי.

העברה של נתונים בין השרת ללקוח – הייתי צריכה לוודא שהמחשב שלי שמשמש כשרת בפרויקט יוכל לעמוד בכמות הזיכרון שצורך המידע שהשתמש מנסה לשמור. לכן הייתי צריכה ללמוד להשתמש ב-ZIP כדי לדחוס מידע. בנוסף לכך, גם היה צריך לוודא שהשרת והלקוח אוספים את כול המידע. למשל, בקשת העלאת הגיבוי של הלקוח היא גדולה מאוד, ועלולה לעלות על המספר המקסימלי של בתים שהשרת יכול לקבל מהודעה. לכן שמתי אלגוריתם שמוודא שכול המידע הגיע באמצעות כך שהוא משווא את אורך ההודעה לשדה header שנקרא content_length אותו העיקרון עובד גם בצד הלקוח.

תיאור תחום הידע – פרק מילולי

יכולות המערכת

יכולות בצד השרת:

בצד השרת ישנן מספר יכולות חשובות שמטרתן לבצע את האינטרקציה עם הלקוח ולטפל בגיבויים.

יכולת: ניהול גיבויים

מהות היכולת:

ניהול הגיבויים של המשתמש, שמירת הקבצים ושליחת הקבצים לצפייה והורדה. בנוסף, הוא יכול ליצור ולהציג נתונים של הדלי, בלומר על הגיבויים השונים השמורים בו ורשימת הקבצים השמורים בגיבויים.

אוסף הפעולות הנדרשות למימוש היכולת:

- יצירת גיבויים של כל הקבצים על מחשב המשתמש וכל הדברים שנמצאים עליו בסדר בו הם שמורים במחשב הלקוח.
- שמירת מידע על הגיבויים (תאריך העלאה, כמות הבתים) במסד הנתונים.
- שמירת מידע על הדלי (כמות הגיבויים בו, כמות הבתים בכול הדלי סך הכול, תאריך העדכון האחרון) במסד הנתונים.
- שליפת המידע שהלקוח מבקש ממסד הנתונים.
- שליחת מידע הגיבויים למחשב המשתמש לפי בקשתו.
- שליחת מידע של קובץ אחד לצורך צפייה.

אובייקטים נחוצים: מרחב שמירת קבצים (storage space), מסד הנתונים, יכולת לשלוח מידע למשתמש (באמצעות socket)

יכולת: ניהול מסד נתונים

מהות היכולת:

ניהול הטבלאות השונות – הוספת מידע, שליפת המידע. מסד הנתונים מכיל את הנתונים של המשתמש (שם משתמש מספר זיהוי הדלי שלו ופרטים מזהים), את הנתונים והמידע על הגיבויים (למשל, כמות הבתים שהם לוקחים, המספר שלהם בכול דלי ועוד).

אוסף הפעולות הנדרשות למימוש היכולת:

- יצירת הטבלאות השונות הנחוצות לפרויקט

- הוספת מידע באירועים המצריכים את זה: התווספות משתמשים, העלאות גיבויים
- שליפת המידע לפי הצורך

אובייקטים נחוצים: מסד נתונים

יכולת: אימות זהות המשתמש

מהות היכולת:

לשרת יש את המידע הנחוץ לבדיקת זהות המשתמש לצורכי אבטחה. הוא יוודא את זהות המשתמש באמצעות תקשורת עם הלקוח.

אוסף הפעולות הנדרשות למימוש היכולת:

- שליפת המידע הרלוונטי ממסד הנתונים
- שליחת שאלות הזיהוי ללקוח
- שליחת אימייל ללקוח
- קבלת תשובות המשתמש
- בדיקת התשובות מול מסד הנתונים

אובייקטים נחוצים: מסד הנתונים, תקשורת עם הלקוח, שרת אימייל.

יכולת: הצפנה

מהות היכולת: השרת יצפין את קבצי המשתמש כך שגם אם תוקף ישיג גישה לשטח האחסון של השרת הוא לא יוכל לדעת מה הם מכילים. כול לקוח יוצפן במפתח אחר.

אוסף הפעולות הנדרש ליכולת:

- פעולת הצפנה
- פעולת יצירת מפתחות
- יכולת לשלוח את המפתח של הלקוח

אובייקטים נחוצים: המידע שצריך להצפין, מחלקות הצפנה שונות, מסד הנתונים, שטח האחסון.

יכולות בצד הלקוח:

היכולות בצד הלקוח מטרטן להתמודד עם המשתמש באופן ישיר ולשלוח בקשות אל השרת.

יכולת: קריאת קבצי הלקוח והעלאתם לשרת**מהות היכולת:**

ללקוח יש גישה לכול הקבצים במחשבו של המשתמש, איתם הוא יכול לשלוח את המידע על מחשבו של המשתמש לשרת כדי ליצור גיבוי.

אוסף הפעולות הנדרש ליכולת:

- מעבר על כול הקבצים החשובים במחשב הלקוח.
- דחיסת הקבצים.
- הפיכת מידע הקבצים מבתים למחרוזות על מנת לשלוח את הבקשה.
- קבלת גישה לקבציו של המשתמש.

אובייקטים נחוצים: מחשב המשתמש, ספריות zlib, base64, json

יכולת: ממשק משתמש**מהות היכולת:**

ממשק המשתמש קיים כמקום בו המשתמש יכול לגשת לפונקציונאליות של המערכת בצורה נוחה.

אוסף הפעולות הנדרש ליכולת:

- חלונות שונים
- הגבה לפעולות המשתמש
- תקשורת עם שאר חלקי התוכנית

אובייקטים נחוצים: הלקוח, הממשק הגרפי, חלקי התוכנה האחרים: חלק ממימוש התקשורת, החלק העובר ומנהל את הקבצים.

מבנה

תיאור הארכיטקטורה

לתוכנה יש שני צדדים : צד שרת וצד לקוח. לכול אחד מהצדדים האלו ישנם כמה חלקי קוד

הבנויים למטרות שונות :

צד שרת :

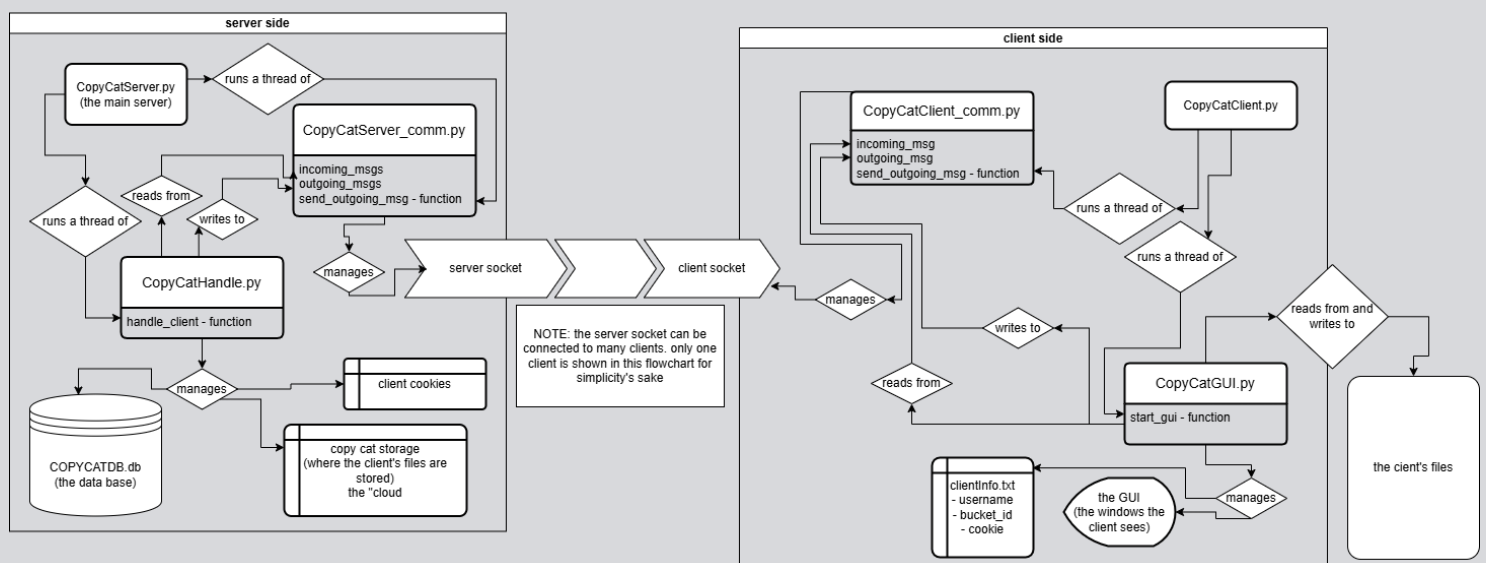
בצד השרת ישנן שלושה קבצי קוד, קובץ מסד הנתונים וקובץ קבועים. הנה פירוט על קבצי הקוד :

CopyCatHandle.py

CopyCatServer.py – זהו הקובץ שמאחד את שאר קבצי השרת ומפעיל אותם. הוא מריץ את פונקציית send_outgoing_msg של CopyCatServer_comm ואת פונקציית handle_client של CopyCatHandle בתהליכונים שונים. הוא מריץ תהליכון של handle_client על כול הודעה חדשה שהשרת מקבל. ככה השרת יכול להתמודד עם בקשות הלקוחות השונים, לקלוט לקוחות חדשים ולשלוח תשובות ללקוחות באותו הזמן ולא להיתקע.

CopyCatServer_comm.py – קובץ זה אחראי לתקשורת של צד השרת.

גרף :



תיאור הטכנולוגיה הרלוונטית

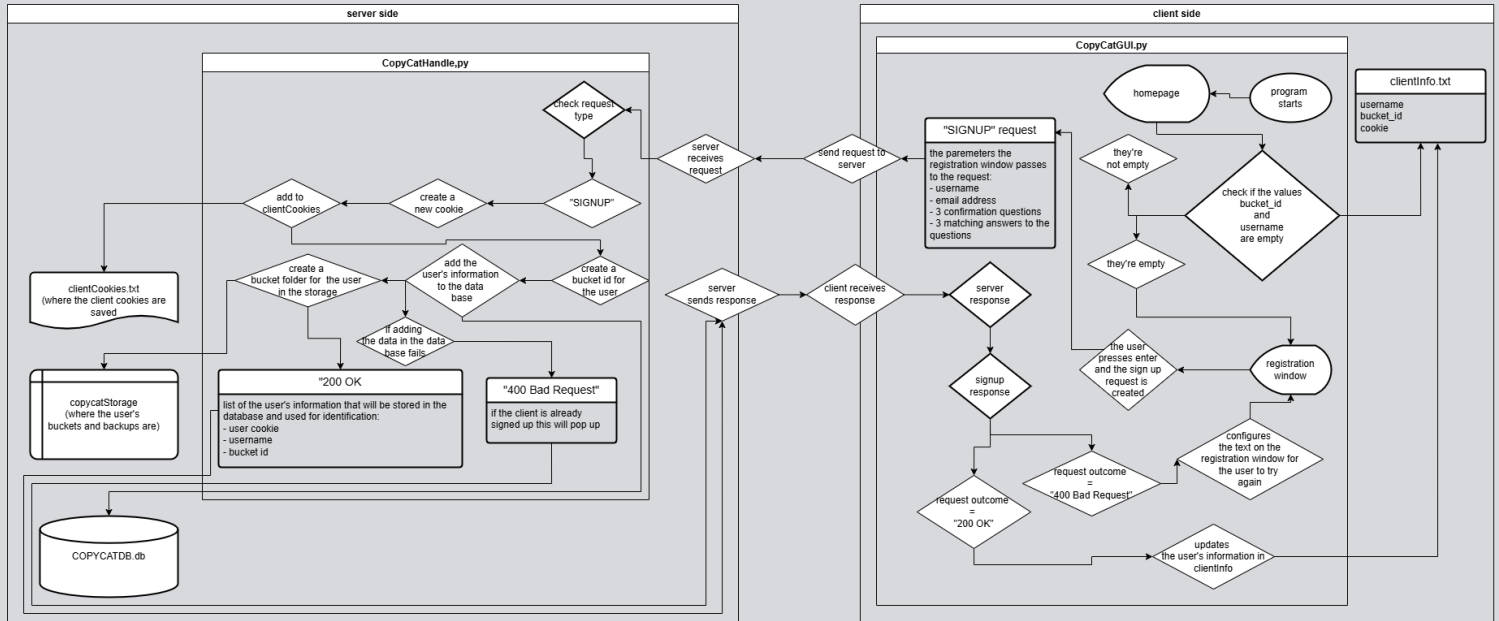
שפת תכנות : פייתון

תקשורת : TCP

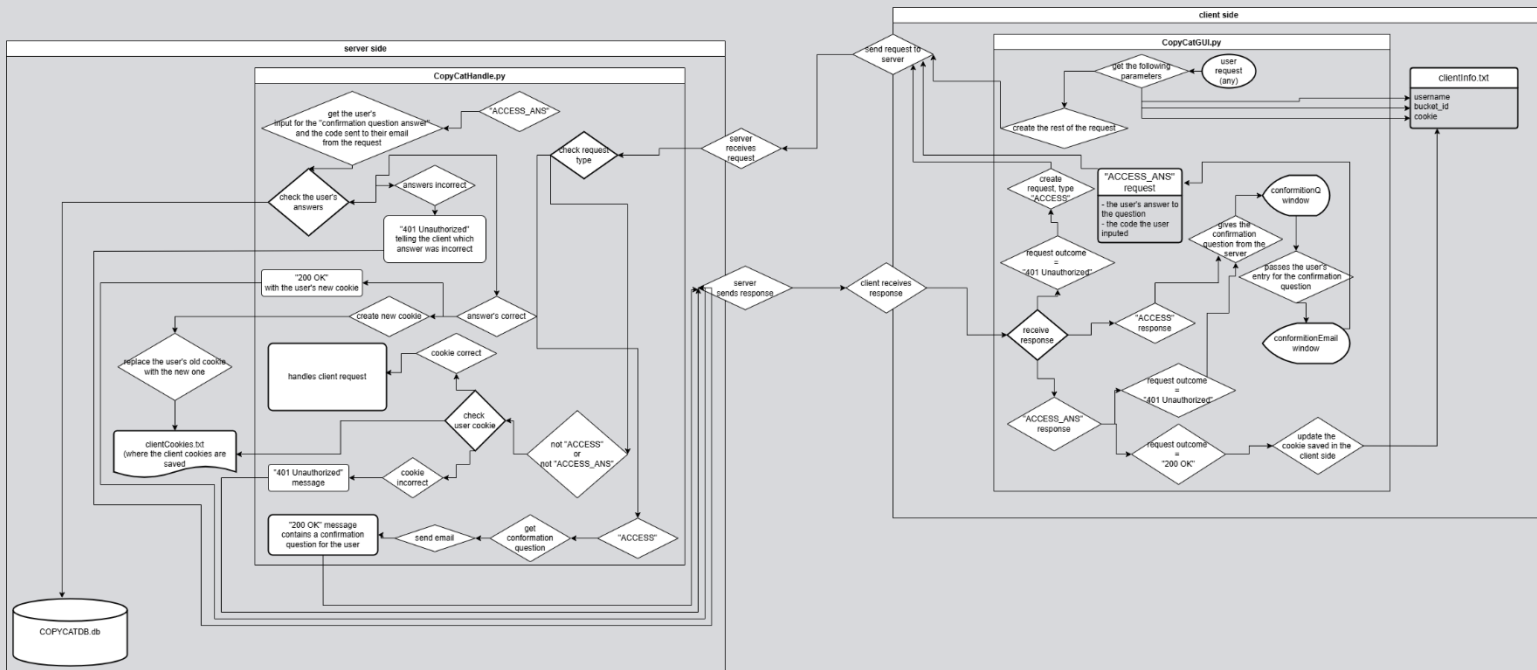
תחומי עניין : שמירה בענן (הענן יהיה המחשב צד שרת)

תיאור זרימת המידע במערכת

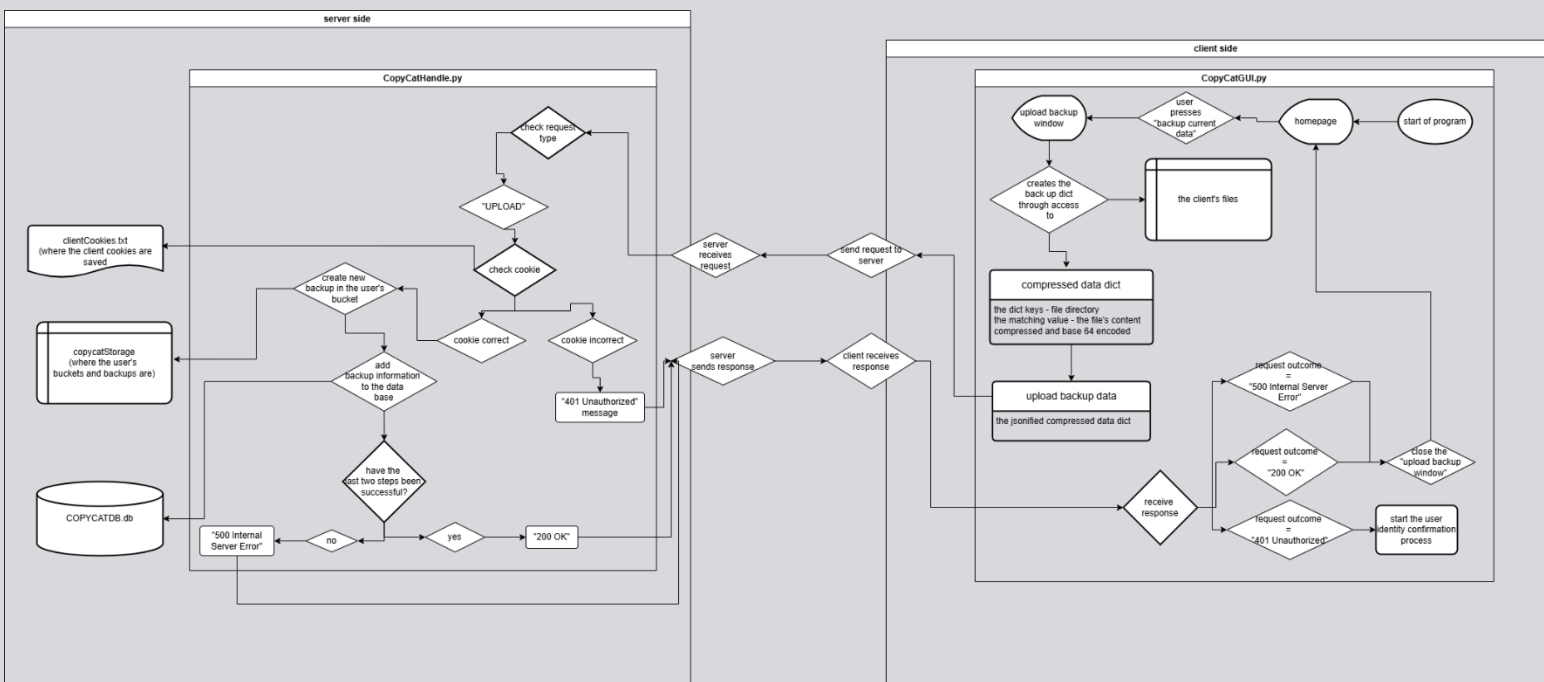
בקשת הירשמות של משתמש עם פתיחת התוכנה בפעם הראשונה:



אימות זהות המשתמש:



העלאת גיבוי מהלקוח לשרת:



תיאור האלגוריתמים המרכזיים במערכת

○ אימות דו שלבי

בעיה אלגוריתמית: אחת מהמטרות שלי בהכנת הפרויקט הזה היא אבטחת מידע. כלומר, לשמור על המידע של התוכנה ושל המשתמשים. אחד מהדברים החשובים שצריכים להתקיים בתוכנית הזאת כדי להגשים את המטרה הוא אימות זהות המשתמש. על התוכנית להיות מסוגלת לאמת את זהות המשתמש בצורה מדויקת כדי למנוע פריצה לחשבונו של המשתמש והשגת המידע האישי שלו על ידי גורמים עוינים.

דוגמאות לאלגוריתמים קיימים: ישנם מספר דרכים לאמת את זהותו של המשתמש בתוכנה, יש את גישת הטוקן, בה המשתמש מוכיח את זהותו באמצעות אובייקט פיזי או דיגיטלי. יש את גישת הסיסמה, בה המשתמש מוכיח את זהותו באמצעות מידע שרק הוא אמור לדעת. יש גם אימות זהות באמצעות העברת מפתחות. בה נשלח צופן שרק מי שיש לו את המפתח אליו יכול לפתור.

הפתרון הנבחר: אני בחרתי באלגוריתם של אימות דו שלבי. הסיבה לכך היא שאלגוריתם זה הוא הכי בטוח ובאותו זמן מספיק נפוץ כדי לא לתסכל את המשתמשים (בעיצוב מוצר צריך לאזן את הרצון של המשתמש לביטחון ובאותו הזמן את הסבלנות שלו).

בתוכנה זה עובד כך. לכול משתמש יש עוגייה, רצף של 128 ביט ששמור אצלו בclientInfo.txt ונמצא בשדה header. אם כול בקשה שהלקוח שולח לשרת בודק את העוגייה. במקרה בו העוגייה לא נכונה השרת מסרב למלא את בקשת המשתמש ועונה לו עם הקוד 401 Unauthorized. ואם הלקוח מקבל את השגיאה הזאת הוא מבקש אימות זהות מהשרת ותהליך אימות הזהות מתחיל.

השרת שולח למשתמש את אחת משאלות הקונפירמציה שהוא הכניס עם הרשמתו הראשונית לתוכנית זהו מרכיב ה"משהו שאתה יודע" (שאלת הקונפירמציה עובדת כמו סיסמא, פשוט במקרה זה זוהי תשובה לשאלה שהמשתמש הכניס לעצמו) ובאותו הזמן השרת שולח אימייל לכתובת האימייל שהמשתמש הכניס עם הרשמתו לתוכנית. הלקוח מקבל את תגובת השרת ומציג את שאלת הקונפירמציה למשתמש. המשתמש מכניס את התשובה שלו ולוחץ 'enter'. מיד אחרי התוכנית מציגה למשתמש את החלון הבא בו המשתמש מתבקש להכניס את הקוד שהוא קיבל באימייל. זהו מרכיב ה"משהו שיש לך" של האימות הדו שלבי. אם הקוד והתשובה שהמשתמש הכניס נכונים השרת יוצר לו עוגייה חדשה וזהותו של המשתמש מאומתת.

○ יצירת גיבוי

בעיה אלגוריתמית: התפקיד העיקרי של התוכנה הוא שמירת גיבויים. זה חשוב לשמור על כך שהגיבויים יקחו כמה שפחות מקום אצל השרת, מכיוון שיכולים להיות הרבה משתמשים ששומרים גיבויים והגיבויים הללו יכולים להכיל כמות רבה מאוד של קבצים. יצירת הגיבוי כוללת: מציאת הקבצים אצל מחשב המשתמש, יצירת המידע לשליחה לשרת, ושמירת המידע אצל השרת בצד שלו.

דוגמא לאלגוריתם קיים: במערכת Linux קיימת פקודה בשם tar, פקודה זו לוקחת כמה קבצים שונים ומאחדת אותם לקובץ מידע אחד, שנקרא archive file. את הקובץ הזה ניתן לדחוס. באמצעות פקודה זו ניתן לקחת כמות גדולה של קבצים, להעביר אותם ולשמור ולגבות אותם.

הפתרון הנבחר: אני לא בחרתי את פתרון פקודת tar מכמה סיבות. סיבה ראשונה, הגיבוי שנוצר באמצעות tar הוא כקובץ אחד, כלומר, אם רוצים לשנות משהו בגיבוי אז צריך להכין את כולו מחדש. לעומת זאת, הפתרון שבחרתי יכול לאפשר שינויים רק בחלק מהקבצים. יתרון זה לא מומש בתוכנית אולם הוא השפיע על הבחירה. יתרון נוסף הוא, אחד מהפיצורים של התוכנית הוא היכולת של המשתמש לבחור קובץ אחד מגיבוי אחד ולפתוח אותו. מה שאפשרי בtar אבל יותר קשה.

האלגוריתם שכתבתי ליצירת הגיבוי הוא זה:

הדבר הראשון שקורה הוא הכנת רשימה של כול הקבצים שצריך לגבות. ללקוח יש קבוע בו רשימה של התיקיות שנבחרו לגיבוי. התוכנית תרוץ עליהן בלולאה, בתוך הלולאה, תהיה רקורסיית עץ. שבאמצעותה התוכנית תעבור על כול קובץ בתוך כול תיקייה ותת תיקייה בתיקיות שנבחרו מראש. הרשימה תהיה הרשימה של כול נתיבי הקבצים. אחרי שנוצרה הרשימה התוכנית תכין מילון. מפתחות המילון יהיו נתיבי הקובץ (מאחרי התיקיות שנתון לגבות) והערכים יהיו בתי המידע של הקבצים. הדרך בה המילון ייוצר היא: התוכנית תעבור בלולאה על הרשימה, לכול קובץ הוא יפתח, יקרא את התוכן, ידחוס את המידע, יקח את המידע הדחוס ויקודד אותו בbase64. את המילון הזה התוכנית לוקחת, הופכת לסטרינג באמצעות json.dumps ושולחת את זה לשרת.

בצד השרת התוכנית מפרידה את מילון המידע משאר ההודעה ומבצעת עליו json.loads. לאחר מכן, היא עוברת בלולאה על איברי המילון, ומפענחת את קידוד base64 ומצפינה את

קבצי המשתמש באמצעות מפתח ייחודי לו ששמור במסד הנתונים. לאחר מכן התוכנית תעדכן את מסד הנתונים בנתוניו של הגיבוי החדש. תיווצר תיקייה חדשה לגיבוי בשטח האחסון של השרת, התוכנית תרוץ על איברי המילון, תיצור נתיבים לכול קובץ לפי מפתחות המילון ובתוך הנתיב תכתוב את המידע החדש.

○ שחזור מגיבוי

בעיה אלגוריתמית: כדי שגיבוי יהיה בעל משמעות צריכה להיות דרך לשחזר אותו. במקרה של התוכנה שלי, אין טעם לגיבויים אם המשתמש לא יכול להוריד אותם ולפתוח למחשב שלהם.

דוגמא מאלגוריתם קיים: פקודת tar ב Linux מסוגלת לשחזר את קובץ archiven חזרה לקבצים המקוריים מהם הוא הוכן.

הפתרון הנבחר: כדי לשחזר את הגיבוי המשתמש ישלח בקשה מתאימה עם מספר הגיבוי שהוא רוצה לשחזר.

השרת, יחלץ את המספר מההודעה, ויכין רשימה של כול הקבצים בגיבוי המסוים הזה. התוכנית תעשה זאת באמצעות רקורסיית עץ.

לאחר מכן התוכנית תרוץ בלולאה על רשימת הקבצים ותכין מילון. המפתחות יהיו נתיב הקבצים (פחות הנתיב לתיקיית שטח האחסון של השרת) והערכים יהיו בתי הקבצים החדשים שקודדו בbase64 ותפענח את קבצי המשתמש השמורים מוצפנים בשטח האחסון. ואז יעברו json.dumps. ואז שולח למשתמש.

בצד המשתמש התוכנה מחלצת את מילון המידע של הגיבוי מהודעת השרת. לאחר מכן, התוכנה עוברת על איברי מילון המידע על כול מפתח התוכנית תיצור נתיב של קובץ ותקח את הערך המתאים ותפענח את קידוד base64. אז תפרוס את המידע החדש ולבסוף תכתוב את המידע למיקום החדש שנוצר.

תיאור סביבת הפיתוח

הפרויקט נכתב בvisual studio code במחשב windows

הספריות שבהן השתמשתי בפרויקט :

צד שרת :

time

select

socket

threading

uuid

sqlite3

os

crypto

ssl

random

json

zlib

smtplib

base64

pathlib

צד לקוח :

customtkinter

threading

time

tempfile

os

ssl

zlib

json

base64

pathlib

sys

socket

Enum

rsa

PIL

תיאור פרוטוקול התקשורת

פרוטוקול התקשורת בתוכנית הינו פרוטוקול שהומצא לה שמתבסס על פרוטוקול HTTP.

הפרוטוקול BOOP (the Backup Ordering and Operating Protocol) הוא מעל שכבת TCP בעל מבנה קצת שונה בהודעות בין הלקוח לשרת. הבסיס זהה, בכול הודעה יש start, content, header. אולם התוכן והכותרות שבהן שונות תלוי במי שולח את ההודעה.

מבנה ההודעות מהלקוח לשרת:

Start – בחלק הראשון ביותר של ההודעה יהיו שני דברים. את השם של הפרוטוקול BOOP ואת שם הבקשה שהלקוח עושה. ישנן שמונה סוגי בקשות שהלקוח יכול לבקש מהשרת שעליהן ארחיב בהמשך.

Header – חלק זה יכיל את השדות הבאים:

- content type – מה הוא סוג התוכן בהודעה, האם זה טקסט או קובץ וכו.
- content length – מה הוא אורך התוכן שההודעה מעבירה.
- Username – מה הוא שם המשתמש של הלקוח ששלח את הבקשה.
- bucket id – מה הוא מספר הזהות של דלי המידע של המשתמש.

Content – יכיל את תוכן ההודעה.

מבנה ההודעות מהשרת ללקוח:

Start – בחלק הראשון ביותר של ההודעה יהיו שני דברים. את השם של הפרוטוקול BOOP ואת תוצאת הבקשה של המשתמש, כלומר את הקוד המספרי המתאים (הקודים הם הקודים של HTTP) למשל 200 אם הכול עבד כמו שצריך.

Header – חלק זה יכיל את השדות הבאים:

- content type – מה הוא סוג התוכן בהודעה, האם זה טקסט או קובץ וכו.
- content length – מה הוא אורך התוכן שההודעה מעבירה.
- server hash

Content – יכול את תוכן ההודעה.

סוגי הבקשות האפשריים ללקוח :

1. ACCESS – בקשה של המשתמש לקבל גישה למידע של הדלי. תגובת השרת תהיה התחלת תהליך הקונפורמציה שבאמצעותו השרת יזהה את המשתמש (יהיה בטוח שזה אכן המשתמש שמקבל את המידע)
2. ACCESS_ANS – הלקוח ישתמש בבקשה זו כדי לשלוח את התשובות של המשתמש לשאלות הזיהוי של השרת.
3. DOWNLOAD – הבקשה של המשתמש להוריד אל המחשב שלו גיבוי מסוים מהדלי שלו.
4. GETBUCKET – בקשה לקבל את המידע על הדלי מהשרת. הלקוח יציג את המידע הזה למשתמש בחלון 'open_backup'
5. GETBACKUP – בקשה לקבל את המידע על גיבוי מסוים שבדלי מהשרת. הלקוח יציג את המידע הזה למשתמש בחלון 'open_file'
6. GETFILE – בקשה לקבל את המידע של קובץ מסוים אחד מגיבוי מסוים אחד בדלי להציג למשתמש.
7. SIGNUP – הבקשה הראשונה שתקרה כשהמשתמש יפעיל את התוכנית בפעם הראשונה. בקשה זו תכיל את כול המידע של המשתמש, כשהשרת יקבל אותה הוא יוסיף את המידע של המשתמש למאגר המידע. זו תהיה הבקשה היחידה בה השדות Username ו-bucket id ישארו ריקים.
8. UPLOAD – בקשה של המשתמש להעלות את המידע על המחשב שלו לענן, כלומר לגבות את המידע הנוכחי שלו.

תיאור מסכי המערכת

חלון הבית

בחלון הבית יש את הדברים הבאים:

את הלוגו כתמונה, ושלושה כפתורים.

הכפתורים הם:

Backup current data – שפותח את create backup window ;

View backup – שפותח את open backup window

אם המשתמש לא רשום, חלון הבית יפתח את חלון ההרשמה והמידע של המשתמש יכנס לDB.



חלון ההרשמה

בחלון ההרשמה המשתמש יצטרך להכניס את השדות

הבאים:

שם המשתמש – שם משתמש שזהה לשם משתמש שכבר

שמור בDB לא יתקבל והתוכנה תחזיר למשתמש שגיאה.

כתובת אימייל – התוכנה תדרוש כתובת אימייל למען

האימות הדו שלבי.

שאלות ותשובות - חלון ההרשמה יבקש מספר שאלות ואת

התשובות להן כדי להשתמש בהן לוודא את זהות המשתמש

בהמשך באימות הדו שלבי.

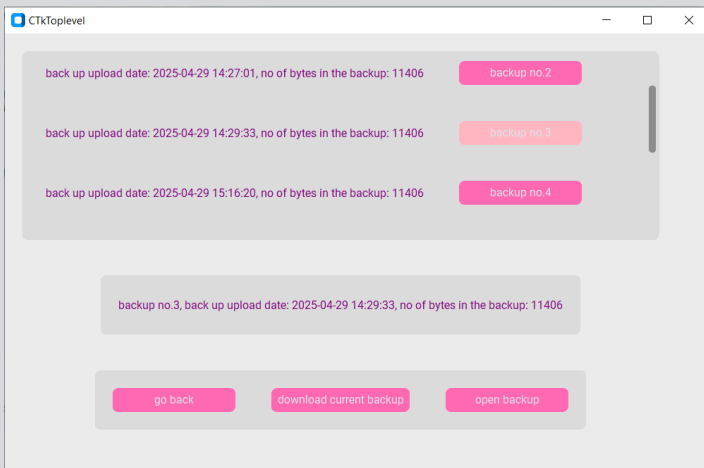
יהיה גם את כפתור ההרשמה שישלח את תשובות המשתמש

ככה שלא ישלח מידע חסר לפני שהמשתמש סיים להקליד.

חלון פתיחת הגיבוי

חלון זה נפתח אם המשתמש לוחץ על כפתור View backup בחלון הבית.

בחלון זה ניתן לראות רשימה של הגיבויים ואת המידע עליהם כמו שהם מוצגים בחלון הבית, אם המשתמש לוחץ על אחד הגיבויים לחיצה כפולה המידע של אותו הגיבוי נפתח במסגרת התחתונה, אם המשתמש לוחץ על כפתור פתיחת הגיבוי המסוים שהמידע שלו נמצא במסגרת התחתונה נפתח בחלון פתיחת הקובץ.



יש גם כפתור חזור שמחזיר את המשתמש אל חלון הבית.

וכפתור Download current backup - שפותח את downloading backup window (חלון הורדת הגיבוי);

חלון פתיחת הקובץ

בחלון זה מוצג המידע השמור בגיבויי שהמשתמש בחר בחלון פתיחת הגיבוי. באמצעות חלון זה המשתמש יכול להסתכל על המידע המגובה שלו.

בחלון זה שתי מסגרות

המסגרת העליונה – מוצגים בה הקבצים והתיקיות שבגיבויי כמו file explorer.

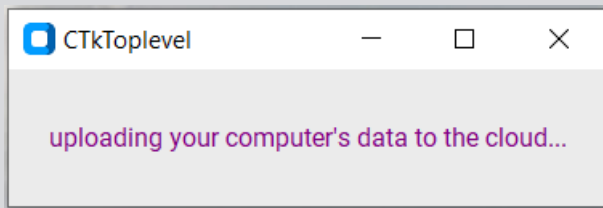
המסגרת התחתונה – אם המשתמש לוחץ פעמיים על קובץ מסוים במסגרת העליונה המידע שלו מופיע במסגרת התחתונה.

בחלון זה שני כפתורים

כפתור פתיחת קובץ – אם המשתמש לוחץ על כפתור זה, הקובץ שנמצא במסגרת התחתונה יפתח לצפייה.

כפתור חזור – מחזיר את המשתמש לחלון פתיחת הגיבוי.

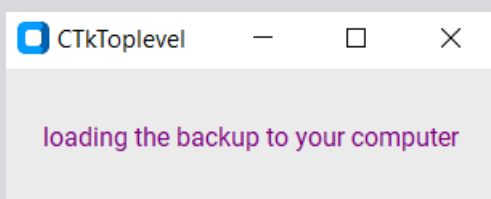
חלון יצירת גיבוי



חלון זה נפתח אם המשתמש לוחץ על כפתור Backup current data במסך הבית. אחרי שכפתור זה נפתח התוכנה עובדת על העברת המידע של המשתמש לענן מאחורי הקלעים. המשתמש יראה מסר טקסט.

כשההורדה תסתיים החלון הזה יסגר אוטומטית המשתמש יחזור למסך הבית.

חלון הורדת גיבוי



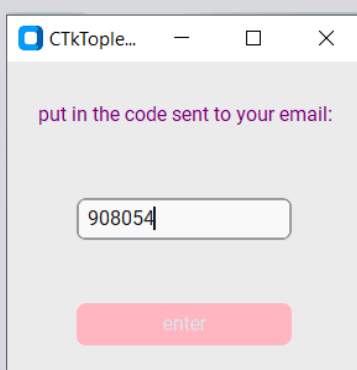
חלון זה נפתח אם המשתמש לוחץ על כפתור Download current backup במסך הבית. כשזה יקרה, השרת יוריד למחשב המשתמש את הגיבוי שנבחר בחלון פתיחת הגיבוי. המשתמש יראה את החלון הזה שיש בו לייבל.



חלונות אימות זהות המשתמש

בפרויקט יהיה אימות דו שלבי שייפתח ויבקש מהמשתמש להזדהות. אימות זה ידרש עם לחיצה על כפתור 'view data' שני שלבי האימות הם:

ידע – המשתמש יתבקש להזין את אחת התשובות לשאלות שהוא הזין עם יצירת המשתמש.



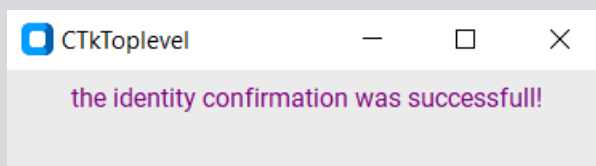
משהו של המשתמש – תשלח הודעת אימייל לכתובת האימייל שהמשתמש הזין עם יצירת המשתמש. מה שישלח יהיה סיסמא שהמשתמש יזין בחלון האימות כדי להוכיח שיש לו את הטלפון של המשתמש כלומר, שהוא המשתמש.

בשני חלונות אלו יהיה entry בה המשתמש יכניס את תשובותיו וכפתור enter שישלח את תשובות המשתמש.

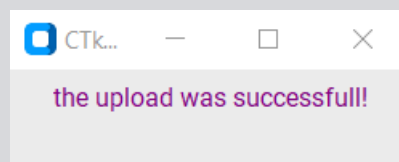
חלונות הצלחה

כשתהליך מצליח, יפתח חלון הצלחה. שאומר למשתמש שמה שהוא ניסה לעשות הצליח. חלונות אלו יפתחו לתהליכים שאינם פתיחת הדלי, הגיבוי או הקובץ, שם ניתן לראות את תגובת התוכנה והצלחתה. חלונות ההצלחה יהיו להעלאת גיבוי, הורדת גיבוי הירשמות למערכת ואימות זהות המשתמש.

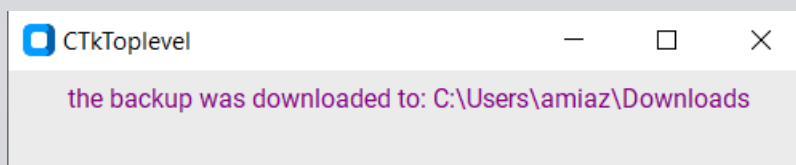
חלון הצלחת אימות זהות המשתמש :



חלון הצלחת העלאת הגיבוי לשרת :

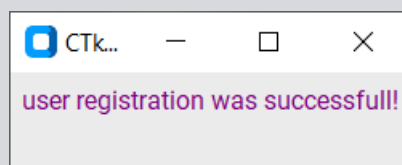


חלון הצלחת הורדת הגיבוי למחשב המשתמש :



חלון זה גם כותב את הנתיב במחשב אליו התוכנית הורידה את הגיבוי.

חלון הצלחת הרשמת המשתמש :



תיאור מבני הנתונים

- מסד נתונים, במאגר הנתונים של הפרויקט – DATA BASE יהיו את הטבלאות הבאות:
 - user_info – טבלה זו תכיל את כול המידע שהתוכנה שומרת על המשתמשים. הטורים בה יהיו:
 - Username – טקסט. שם המשתמש של הלקוח, זה יהיה המפתח הראשי של הטבלה והמערכת לא תאפשר שיהיו שני משתמשים עם אותו שם משתמש. מפתח זר לטבלה answers_to_questions
 - bucket_id – טקסט. מספר הזיהוי של דלי המידע של המשתמש, דלי המידע יהיה המקום במחשב השרת שבו גיבויי המשתמש ישמרו. לכול משתמש יהיה דלי אחד, מספר הזיהוי יוגרל בצורה רנדומלית וישמש כהשם של מקום האיחסון ב"ענן". גם מספר הזיהוי של הדלי יהיה חייב להיות יחודי. זהו גם מפתח זר לטבלה bucket_info
 - Email_address – טקסט. כתובת האימייל של המשתמש לזיהוי המשתמש באמצעות שליחת קוד בהודעת אימייל.
 - User_key – טקסט, המפתח להצפנה של קבצי המשתמש.
 - User_nonce – טקסט, החצי השני של המפתח. כדי לייצר טיפוס מסוג crypto.cipher.AES ולוודא שההצפנה שלו תהיה זהה לפיענוח יש לתת את משתנה זה.
 - conformation_question1 – טקסט. שלושת המשתנים הבאים משמשים אותה מטרה, אם הירשמות למערכת המשתמש יכניס שלוש שאלות ושלוש תשובות, תשובה לכול שאלה. כול פעם שהשרת ירצה לזהות את המשתמש הוא יבחר אחת מהשאלות בצורה רנדומלית ויבקש מהמשתמש לענות על אחת מהן. אלו גם מפתחות זרים לטבלה
 - conformation_question2
 - conformation_question3
 - bucket_info – טבלה זו תכיל את המידע על הדליים בהם הגיבויים של המשתמשים שמורים. הטורים בה יהיו:
 - bucket_id – טקסט. מספר הזיהוי של דלי המידע של המשתמש, דלי המידע יהיה המקום במחשב השרת שבו גיבויי המשתמש ישמרו. לכול משתמש יהיה דלי אחד, מספר הזיהוי יוגרל בצורה רנדומלית וישמש כהשם של מקום האיחסון ב"ענן". גם מספר הזיהוי של הדלי יהיה חייב להיות יחודי. בטבלה זו מספר הזיהוי של הדלי יהיה המפתח הראשי. זהו גם מפתח זר לטבלה user_info.
 - last_update – תאריך הפעם האחרונה שהדלי התעדכן.
 - backup_num – מספר. מספר הגיבויים השמורים בדלי.
 - bytes_in_bucket – מספר. כמות הזיכרון שהדלי מנצל.

Backup_info – טבלה שמכילה את נתונים כול גיבוייהם של המשתמשים. הטורים השמורים בה יהיו :

- bucket_id – טקסט, אחד משני מפתחות ראשיים. זהו מספר הזיהוי של הדלי בו שמור הגיבוי. בנוסף הוא גם מפתח זר לbucket_info.
- serial_num – אינטגר, המפתח הראשי השני. זה המספר המזהה של הגיבוי בתוך הדלי שגם מראה כמה גיבויים נוצרו לפניו.
- upload_date – סוג DATE, התאריך (כולל שעה דקה ושנייה) בו נוצר הגיבוי.
- bytes_in_backup – אינטגר, מספר הבתים בגיבוי.

answers_to_questions – בתהליך הרישום המשתמש מכניס תשובות ושאלות שמטרתן לעזור ולזהות אותו בהמשך. בטבלה זו התשובות והשאלות ירשמו וישלפו בתהליך הזיהוי. הטורים בטבלה יהיו :

- Username – טקסט. שם המשתמש של הלקוח, זה יהיה המפתח הראשי של הטבלה והמערכת לא תאפשר שיהיו שני משתמשים עם אותו שם משתמש. מפתח זר לטבלה user_info
- conformation_question1 – טקסט. שלושת המשתנים הבאים משמשים אותה מטרה, אם הירשמות למערכת המשתמש יכניס שלוש שאלות ושלוש תשובות, תשובה לכול שאלה. כול פעם שהשרת ירצה לזהות את המשתמש הוא יבחר אחת מהשאלות בצורה רנדומלית ויבקש מהמשתמש לענות על אחת מהן. אלו גם מפתחות זרים לטבלה answers_to_questions
- conformation_question2
- conformation_question3
- conformation_answer1 – טקסט. אלו יהיו התשובות לשאלות הקונפרמציה. conformation_answer1 תהיה התשובה לconformation_question1 וכך הלאה בהתאמה.
- conformation_answer2
- conformation_answer3

סקירת חולשות ואיומים

שכבת האפליקציה:

○ מסד הנתונים

במסד הנתונים ישנן שתי בעיות אבטחה גדולות שצריך לפתור. דבר ראשון יש להצפין את מסד הנתונים על מנת למנוע מתוקפים לגשת למידע על המשתמשים. לא הצפנתי את מסד הנתונים מכיוון שלא מצאתי סיפרייה ואו תוכנה מתאימה לכך בחינם, ובנוסף לא עשיתי את זה בקוד מכיוון שזה היה צורך קובץ נוסף שיחזיק מידע על ההצפנה – מה שיוביל לפרצה דומה לפרצה המקורית – או לשמור את כול דברי ההצפנה בקוד מה שהיה גורם לאיבוד הנתונים בכול איתחול מחדש של השרת.

דבר נוסף שצריך להגן מפניו בנוגע לעבודה עם מסד נתונים הוא sql injection, כלומר, הסכנה שהמשתמש יכניס קוד שמטרתו להשיג מידע ממסד הנתונים. הדרך שבה התמודדתי אם סכנה זו היא באמצעות escaping. הדרך בה מימשתי זאת היא שבכול מקום בו התוכנית משתמשת במידע מהמשתמש לשאילתה היא קודם תיצור טאפל בו יהיו הערכים הנחוצים לשאילתה לפי הסדר בו הם נחוצים בשאילתה, בשאילתה עצמה יהיה כתוב '!' בכול מקום בו פרמטר אמור להופיע. הפעולה execute של הספרייה sqlite3 תקבל שני ערכים. השאילתה ואת הטאפל בו נמצאים הערכים שהשאילתה צריכה.

○ אימות זהות המשתמש

אימות הזהות של המשתמש קורה בשתי דרכים.

הדרך הראשונה, היא שללקוח יש שלושה ערכים שהוא מביא לשרת בכול בקשה. את שם המשתמש, את מספר הזהות של הדלי ואת העוגייה. כשהשרת מקבל את הבקשה הוא בודק את הערכים האלו. אם העוגייה לא נכונה הוא יבקש מהמשתמש אימות זהות ותהליך הקונפורמציה יתחיל. אם מספר זהות הדלי ושם המשתמש לא נכונים התוכנית תבקש מהמשתמש להירשם מחדש.

הסיבה לכך היא שמספר הזהות של הדלי הוא דבר שניתן למשתמש מהשרת אם הרשמתו לפרויקט. שלא כמו שם המשתמש שהוא הכניס בעצמו. לכן אם מספר זהות הדלי לא נכון (או לא מתאים לשם המשתמש) התוכנית תניח שהלקוח שפונה אליה הוא לא אותו המשתמש או משהו שמנסה להתחזות למשתמש ותכריח אותו להתחבר מחדש.

לעומת זאת, העוגייה נועדה לסמן session ולהיות משהו זמני. כשהמשתמש סוגר את התוכנה session נגמר ויתחיל מחדש אם פתיחת התוכנה מחדש. זמניות העוגיות לא מומשה בתוכנה אבל כול הקוד הבנוי מסביב לעוגיות נכתב כבסיס לזה.

תהליך אימות זהות המשתמש (הקונפורמציה) הוא בעל שני שלבים. בשלב הראשון התוכנית תציג ללקוח שאלה שהוא הכניס בעצמו, על הלקוח להכניס את התשובה לשאלה זו שהוא הכניס בעת הרשמתו לפרויקט.

לאחר מכן המשתמש יצטרך להכניס קוד שישלח אליו באימייל.

○ שמירת קבצי המשתמש

תפקידה העיקרי של התוכנית הוא לשמור על קבצי המשתמש בענן מאובטח ובצורה מסודרת, לכן קבצי המשתמש שמורים בענן בצורה מוצפנת. כשהמשתמש מתחבר לראשונה לתוכנית, שניים מהדברים שהשרת שומר בטבלת `user_info` יחד עם שאר המידע של המשתמש הם המשתנים `user_key` ו-`user_nonce` שני המשתמשים הללו מכילים את המידע הנחוץ כדי ליצור טיפוס מסוג `crypto.cipher.AES` שבאמצעות התוכנית מצפינה את המידע של הקבצים של המשתמש וכשהמשתמש רוצה את להוריד את הגיבוי או לצפות בקובץ התוכנית תפענח את המידע ותשלח ללקוח אותו. לא ניתן לפענח את המידע של קבצי המשתמש בלי גישה למסד הנתונים. לכול משתמש יש `user_key` ו-`user_nonce` ייחודי. ככה גם אם נחשף המידע של משתמש אחד האחרים מוגנים.

הצפנת AES הינה הצפנה מסוג צופן בלוקים. כלומר, הוא לוקח את המידע להצפנה ומחלק אותו לבלוקים בגודל מסוים שאותם הוא מצפין. גודל הבלוקים ב-AES הוא 16 בתים, אותם התוכנה מסדרת במטריצה של ארבע על ארבע ועליה היא עושה את פעולות ההצפנה. גודל המפתח קובע כמה "סבבים" ההצפנה תעשה, כלומר כמה פעמים היא תחזור על הפעולות שהיא עושה להצפנה. שלבי ההצפנה בכול סבב הם:

1. יצירת המפתח של הסבב, מפתח הסבב נוצר על בסיס המפתח הראשי (בתוכנית זו המפתח הראשי הוא מש שמור כ-`user_key` במסד הנתונים) באמצעות ה-`AES key schedule`.
2. כול אחד מה 16 בתים עובר XOR על אחד מבתי מפתח הסבב (כלומר עושים [בית מהבלוק] XOR [בית ממפתח הסבב]).
3. מחליפים את המיקום של כמה בתים נבחרים.
4. מחליפים את מקומן של שלוש שורות במעגליות.
5. מערבבים ומחברים בין הבתים של כול עמודה.
6. חוזרים על שלבים 2-5 בכול סבב אך בסבב האחרון מבצעים את השלבים 3, 4, 2 בסדר הזה.

שכבת התעבורה:

שכבת התעבורה של הפרויקט מוגנת באמצעות SSL. לשרת יש שני קבצי PEM באמצעותם הוא עוטר את ה-`socket` שלו וה-`socket` של המשתמש. מה שמאפשר לתקשורת בטוחה ביניהם.

מימוש הפרויקט

חלק א

מחלקות שלי :

Client_comm

קובץ בו המחלקה : copyCatClient_comm.py

תפקיד המחלקה : מחלקה זו מנהלת את התקשורת בצד הלקוח

תכונות המחלקה :

- Incoming_msg – סוג מחרוזת, במשתנה זה נשמרת התגובה הכי עדכנית של השרת לבקשת הלקוח. השימוש של משתנה זה הוא שממנו נקראת תגובת השרת בחלקים אחרים של הקוד שלהם אין גישה לsocket.
- Outgoing_msg – סוג מחרוזת. במשתנה זה נשמרת הבקשה הכי עדכנית של המשתמש לשרת. השימוש של משתנה זה הוא שבו כבחלקים אחרים של הקוד שלהם אין גישה לsocket כותבים את הבקשה לשרת.
- Lock – מסוג Threading.lock נועל את המשתנים Incoming_msg וOutgoing_msg מכיוון שיש כמה תהליכים שניגשים אליהם. משתנה זה מונע בעיות כמו למשל שתהליך אחד ינסה לקרוא את ערך המשתנה בזמן שתהליך אחר משנה אותו.
- Client_socket – מסוג socket.socket הוא החיבור לשרת. ממנו אפשר לקבל את תגובת השרת ולשלוח את בקשות המשתמש.

פעולות במחלקה :

- Send – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : שולח את Outgoing_msg, מקבל את תגובת השרת ושם אותה Incoming_msg ומוודא שהוא קיבל את כול תגובת השרת ולא פיספס משהו. הפעולה לא מחזירה ערך.
- Set_outgoing_msg – טענת כניסה : message, ערך זה שווה להודעה שהמשתמש רוצה לשלוח לשרת. טענת יציאה : הפונקציה שמה את הערך שנתנו לה בOutgoing_msg. הפונקציה לא מחזירה ערך.
- Get_outgoing_msg – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה מחזירה את המשתנה Outgoing_msg.
- Reset_incoming – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה הופכת את הערך של המשתנה Incoming_msg ל"". הפעולה לא מחזירה ערך.
- Get_incoming_msg – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה מחזירה את המשתנה Incoming_msg.
- Disconnect – טענת כניסה : הפעולה מקבלת רק את self. הפעולה מנתקת את החיבור של Client_socket וסוגרת אותו.
- Receive – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה מקבלת הודעה מהשרת ושמה את הערך Incoming_msg.

Server_comm

קובץ בו המחלקה : copyCatServer_comm.py

תפקיד המחלקה :

תכונות המחלקה :

- Incoming_msgs – סוג רשימה של טאפלים. בכול טאפל יש מחרוזת של בקשה מלקוח וטיפוס מסוג socket.socket, שהוא הסוקט של הלקוח ששלח את הבקשה. מהרשימה הזאת השרת יקרא את הבקשה ויענה לה. בנוסף לכך, עם כול לקוח שמצטרף ושולח בקשה, תתווסף בקשה לרשימה. כול בקשה שנענית על ידי השרת תורד מהרשימה.
- Outgoing_msgs – סוג רשימה של טאפל. בכול טאפל יש מחרוזת של תגובה לבקשת לקוח וטיפוס מסוג socket.socket, שהוא הסוקט של הלקוח ששלח את הבקשה. מכאן ישלחו התגובות באמצעות תהליך שתפקידו לשלוח כול תגובה שנמצאת במשתנה זה. כשתגובה מרשימה זו נשלחת, מוציאים אותה מהרשימה.
- Lock - מסוג Threading.lock נועל את המשתנים Incoming_msgs וOutgoing_msgs מכיוון שיש כמה תהליכים שניגשים אליהם. משתנה זה מונע בעיות כמו למשל שתהליך אחד ינסה לקרוא את ערך המשתנה בזמן שתהליך אחר משנה אותו.
- Server - מסוג socket.socket הוא הסוקט של השרת. אליו הלקוחות יתחברו.
- Clients – מסוג רשימה של socket.socket. כול פעם שלקוח מתחבר לשרת הוא מוסף לרשימה זאת.
- Messages – סוג רשימה של טאפלים. בכול טאפל יש מחרוזת של בקשה מלקוח וטיפוס מסוג socket.socket, שהוא הסוקט של הלקוח ששלח את הבקשה. לפני שהבקשה מוספת לIncoming_msgs היא נמצאת במשתנה זה. שאוסף את הדברים השונים שנשלחים מכול הלקוחות המחוברים.

פעולות במחלקה :

- Comms – טענת כניסה : הפעולה מקבלת רק את הself. טענת יציאה : הפעולה מבצעת לולאה אינסופית בה היא מקבלת לקוחות, אוספת מהם מידע, מוודאת שהיא מקבלת כול בקשה במלואה, ומוסיפה כול בקשה שמתווספת לIncoming_msgs. הפונקציה לא מחזירה ערך.
- Add_response – טענת כניסה : client_socket מסוג socket.socket, request מסוג מחרוזת וresponse מסוג מחרוזת. טענת יציאה : הפונקציה תמצא את הטאפל (client_socket, request) בתוך Incoming_msgs ותוציא אותו. לאחר מכן היא תוסיף את הטאפל (client_socket, response) לOutgoing_msgs. הפונקציה לא מחזירה ערך.
- Send_message – טענת כניסה : client_socket מסוג socket.socket, response מסוג מחרוזת. טענת יציאה : הפונקציה מחפשת את הטאפל (client_socket, response) בתוך Outgoing_msgs. אם היא מוצאת אותו, היא שולחת את התגובה בטאפל ומסירה את הטאפל מהרשימה. הפונקציה לא מחזירה ערך.
- Send_messages – טענת כניסה : הפעולה מקבלת רק את הself. טענת יציאה : עוברת על כול הרשימה Outgoing_msgs שולחת כול תגובה ומורידה את הטאפל שהכיל אותה מהרשימה. הפונקציה לא מחזירה ערך.

- Remove_from_incoming – טענת כניסה : client_socket מסוג socket.socket , request מסוג מחרוזת. טענת יציאה : מוצאת את הטאפל (client_socket,request) ברשימה Incoming_msgs ומוציאה אותו. הפונקציה לא מחזירה ערך.

Handle_backups

קובץ בו המחלקה : copyCatGUI.py

תפקיד המחלקה : מחלקה זו מנהלת את הקבצים בצד לקוח. היא יוצרת את המילון שישלח לשרת כדי ליצור גיבויי והיא האחת שתוריד את הגיבויים למחשב המשתמש.

תכונות המחלקה : אין

פעולות במחלקה :

- Download_backup_to_user – טענת כניסה : backup_data מסוג מילון. טענת יציאה : הפונקציה תיקח את המילון שהיא מקבלת, תרוץ עליו וממנו תוריד את הגיבוי למחשב המשתמש. הפונקציה לא מחזירה ערך.
- Make_file_list – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפונקציה רצה על רשימת התיקיות לגיבוי, כול לולאה היא קוראת לMake_file_list2. הפונקציה מחזירה רשימה של מחרוזות.
- Make_file_list2 – טענת כניסה : current_path מסוג מחרוזת וfile_list מסוג רשימה של מחרוזות. טענת יציאה : פעולה זו היא פעולה רקורסיבית של עצים. פעולה זו עוברת על כול הקבצים, מוסיפה אותם לרשימה וקוראת לעצמה כדי להיכנס לתיקיות שבתוך תיקיות. הפונקציה מחזירה רשימה של מחרוזות.
- Compress_the_computer_data – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפונקציה הזאת קוראת לMake_file_list ומקבלת רשימה של כול הקבצים בתיקיות לגיבוי, היא רצה על הרשימה ויוצרת מילון שבו המפתח זה הנתבי לקובץ והערך הוא מידע הקובץ דחוס ומקודד. הפונקציה מחזירה מילון.

Handle_storage

קובץ בו המחלקה : copyCatHandle.py

תפקיד המחלקה : מחלקה זו מנהלת את שטח האחסון של השרת.

תכונות המחלקה : אין

פעולות במחלקה :

- Storage_exists – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפונקציה בודקת אם התיקייה של שטח האחסון קיימת. אם היא קיימת הפונקציה תחזיר True ואם לא היא תחזיר False.
- Get_backup_file_list – טענת כניסה : bucket_id מסוג אינטגר ו backup_num מסוג אינטגר. טענת יציאה : הפונקציה עוברת על כול הקבצים בתיקיית הגיבוי שנתון לה בדלי הנתון לה. היא מוסיפה את הקבצים לרשימה וקוראת לFiles_in_folder. הפונקציה מחזירה רשימה של מחרוזות.
- Files_in_folder – טענת כניסה : folders מסוג רשימה של מחרוזות, directory מסוג מחרוזת וfile_list מסוג רשימה של מחרוזות. טענת יציאה : פעולה זו היא רקורסיבית. היא

- מוסיפה את כול הקבצים בתיקייה הנוכחית לרשימה, ואז קוראת לעצמה ונכנסת לתיקייה שהייתה בתוך התיקייה וכו הלאה. הפונקציה מחזירה רשימה של מחרוזות.
- `Get_backup` - טענת כניסה: `bucket_id` מסוג אינטגר ו `backup_num` מסוג אינטגר. טענת יציאה: הפונקציה קוראת ל `Get_backup_file_list` ומקבלת ממנה את רשימת כול הקבצים בגיבוי. לאחר מכן היא רצה על הרשימה ומייצרת מילון בו המפתחות הם הנתבים לקבצים והערכים הביטים המקודדים והדחוסים של הגיבוי. הפונקציה מחזירה מילון.
 - `Get_bytes_in_bucket` - טענת כניסה: טענת כניסה: `bucket_id` מסוג אינטגר. טענת יציאה: הפונקציה רצה על רשימה של כול הגיבויים בדלי, לכול גיבויי הפונקציה קוראת ל `Get_bytes_in_backup` וסוכמת את המספר לתוך סכום הבתים של הדלי. הפונקציה מחזירה איטגר.
 - `Get_bytes_in_backup` - טענת כניסה: `bucket_id` מסוג אינטגר ו `backup_num` מסוג אינטגר. טענת יציאה: הפעולה סופרת את מספר הבתים בכול קובץ בתיקייה הראשית של הגיבוי (באמצעות `Get_bytes_in_file`) וסופרת את מספר הבתים בתיקיות שבתוך התיקייה הראשית באמצעות קריאה ל `Bytes_in_folder`. הפונקציה מחזירה איטגר.
 - `Bytes_in_folder` - טענת כניסה: `folders` מסוג רשימה של מחרוזות, `directory` מסוג מחרוזת ו `bytes_num` מסוג אינטגר. טענת יציאה: פעולה זו היא רקרוסיבית. היא סוכמת את מספר הבתים בכול הקבצים בתיקייה הנוכחית לרשימה, ואז קוראת לעצמה ונכנסת לתיקייה שהייתה בתוך התיקייה וכו הלאה. הפונקציה מחזירה איטגר.
 - `Get_file` - טענת כניסה: `path` מסוג מחרוזת. טענת יציאה: הפונקציה פותחת את הקובץ, קוראת את התוכן שלו, מקודדת את התוכן ומחזירה את התוכן המקודד.
 - `Get_bytes_in_file` - טענת כניסה: `path` מסוג מחרוזת. טענת יציאה: הפונקציה סופרת את כמות הבתים בפונקציה ומחזירה את המספר.
 - `Create_user_bucket` - טענת כניסה: `bucket_id` מסוג אינטגר. טענת יציאה: הפונקציה יוצרת תיקייה בשטח האחסון של השרת ששמה הוא המספר שהיא קיבלה. הפונקציה לא מחזירה ערך.
 - `Create_new_backup` - טענת כניסה: `bucket_id` מסוג אינטגר, `user_data` מסוג מילון. טענת יציאה: הפונקציה יוצרת גיבוי חדש בתוך הדלי עם המידע מהמילון. הפונקציה יוצרת את הקבצים בתוך הגיבוי באמצעות `Create_file_in_backup`. בנוסף לכך היא גם קוראת ל `new_backup_update` כדי לעדכן את מסד הנתונים. הפונקציה לא מחזירה ערך.
 - `Create_file_in_backup` - טענת כניסה: `root` מסוג מחרוזת, `file_data` מסוג בתים. טענת יציאה: הפונקציה כותבת לתוך הנתב שהיא קיבלה את המידע שהיא קיבלה. הפונקציה לא מחזירה ערך.

Handle_DB

קובץ בו המחלקה: `copyCatHandle.py`

תפקיד המחלקה: מנהלת את מסד הנתונים של התוכנית.

תכונות המחלקה:

- `Conn` – `database connection` באמצעותו אפשר לעשות פעולות על מסד הנתונים.

פעולות במחלקה:

- Init – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : בנוסף להכנת האובייקט פעולה זו גם מכינה את כול הטבלאות של מסד הנתונים.
- DB_exists - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפונקציה בודקת אם הקובץ של מסד הנתונים קיים. אם הוא קיים הפונקציה תחזיר True ואם לא היא תחזיר False.
- Sign_up_client – טענת כניסה : user_info רשימה של מחרוזות. טענת יציאה : מכניסה את הנתונים של המשתמש החדש לשורה חדשה בטבלאות user_info ו answers_to_questions. בנוסף לכך הפונקציה מגרילה ויוצרת מספר דלי למשתמש וקוראת ל Create_user_bucket, שיוצר דלי חדש למשתמש, לאחר מכן הפונקציה מוסיפה את המידע החדש של הדלי החדש לטבלה bucket_info. הפונקציה מחזירה את ערך מספר הזהות של הדלי.
- Check_username_bucketID - טענת כניסה : username מסוג מחרוזות ו bucket_id מסוג מחרוזות. טענת יציאה : הפונקציה בודקת בשאילתה בטבלה user_info האם קיים משתמש שאלו שם המשתמש והדלי שלו. היא תחזיר True אם כן ואם לא היא תחזיר False.
- Check_user_answer_in_DB - טענת כניסה : username מסוג מחרוזות ו user_answer מסוג מחרוזות. טענת יציאה : הפונקציה מבעת שאילתה בטבלה answers_to_questions ובודקת האם התשובה של המשתמש קיימת בין התשובות ששמורות לשם משתמש שלו. היא תחזיר True אם כן ואם לא היא תחזיר False.
- Get_user_backup_num - טענת כניסה : bucket_id מסוג מחרוזות. טענת יציאה : הפונקציה מבצעת שאילתה בטבלה bucket_info ומחזירה את מספר הגיבויים בדלי שניתן לה.
- Get_user_email - טענת כניסה : username מסוג מחרוזות. טענת יציאה : הפונקציה מבצעת שאילתה בטבלה user_info ומחזירה את כתובת האימייל של המשתמש ששם המשתמש שלו ניתן לה.
- Get_user_nonce – טענת כניסה : username מסוג מחרוזות. טענת יציאה : שולף את ה nonce מטבלת user_info. ומהמיר אותו לבתים באמצעות Base64. יחזיר את ה nonce כבתים.
- Get_user_key – טענת כניסה : username מסוג מחרוזות. טענת יציאה : שולף את ה מפתח של המשתמש מטבלת user_info. ומהמיר אותו לבתים באמצעות Base64. יחזיר את המפתח כבתים.
- Get_a_users_conQuestion - טענת כניסה : username מסוג מחרוזות ו question_num מסוג אינטגר. טענת יציאה : הפונקציה מבצעת שאילתה בטבלה answers_to_questions ומחזירה אחת מהשאלות לפי איזה מספר קיבלה הפונקציה.
- Get_backup_info - טענת כניסה : bucket_id מסוג מחרוזות. טענת יציאה : הפונקציה מבצעת שאילתה בטבלה bucket_info ומחזירה את כול המידע השמור על אודות דלי זה.
- Set_user_backup_num - טענת כניסה : bucket_id מסוג מחרוזות ו backup_num מסוג אינטגר. טענת יציאה : הפונקציה מעדכנת את טבלה bucket_info ושמה בעמודה

backup_num של השורה של דלי המסוים את מספר הגיבויים שהיא קיבלה. הפונקציה לא מחזירה ערך.

- Set_lateset_update - טענת כניסה : bucket_id מסוג מחרוזת. טענת יציאה : הפונקציה מעדכנת את טבלה bucket_info ושמה בעמודה last_update של השורה של דלי המסוים את הזמן הנוכחי. הפונקציה לא מחזירה ערך.
- Set_bytes_in_bucket - טענת כניסה : bucket_id מסוג מחרוזת. טענת יציאה : הפונקציה מעדכנת את טבלה bucket_info ושמה בעמודה bytes_in_bucket של השורה של דלי המסוים את כמות הבתים בדלי לפי הפונקציה Get_bytes_in_bucket. הפונקציה לא מחזירה ערך.
- Set_bytes_in_backup - טענת כניסה : bucket_id מסוג מחרוזת ו backup_num מסוג אינטגר. טענת יציאה : הפונקציה מעדכנת את טבלה backup_info ושמה בעמודה bytes_in_backup של השורה של דלי המסוים את כמות הבתים בדלי לפי הפונקציה Get_bytes_in_backup. הפונקציה לא מחזירה ערך.
- Set_new_backup - טענת כניסה : bucket_id מסוג מחרוזת ו backup_num מסוג אינטגר. טענת יציאה : מכניס לתוך הטבלה backup_info שורה לגיבוי חדש. הפונקציה לא מחזירה ערך.
- New_backup_update - טענת כניסה : bucket_id מסוג מחרוזת ו backup_num מסוג אינטגר. טענת יציאה : מעדכנת את טבלאות bucket_info ו backup_info בעקבות גיבוי חדש. קוראת לפונקציות Set_lateset_update, Set_user_backup_num, Set_bytes_in_bucket ו Set_new_backup. הפונקציה לא מחזירה ערך.

Handle_security

קובץ בו המחלקה : copyCatHandle.py

תפקיד המחלקה : לנהל את כול החלקים השונים בקוד שקשורים לאבטחה. תכונות המחלקה :

Client_cookies – מילון שנוצר באמצעות קריאה מקובץ clientsCookies.txt. מילון שהמפתחות שלו הם שמות המשתמשים והערכים הם העוגיות שלהם.

- User_codes – מילון שהמפתחות הם שמות המשתמש והערכים הם סיסמאות שהוגרלו באקראי בשביל תהליך האימות הדו שלבי.

פעולות במחלקה :

- Create_user_key – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה :
- Encrypt_user_data - טענת כניסה : username מסוג מחרוזת ו data מסוג בתיים. טענת יציאה : שולף את key ואת noncen ממסד הנתונים באמצעות הפונקציות Get_user_key ו Get_user_nonce. לאחר מכן הוא יוצר טיפוס מסוג crypto.cipher.AES, מצפין איתו את הבתים שקיבל ויחזיר את הבתים המוצפנים.
- Decrypt_user_data - טענת כניסה : username מסוג מחרוזת ו encrypted_data מסוג בתיים. טענת יציאה : שולף את key ואת noncen ממסד הנתונים באמצעות הפונקציות Get_user_key ו Get_user_nonce. לאחר מכן הוא יוצר טיפוס מסוג

- crypto.cipher.AES, מפענח איתו את המידע המוצפן שקיבל ויחזיר את הבתים המפוענחים.
- Send_email - טענת כניסה : username מסוג מחרוזת. טענת יציאה : הפונקציה מקבלת את כתובת האימייל של המשתמש Get_user_email ואז שולחת אימייל. הפונקציה לא מחזירה ערך.
 - Create_new_cookie - טענת כניסה : username מסוג מחרוזת. טענת יציאה : הפונקציה מייצרת עוגייה חדשה, מוסיפה את העוגייה למילון Client_cookies ולקובץ clientsCookies.txt. הפונקציה מחזירה את ערך העוגייה.
 - Check_con_answers - טענת כניסה : username מסוג מחרוזת, user_answer מסוג מחרוזת, user_code מסוג מחרוזת. טענת יציאה : הפונקציה בודקת אם התשובה של המשתמש לשאלה נכונה באמצעות קריאה לפונקציה Check_user_answer_in_DB. ובודקת את הקוד מול המילון User_codes. הפונקציה תחזיר טאפל של שני ערכים בוליאנים. הערך הראשון יהיה האם התשובה לשאלה נכונה והערך השני האם הקוד היה נכון.
 - Check_cookie - טענת כניסה : username מסוג מחרוזת, cookie מסוג מחרוזת. טענת יציאה : הפונקציה בודקת אם המילון Client_cookies האם העוגייה הנתונה היא העוגייה של המשתמש. היא תחזיר True אם כן ואם לא תחזיר False.

BOOP_send

קובץ בו המחלקה : copyCatHandle.py

תפקיד המחלקה : זאת המחלקה שמייצרת הודעות לשליחה אצל השרת. ממנה שאר המחלקות שמייצרות הודעות לשליחה יורשות. תכונות המחלקה :

- Start – מחרוזת. מכילה שני דברים, שם הפרוטוקול וקוד התוצאה של בקשת המשתמש (האם הבקשה הצליחה או נכשלה)
 - Header – מחרוזת, מכיל את השדות הבאים : סוג התוכן ואורך התוכן.
 - Content – מחרוזת. יכול להכיל הרבה דברים. תלוי לחלוטין בסוג התגובה.
- פעולות במחלקה :
- Set_header – טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : מכניסה ל Header את הערכים המתאימים (מחשבת את אורך Content למשל). הפונקציה לא מחזירה ערך.
 - Send - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפונקציה לוקחת את שלושת תכונות המחלקה ומחברת ביניהן עם \r\n ביניהן. הפונקציה מחזירה את המחרוזת של תכונות המחלקה המחוברות.

BOOP_recieve

קובץ בו המחלקה : copyCatHandle.py

תפקיד המחלקה : לנתח את הבקשות שנשלחות על ידי הלקוחות ולהקל על הוצאת המידע מהן.
יש הרבה מחלקות שיורשות ממנה ומוסיפות פונקציות נוספות וספציפיות יותר לסוגי בקשות שונים.

תכונות המחלקה :

- `_start` - מחרוזת. מכילה שני דברים, שם הפרוטוקול ושם הבקשה.
 - `_header` – מחרוזת, מכיל את השדות הבאים : סוג התוכן, אורך התוכן, עוגיית המשתמש, שם המשתמש ומספר הזהות של הגיבוי.
 - `_content` - מחרוזת. יכול להכיל הרבה דברים. תלוי לחלוטין בסוג הבקשה.
- פעולות במחלקה :

- `Get_client_request_type` – טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את שם הבקשה כמחרוזת.
- `Get_content` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את המשתנה `_content`.
- `Get_header_length` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את אורך המשתנה `_header` כאינטגר.
- `Split_header` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את `_header` כרשימה שהופרדה בכול ;.
- `Get_content_type` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את ערך שדה סוג התוכן של `_header` כמחרוזת.
- `Get_content_length` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את ערך שדה אורך התוכן של `_header` כמחרוזת.
- `Get_cookie` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את ערך שדה העוגייה של `_header` כמחרוזת.
- `Get_username` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את ערך שדה שם המשתמש של `_header` כמחרוזת.
- `Get_bucket_id` - טענת כניסה : הפעולה מקבלת רק את `self`. טענת יציאה : הפעולה תחזיר את ערך שדה מספר הזהות של הדלי של `_header` כמחרוזת.

`BOOP_send`

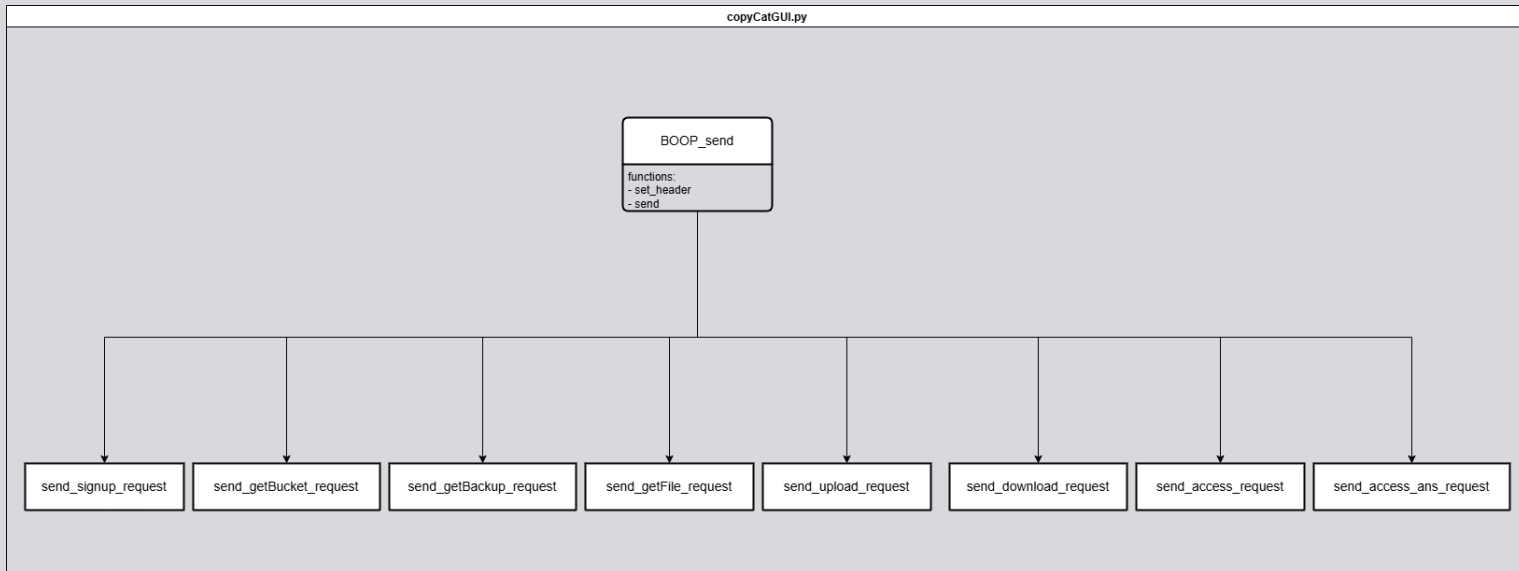
קובץ בו המחלקה : `copyCatGUI.py`

תפקיד המחלקה : זאת המחלקה שמייצרת הודעות לשליחה אצל הלקוח. ממנה שאר המחלקות שמייצרות הודעות לשליחה יורשות.

תכונות המחלקה :

- `Start` - מחרוזת. מכילה שני דברים, שם הפרוטוקול ושם הבקשה.
 - `Header` - מחרוזת, מכיל את השדות הבאים : סוג התוכן, אורך התוכן, עוגיית המשתמש, שם המשתמש ומספר הזהות של הגיבוי.
 - `Content` - מחרוזת. יכול להכיל הרבה דברים. תלוי לחלוטין בסוג הבקשה.
- פעולות במחלקה :

- Set_header - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : מכניסה ל Header את הערכים המתאימים (מחשבת את אורך Content למשל). הפונקציה לא מחזירה ערך.
- Send - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפונקציה לוקחת את שלושת תכונות המחלקה ומחברת ביניהן עם מלא ביניהן. הפונקציה מחזירה את המחרוזת של תכונות המחלקה המחוברות.



BOOP_recieve

קובץ בו המחלקה : copyCatGUI.py

תפקיד המחלקה : לנתח את התגובות שנשלחות על ידי השרת ולהקל על הוצאת המידע מהן. יש הרבה מחלקות שיושרות ממנה ומוסיפות פונקציות נוספות וספציפיות יותר לסוגי בקשות שונים. תכונות המחלקה :

- _start - מחרוזת. מכילה שני דברים, שם הפרוטוקול וקוד התוצאה של בקשת המשתמש (האם הבקשה הצליחה או נכשלה).

- _header - מחרוזת, מכיל את השדות הבאים : סוג התוכן ואורך התוכן.

- _content - מחרוזת. יכול להכיל הרבה דברים. תלוי לחלוטין בסוג התגובה.

פעולות במחלקה :

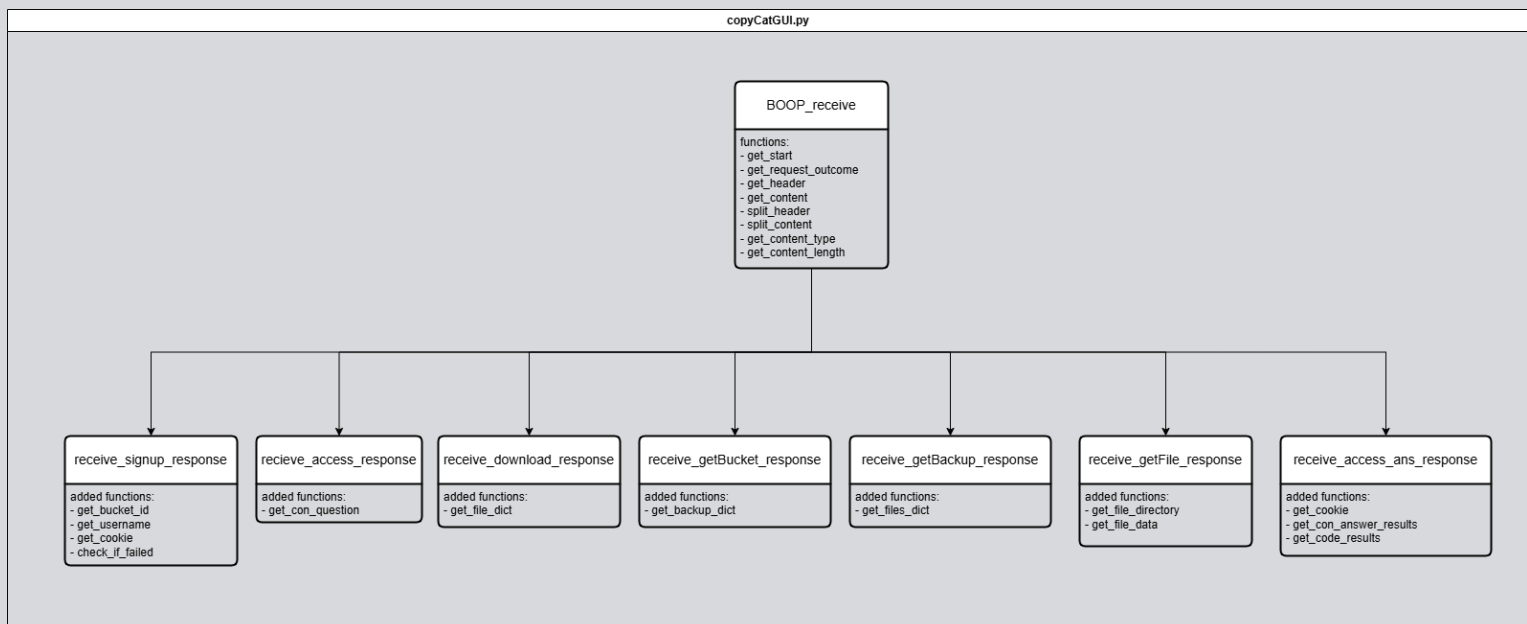
- Get_start - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה תחזיר את המשתנה _start כמחרוזת.

- Get_request_outcome - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה תחזיר קוד התוצאה של בקשת המשתמש כמחרוזת.

- Get_header - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה תחזיר את המשתנה _header כמחרוזת.

- Get_content - טענת כניסה : הפעולה מקבלת רק את self. טענת יציאה : הפעולה תחזיר את המשתנה _content כמחרוזת.

- Split_header – טענת כניסה : key מסוג צ'אר. טענת יציאה : יפריד את המשתנה _content לרשימה לפי key. כלומר בכול מקום שבו key יופיע החלק שאחריו יהיה האיבר הבא ברשימה. יחזיר רשימה.
- Split_content - טענת כניסה : key מסוג צ'אר. טענת יציאה : יפריד את המשתנה _header לרשימה לפי key. כלומר בכול מקום שבו key יופיע החלק שאחריו יהיה האיבר הבא ברשימה. יחזיר רשימה.
- Get_content_type - טענת כניסה : הפעולה מקבלת רק את הself. טענת יציאה : הפעולה תחזיר את שדה סוג התוכן ממשתנה _header כמחרוזת.
- Get_content_length - טענת כניסה : הפעולה מקבלת רק את הself. טענת יציאה : הפעולה תחזיר את תחזיר את שדה אורך התוכן ממשתנה _header כמחרוזת.



מחלקות מיובאות :

אין

חלק ב

○ אימות דו שלבי

קטעי קוד רלוונטיים:

הקוד שכקורא את העוגייה מהקובץ:

```
with open("clientsCookies.txt",'r') as f:
    try:
        file_content = f.read()
        self.client_cookies = json.loads(file_content)
    except json.decoder.JSONDecodeError:
        self.client_cookies = { }
```

מקום הקוד בפרויקט: בפעולת init של מחלקת Handle_security בקובץ copyCatHandle.py. תפקיד הקוד: קטע קוד זה קורא מהקובץ בו שמורות העוגיות של המשתמש עם הפעלת השרת, הסיבה לכך היא שאם השרת נסגר ומופעל מחדש הוא עדיין יזכור את העוגיות של המשתמשים שלו מלפני שנסגר. אם לא היה את הקטע קוד הזה השרת היה לא נותן לאף משתמש להשתמש בתוכנה והיה דורש אימות משתמש מכולם כול פעם שהיה נסגר ונפתח. הסבר הקוד: קטע זה פותח את הקובץ בו שמורות העוגיות של המשתמש, הטקסט הינו מילון שעבר json.dumps ולכן כול התוכן שלו ישוחזר באמצעות json ויוכנס לתוך התכונה client_cookies של Handle_security.

הבדיקה של השרת לראות האם העוגייה נכונה:

```
def check_cookie(self,username,cookie):
    if self.client_cookies[username] == cookie:
        return True
    return False
```

מקום הקוד בפרויקט: זוהי פעולה בתוך מחלקת Handle_security בקובץ copyCatHandle.py. תפקיד הקוד: הקוד בודק את עוגיית המשתמש. באמצעות קוד זה השרת יודע אם העוגייה של המשתמש נכונה והאם לבקש מהמשתמש לאמת את זהותו. הסבר הקוד: הפעולה מקבלת שם משתמש ועוגייה ובודקת אם העוגייה השמורה כערך של המפתח שהוא שם המשתמש במילון זהה לעוגייה שהיא קיבלה. אם העוגייה זהה הפעולה תחזיר True. אם לא היא תחזיר False.

קבלת בקשת ACCESS (שליחת האימייל, שליפת השאלה)

יצירת התגובה:

```
class send_con_question(BOOP_send):
    def __init__(self, request_outcome, username=None):
        super().__init__(request_outcome)
        if request_outcome == "200 OK":
```

```
security.send_email(username)
self.content = f'confirmation-question: {self.get_con_question(username)}';
self.set_header()
```

מקום הקוד בפרויקט: פעולה בונה של מחלקת send_con_question שיורשת מBOOP_sendm בקובץ copyCatHandle.py.

תפקיד הקוד: הפעולה קוראת לפעולות שונות ובאמצעותן היא שולפת שאלת קונפירמציה ושולחת אימייל למשתמש. הפעולה היוצרת מכניסה לתכונות שונות שלה ערכי מחרוזת. המחרוזות הללו יוחברו ויהפכו להודעה שתשלח למשתמש.

הסבר הקוד: התכונה מקבלת request_outcome מסוג מחרוזת ושם משתמש מסוג מחרוזת, אם לא מוכנס פרמטר שני ערך ברירת המדל של שם המשתמש הוא None. הפונקציה עושה את הפעולה ההתחלתית של המחלקה שהיא יורשת ממנה BOOP_send (זה מסתכם בכך שהיא יורשת את כול הדברים שנעשים במקרה בו request_outcome לא שווה ל"OK 200" והשרת נכשל למלא את בקשת המשתמש מאיזשהי סיבה, וגם הצבת ערך self.start). במקרה בו request_outcome שווה ל"OK 200" הפונקציה תקרא לsend_email ותשלח לו את שם המשתמש. לאחר מכן היא תכניס לתכונה content את שאלת אימות הזוהות שהיא תשיג באמצעות קריאה לפעולה get_con_question. לבסוף היא תשים ערך לתכונה header באמצעות הפונקציה set_header שהיא יורשת מBOOP_sendm.

שליחת אימייל:

```
def send_email(self,username):
    print(username)
    destination = data_base.get_user_email(username)
    if destination != None:
        s = smtplib.SMTP('smtp.gmail.com', 587)
        s.starttls()
        s.login(const.server_email_adress, const.email_app_password)
        code = random.randint(100000, 999999)
        self.user_codes[username] = code
        message = f'subject: reconnecting to copycat\rhello user!\rwe see you are
trying to reconnect to the copycat server!\renter this code to confirm your
identity\r{code}'
        try:
            s.sendmail(const.server_email_adress, destination, message)
        finally:
            s.quit()
    .copyCatHandle.py בקובץ handle_security בפעולה במחלקה
```

תפקיד הקוד: הקוד שולח הודעת אימייל למשתמש שהוא קיבל. בתוך ההודעה יהיה הקוד שהמשתמש יצטרך להזין כדי להוכיח את זהותו לתוכנית.

הסבר הקוד: הפונקציה מקבלת את שם המשתמש כמחרוזת. היא קוראת לפונקציית `get_user_email` שתחזיר לה את כתובת האימייל שאליה יש לשלוח את האימייל. אם הוחזרה כתובת אימייל (כלומר `get_user_email` החזירה ערך) אז הפונקציה תיצור שרת SMTP שיהיה מחובר לכתובת האימייל של התוכנית, לאחר מכן הפונקציה תגריל את הקוד שהיא תשלח למשתמש ותוסיף א הקוד לתכונה של `handle_security`, `user_codes` שהינו מילון. המפתח יהיה שם המשתמש והערך הקוד. ככה התוכנית תוכל לבדוק את הקוד שהמשתמש הזין בהמשך. לאחר מכן התוכנית תשלח את האימייל אם הקוד למשתמש.

שליפת שאלה:

```
def get_a_users_conQuestion(self, username, question_num):
    user_info_params = (username,)
    match question_num:
        case 1:
            grab_question = self.conn.execute(f"SELECT conformation_question1
FROM answers_to_questions WHERE username = ?", user_info_params)
            question = grab_question.fetchone()[0]
        case 2:
            grab_question = self.conn.execute(f"SELECT conformation_question2
FROM answers_to_questions WHERE username = ?", user_info_params)
            question = grab_question.fetchone()[0]
        case 3:
            grab_question = self.conn.execute(f"SELECT conformation_question3
FROM answers_to_questions WHERE username = ?", user_info_params)
            question = grab_question.fetchone()[0]
    return question
```

מקום הקוד בפרויקט: פעולה במחלקה `handle_DB` בקובץ `copyCatHandle.py`.
תפקיד הקוד: הפעולה שולפת את אחת משאלות הקונפירמציה של המשתמש ומחזירה אותה.
הסבר הקוד: הפונקציה מקבלת את שם המשתמש כמחרוזת ומספר בין 1 ל 3. המספר יקבע איזו מהשאלות השרת ישלח למשתמש. הפונקציה מבצעת שאילתה ושולפת את השאלה המתאימה למספר שקיבלה ומחזירה אותו.

רשימת העוגייה החדשה אצל הלקוח:

```
class client_info:
    def __init__(self):
        info_list = []
        with open("clientInfo.txt", 'r') as f:
            for line in f:
                line_parts = line.split("=")
                _, value = line_parts
```

```

if value[1:len(value)-1] == "0":
    info_list.append(None)
else:
    info_list.append(value[1:len(value)-1])
self.username,self.bucket_id,self.cookie = info_list

```

```

def update_info(self):
    info_list = []
    with open("clientInfo.txt",'r') as f:
        for line in f:
            _,value = line.split("=")
            info_list.append(value[1: :])
    self.username,self.bucket_id,self.cookie = info_list

```

```

def set_cookie(self, cookie):
    with open("clientInfo.txt",'w') as f:
        f.write(f"username = {self.username}\nbucket_id =
{self.bucket_id}\ncookie = {cookie}")

```

מקום הקוד בפרויקט: מחלקת client_info בקובץ copyCatGUI.py.

תפקיד הקוד: מחלקה זו קוראת וכותבת לקובץ clientInfo.txt. באמצעותה הלקוח מעדכן את המידע שלו ומסוגל לזכור את הפרטים הללו גם אם התוכנית בצד של הלקוח נסגרת ונפתחת מחדש.

הסבר הקוד: בפעולת האיתחול התוכנית קוראת מהקובץ, מכינה רשימה של הערכים בו ומכניסה אותם לתכונות המחלקה username, bucket, cookie. הפעולה update_info קוראת מהקובץ, יוצרת רשימה של הערכים בו ומעדכנת את תכונות המחלקה בהתאם. הפעולה set_cookie מקבלת את העוגייה החדשה של המשתמש, פותחת את הקובץ ומעדכנת את עגייה המשתמש שכתובה בו.

○ יצירת גיבוי

קטעי קוד רלוונטיים:

יצירת רשימת הקבצים להעלאה:

```

def make_file_list(self):
    all_dir_list = []
    for dir in const.top_dirs:
        in_list = self.make_file_list2(dir,all_dir_list)
    return all_dir_list

```

```

def make_file_list2(self,current_path,file_list):

```



```

dir_tuple = next(os.walk(current_path))
_,folders,files = dir_tuple
for file in files:
    file_path = os.path.join(current_path,file)
    file_list.append(file_path)
for folder in folders:
    new_path = os.path.join(current_path,folder)
    self.make_file_list2(new_path,file_list)
return file_list

```

מקום הקוד בפרויקט: פעולות במחלקה handle_backups בקובץ copyCatGUI.py.
תפקיד הקוד: הקוד הזה יוצר רשימה של כול הקבצים במחשב של המשתמש שיש לשלוח אותם ליצירת גיבוי אצל השרת.

הסבר הקוד: הפעולה make_file_list יוצרת רשימה ריקה, היא עוברת בלולאה על כול תיקייה ברשימת התיקיות שיש לגבות, בכול תיקייה היא קוראת לפעולת העזר make_file_list2. פעולת העזר make_file_list2 מקבלת את נתיב הקובץ/ התיקיה הנוכחית כמחרוזת ואת רשימת הקבצים. הפעולה לאחר מכן יוצרת רשימה של כול התיקיות בתיקיה הנוכחית וכול הקבצים בתיקיה הנוכחית. הפעולה מוסיפה את נתיבי הקבצים לרשימה שהיא קיבלה, ועוברת בלולאה על רשימת התיקיות, על כול תיקייה היא קוראת לעצמה בקריאה רקורסיבית. ושולחת לעצמה את הנתיב של התיקיה הנוכחית ואת רשימת הקבצים. פעולה זו מבצעת רקורסיית עץ שבאמצעותה היא מסוגלת לעבור על כול הקבצים בכול התיקיות והתת תיקיות התוך התיקיות שיועדו לגיבוי.

יצירת מילון המידע:

```

def compress_the_computer_data(self):
    file_list = self.make_file_list()
    computer_data = { }
    for file in file_list:
        print(file)
        with open(file,'rb') as f:
            content = f.read()
            compressed_data = zlib.compress(content)
            encoded_data = base64.b64encode(compressed_data).decode()
            file_name = file
            for dir in const.top_dirs:
                if dir in file:
                    file_name = file[len(dir)+1:]
            computer_data[file_name] = encoded_data
    return computer_data

```

מקום הקוד בפרויקט: פעולה במחלקה handle_backups בקובץ copyCatGUI.py.

תפקיד הקוד: הפעולה יוצרת מילון של כול המידע להעלאה לשרת. מילון זה בנוי כך שהמפתחות הם הנתבי לקובץ והערך הוא המידע החדש ששמור בקבצים.

הסבר הקוד: הפעולה קוראת לפעולה `make_file_list` שמחזירה לה רשימה של כול נתבי הקבצים בתיקיות שיועדו לגיבוי. לאחר מכן הפונקציה עוברת על רשימת הקבצים בלולאה, היא קוראת כול קובץ, דוחסת את התוכן שלו ומצפינה אותו על בסיס 64 (מה שיאפשר להפוך את המילון למחרוזת באמצעות JSON אחר כך). היא גם לוקחת את שם הקובץ ומורידה ממנו את המיקום שלו במחשב המשתמש. זה יאפשר לשרת ליצור מיד את הקובץ במקום שניתן לו ולא להסתבך אם הנתבי ועם מה להוריד ממנו. הפונקציה תשים את הנתבי המקוצר כהמפתח במילון ואת המידע החדש והמוצפן כהערך. לבסוף הפונקציה תחזיר את המילון.

הפונקציה אצל השרת ששומרת בשטח האחסון של המשתמש ומצפינה את הקבצים:

`def create_new_backup(self, bucket_id, username, user_data):` #user data is the big bad dictionary with all the file paths and file data in it

`print("in create_new_backup")`

`backup_num = data_base.get_user_backup_num(bucket_id)`

`backup_num += 1`

`root = os.path.join(const.base_dir, str(bucket_id), str(backup_num))`

`try:`

`os.makedirs(root, exist_ok=True)`

`print(f'the directory {root} was created successfully')`

`for key,value in user_data.items():`

`file_location = key`

`newPath = os.path.join(root, file_location)`

`self.create_file_in_backup(username, newPath, value)`

`data_base.new_backup_update(bucket_id, backup_num)`

`except OSError as error:`

`print("in line 513")`

`print(error)`

מקום הקוד בפרויקט: פעולה במחלקה `handle_storage` בקובץ `copyCatHandle.py`.

תפקיד הקוד: הפונקציה תיצור גיבוי חדש בשטח האחסון, תוסיף אליו את קבצי המשתמש ותקרא לפונקציה שתעדכן את מסד הנתונים.

הסבר הקוד: הפונקציה מקבלת את מספר הזהות של דלי המשתמש, את שם המשתמש ואת מילון הקבצים (המילון בו המפתח הוא הנתבי והערך הוא המידע בקובץ). הפונקציה תיצור תיקייה חדשה לגיבוי החדש בתוך דלי המשתמש, שם תיקיית הגיבוי החדש יהיה המספר של הגיבוי. כלומר אם זה הגיבוי השני השם יהיה '2'. לאחר מכן, הפונקציה תעבור בלולאה על איברי המילון שקיבלה, תיצור את המיקום השמור במפתח בתוך תיקיית הגיבוי ותקרא לפונקציית `create_file_in_backup` שתכתוב לתוך המיקום את מידע הקובץ. לבסוף היא תקרא לפונקציית `data_base.new_backup_update` שתעדכן את מסד הנתונים.

○ שחזור גיבוי

קטעי קוד רלוונטיים:

ההכנה של הגיבוי לשליחה בשרת:

הכנת רשימת הקבצים הרלוונטית:

```
def get_backup_file_list(self, bucket_id, backup_num):
    print("in get file list")
    file_list = []
    backup_path = os.path.join(const.base_dir, str(bucket_id), str(backup_num))
    os_walk_result = next(os.walk(backup_path))
    _, folders, files = os_walk_result
    for file in files:
        file_path = os.path.join(backup_path, file)
        file_list.append(file_path)
    return self.files_in_folder(folders, backup_path, file_list)
```

```
def files_in_folder(self, folders, directory, file_list):
    for folder in folders:
        current_path = os.path.join(directory, folder)
        os_walk_result = next(os.walk(current_path))
        _, inner_folders, files = os_walk_result
        for file in files:
            file_path = os.path.join(current_path, file)
            file_list.append(file_path)
        self.files_in_folder(inner_folders, current_path, file_list)
    return file_list
```

מקום הקוד בפרויקט: פעולות במחלקה `handle_storage` בקובץ `copyCatHandle.py`.

תפקיד הקוד: ליצור מילון של קבצים למען שליחה ללקוח להורדה.

הסבר הקוד: האלגוריתם בפעולות אלו כמעט זהה לחלוטין ל-`make_file_list1`

`make_file_list2`. ההבדל העיקרי הוא שהפעולה `get_backup_file_list` מקבלת את מספר הדלי ומספר הגיבוי בהם היא משתמשת כדי ליצור את הנתביב עליו היא עוברת.

הכנת המילון:

```
def get_backup(self, bucket_id, username, backup_num):
    #this function will create an information dictionary (key = directory, value =
    data)
    print("in get backup")
    file_list = self.get_backup_file_list(bucket_id, backup_num)
    backup_data = { }
    for file in file_list:
```

```

with open(file,'rb') as f:
    file_name = file[len(const.base_dir)+6:]
    encrypted_content = f.read()
    content = security.decrypt_user_data(username,encrypted_content)
    backup_data[file_name] = base64.b64encode(content).decode()
return backup_data

```

מקום הקוד בפרויקט: פעולה במחלקה `handle_storage` בקובץ `copyCatHandle.py`.
תפקיד הקוד: הקוד לוקח את רשימת הקבצים והופך אותה למילון עם המידע הנחוץ. את המילון הוא מחזיר.

הסבר הקוד: כמעט זהה לפעולה `compress_the_computer_data` בצד הלקוח עם ההבדלים הבאים: הקבצים ששמורים בצד שרת אף פעם לא נפרסים. לכן השרת לא ידחוס אותם מחדש בעת שליחה. בנוסף לכך, כשהשרת שומר קובץ אצלו הוא מצפין אותו בצופן AES באמצעות `Key` ו-`Nonce` יחודיים ללקוח שהוא שולף ממסד הנתונים. לכן בהכנה לשליחה הפונקציה תקרא ל-`security.decrypt_user_data`. מלבד זאת שתי הפונקציות זהות, שתיהן יצפינו בבסיס 64 ויחזירו את המילון כדי שהוא יוכל להפוך למחרוזת לשליחה.

ההורדה של הגיבוי אצל המשתמש:

```

def download_backup_to_user(self, backup_data):
    print("in download backup to user")
    for key,filecontent in backup_data.items():
        print(f"file name: {key}")
        filepath = os.path.join(const.goto_path,key) #this will be the full path in the
        user's computer
        print(f"file path: {filepath}")
        compressed_data = base64.b64decode(filecontent.encode())
        file_data = zlib.decompress(compressed_data)
        try:
            pathlib.Path(os.path.dirname(filepath)).mkdir(parents=True,
            exist_ok=True)
            with open(filepath,'wb') as f:
                f.write(file_data)
            print(f"wrote successfully into {filepath}")
        except OSError as error:
            print("at except")
            print(error)

```

מקום הקוד בפרויקט: פעולה במחלקה `handle_backups` בקובץ `copyCatGUI.py`.
תפקיד הקוד: פונקציה זו תוריד את הגיבוי שהיא קיבלה מהשרת למחשב המשתמש.
הסבר הקוד: הפונקציה מקבלת את מילון הקבצים (המילון בו המפתח הוא הנתבי והערך הוא המידע בקובץ). הפונקציה תעבור על איברי המילון בלולאה, ועבור כול צמד תצרף

את נתיב הקובץ לנתיב הקבוע אליו שיועד להורדה (ברירת המחדל של התוכנית זה אזור ההורדות), תיצור את הנתיב הזה, תפתח אותו, ותכתוב אליו את המידע של הקובץ אחרי שהוא נפרס. הפונקציה תעשה את זה לכול הקבצים עד שכול הגיבוי יורד למחשב המשתמש.

חלק ג

הבדיקה	מטרת הבדיקה	מה בוצע	תוצאות הבדיקה	בעיות ופתרונן
דימיתי הירשמות ראשונה למערכת	בדיקת היכולות הבסיסיות של התוכנית כמו התקשורת, ממשק המשתמש וניהול מסד הנתונים ושטח האחסון על ידי השרת. בנוסף לכך בדיקת שאר היכולות שנחוצות להרשמה.	הכנסת פרטי משתמש לחלון ההרשמה	מידע של המשתמש שהוזן לא הגיע לשרת, השרת נתקע ולא הגיב למשתמש, מידע הלקוחות לא הוכנס למסד הנתונים.	השרת והלקוח לא היו מחוברים זה לזה כמו שצריך. הבעיות העיקריות היו בשרת, היו בעיות בקריאת ההודעה והניתוח שלה. בעיות אלו נפתרו בקלות עם הוספות גרשיים או תיקון של תווים בודדים. מסד הנתונים לא נוצר בתוך הקוד כמו שצריך.
העלאת גיבוי לשרת	המטרה הייתה לבדוק את יכולות הלקוח והשרת להתנהל עם הקבצים. בנוסף לכך, בדיקה זו בדקה את היכולת של התוכנית להתמודד עם הודעות ארוכות.	העלאתי ממחשב הלקוח את הגיבוי למחשב השרת.	הלקוח נכשל בהכנת הגיבוי, השרת קרס, השרת נכשל בשמירת הגיבוי. בנוסף לכך, המשתמש שלח את המיקום של הקבצים במחשב שלו מה שגרם לבעיות בשמירת הקבצים אצל השרת.	א. הלקוח נכשל בהכנת הגיבוי משתי סיבות: סיבה ראשונה היא שהכנת הגיבוי נעשתה בריקורסיה של עצים והייתה בה תקלה. הסיבה השנייה היא שלא היה תנאי עצירה מה שגרם לתוכנית להמשיך לרוץ כשהיא לא הייתה מסוגלת יותר. ב. השרת לא הצליח לשמור גיבוי מכיוון שהוא לא קיבל את המילון השלם עם הקבצים. הסיבה לבעיה זו היא שההודעה של הלקוח נקטעה בגלל מגבלת הגודל של ההודעה שמתקבלת שהייתה בשרת (ובלקוח) הפתרון לכך הוא

				שכשהשרת מקבל הודעה, הוא קורא את השדה בראש ההודעה בו כתוב את אורך תוכן ההודעה. אז השרת יחכה ויקשיב עד שתוכן ההודעה הוא באורך שנתון לו. ג. לשרת היו בעיות רקורסיה דומות לאלו שהיו בצד הלקוח. ד. שיניתי את הקוד ככה שהמשתמש מוריד את כול הנתיב של הקובץ עד התיקייה הראשונה שהוא רוצה לגבות אצל השרת.
פתיחת דלי	מטרת הבדיקה היא לבדוק את סוג בקשת GETBUCKET את תגובת השרת ויכולת הלקוח להציג אותה.	שלחתי לשרת בקשת פתיחת דלי.	השרת לא שלח את הפרטים הנכונים על הדלי, היו טעויות בדרך בה הלקוח הציג את המידע.	הייתה טעות בפונקציה שהכינה את תגובת השרת. GUI היה צריך תיקונים כדי שהלקוח יוכל לראות את המידע שבדלי כמו שצריך.
פתיחת גיבוי	מטרת הבדיקה הייתה לבדוק את היכולת לפתוח את הגיבוי כדי שהמשתמש יוכל לראות איך הקבצים מסודרים בו.	נשלחה לשרת בקשת פתיחת גיבוי.	המידע שהשרת שלח היה לא נכון, בנוסף לכך, הוצג מול המשתמש את מקום השמירה של הקבצים בתוך שטח האחסון של השרת.	תיקנתי את הפונקציה של השרת ככה שהיא שלחה את המידע בדרך שמחשב הלקוח ציפה לקבל אותו ובנוסף לכך וידאתי שהשרת מוריד את כול נתיב הקובץ עד התיקייה שהמשתמש גיבה.
הורדת גיבוי	לוודא שהשרת מצליח לשלוח את הקובץ	בחרתי גיבוי מסוים וביקשתי	הלקוח נכשל בלהוריד את הגיבוי, כשהוא	ללקוח הייתה בעיה להקשיב להודעות גדולות, הפתרון היה באמצעות שדה אורך התוכן

	ושהמשתמש מצליח להוריד אותו בצורה תקינה.	מהשרת להוריד אותו למחשב הלקוח.	הצליח הוא הוריד אותו למקום הלא נכון.	בראש ההודעה, הלקוח עכשיו עובר בלולאה ומקבל את ההודעה עד שהיא שלמה. ההורדה למקום הלא נכון נגרמה מטעות בעריכת הנתונים של הקבצים.
פתיחת קובץ	מטרת הבדיקה הייתה להצליח להגיע למצב בו התוכנה פותחת קובץ מסוים לצפיית המשתמש בלי לשמור אותו על מחשב המשתמש.	בחרתי קובץ מסוים מגיבוי מסוים וביקשתי מהשרת לפתוח אותו.	הקובץ לא הצליח להיפתח כמה פעמים.	הקובץ לא נפתח מכמה סיבות: הסיבה הראשונה הייתה שהתוכנית לא פרסה את הקובץ הדחוס לפני שהיא ניסתה להראות אותו למשתמש. הסיבה השנייה הייתה תקלה בקובץ הזמני שבאמצעותו המשתמש רואה את הקובץ שבחר, התוכנית הייתה סוגרת את הקובץ מה שהיה גורם לו להימחק.
אימות זהות המשתמש	מטרת הבדיקה הייתה לבדוק את האימות הדו שלבי.	פתחתי את הקובץ עליו שמור המידע של המשתמש ומחקתי את הערך שהיה בעוגייה והחלפתי אותו בערך אחר. לאחר מכן הפעלתי את התוכנית וניסיתי ללחוץ על כפתור העלאה או צפייה בדלי.	מיצג המשתמש הציג את שאלת האימות בצורה שהמשתמש לא יכל לראות, הודעת האימייל לא נשלחה.	ממדי החלונות והסידור שלהם ב-GUI היה צריך תיקון, בנוסף לכך היה צורך להכין סיסמה מיוחדת לחשבון האימייל של הפרויקט כדי לגרום לאלגוריתם שליחת האימייל לעבוד.
הרצה עם שני לקוחות	מטרת הבדיקה היא לראות ששני לקוחות יכולים להשתמש	השגתי שני מחשבים נוספים, התקנתי עליהם	המחשבים לא הצליחו להתחבר לשרת, התוכנה ביקשה	הבעיה הראשונה הייתה יותר חומרית מתוכנית. הייתי צריכה לכבות את חומת האש של

כול המחשבים המעורבים ולחבר את כולם לאותה רשת. הסיבה לכך שהתוכנה ביקשה מהם להירשם מחדש היא שמסד הנתונים עבר לעבוד עם טרנסקציות והיה חסר COMMIT. הסיבה שהמחשבים לא יכלו להתחבר מחדש היא שהשרת חשב שהם כבר היו מחוברים ולכן לא טרח לחבר אותם מחדש. הפתרון היה לומר לשרת למחוק את הלקוח מרשימת הלקוחות לפני שהלקוח מתנתק.	מהמשתמשים להירשם מחדש כול פעם שהם ניסו לעשות משהו, כשמחשב סגר את התוכנה הוא לא יכל לפתוח אותה מחדש כול עוד השרת לא אותחל.	את התוכנה והרצתי אותם בו זמנית.	בתוכנית בו זמנית.	
---	--	--	--------------------------	--

מדריך למשתמש

עץ קבצים

קבצי המערכת :

- _backup
- clientsCookies.txt
- clientSideConstants.py
- copyCatClient.py
- copyCatClient_comm.py
- COPYCATDB.db
- copyCatGUI.py
- copyCatHandle.py
- copyCatLogo.png
- copyCatServer.py
- copyCatServer_comm.py
- request_types.py
- key.pem
- cert.pem
- serverSidesConstants.py

התקנת המערכת

- הסביבה הנדרשת
 - התוכנית עובדת רק על מחשבי windows
- פירוט הכלים הנדרשים
 - נדרש שעל מחשב המשתמש יהיה מותקן פייתון ואת הספריות הבאות : `sqlite3`, `customtkinter`, `crypto`, `PIL`
- מיקומי קבצים
 - הקבצים שצריכים להיות על מחשב המשתמש הם : `copyCatClient.py`, `clientInfo.txt`, `clientSideConstants.py`, `copyCatGUI.py`, `copyCatClient_comm.py`, `copyCatLogo.png`, `request_types.py`. הם כולם צריכים להיות באותה התיקייה, כדי להפעיל את התוכנית יש להריץ את הקובץ `copyCatClient.py`.
- נתונים התחלתיים
 - : `clientInfo.txt`
 - המצב ההתחלתי של הטקסט בקובץ זה צריך להיות
 - `username = 0`
 - `bucket_id = 0`
 - `cookie = 0`
- : `clientSideConstants.py`
 - בתוך קובץ זה הערך של המשתנה `top_dirs` תהיה רשימה של התיקיות שהמשתמש ירצה לשמור בענן.
 - במשתנה `goto_path` על המשתמש לכתוב את מיקום התיקייה אליה השרת יוריד את הגיבויים
- רשת
 - השרת והלקוח צריכים להיות מסוגלים לדבר זה עם זה ב-TCP/IP
- ארכיטקטורה מינימלית נדרשת
 - צריך שיהיה לקוח ושהשרת ירוץ.

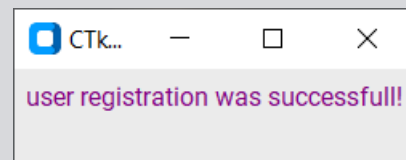
משתמשי המערכת

○ אופן הפעלה

אם פתיחת הפרויקט החלון הראשון שיפתח הוא חלון ההרשמה:

מכיוון שהמשתמש הוא משתמש חדש התוכנה תרצה שהוא ירשם למערכת. כדי להירשם הוא יצטרך להכניס את השדות הבאים: שם המשתמש שלו, כתובת האימייל שלו (שיישמר אצל המערכת שתשלח לו אימיילים במקרה הצורך), ושאלות ותשובות. השאלות והתשובות הן בהתאמה. כלומר, confirmation answer 1 תהיה התשובה המתאימה ל confirmation question 1. מומלץ למשתמש לשים לעצמו שאלות שהוא יזכור את התשובה להן.

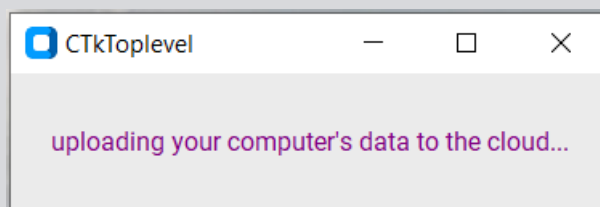
אם ההרשמה הצליחה המשתמש יראה את החלון הבא:



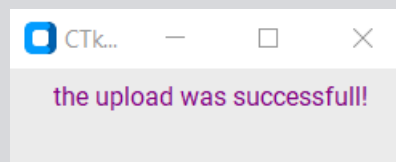
אחרי שהמשתמש סיים את ההרשמה ולחץ על כפתור 'sign in' יפתח מסך הבית:



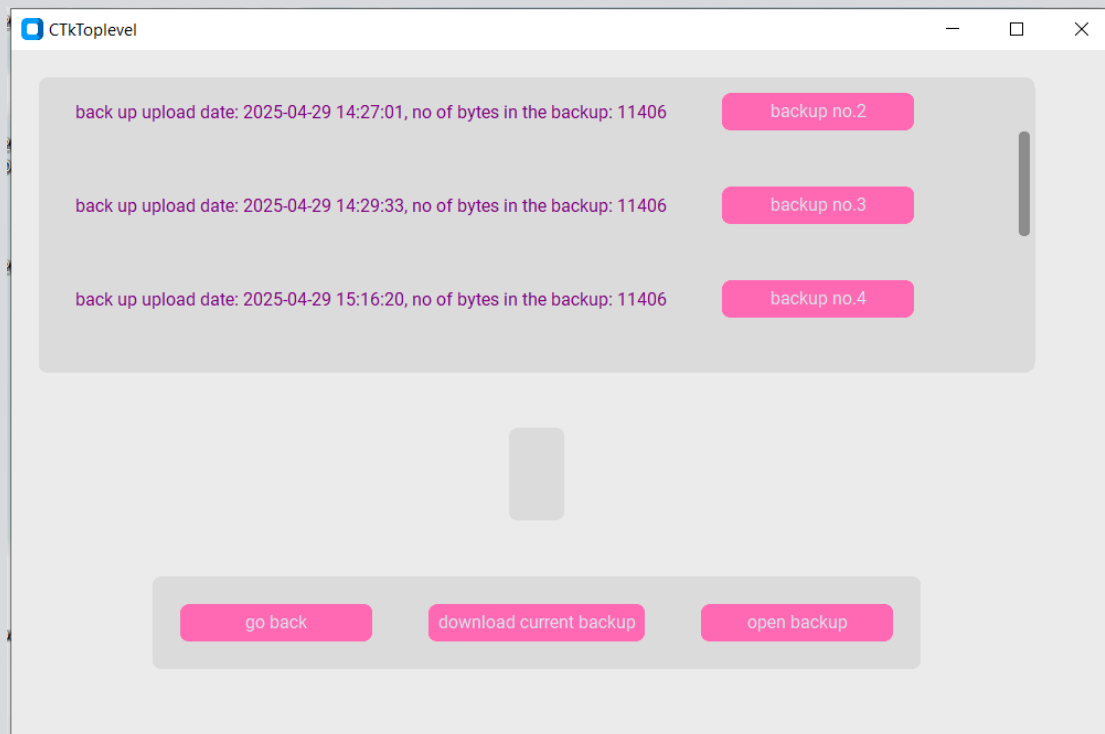
במסך הבית ישנם שני כפתורים: כפתור 'backup current data' וכפתור 'view bucket'. הכפתור הראשון יעלה את המידע הנוכחי של המשתמש לשרת, בזמן שהעלאה הזאת תקרה יפתח החלון:



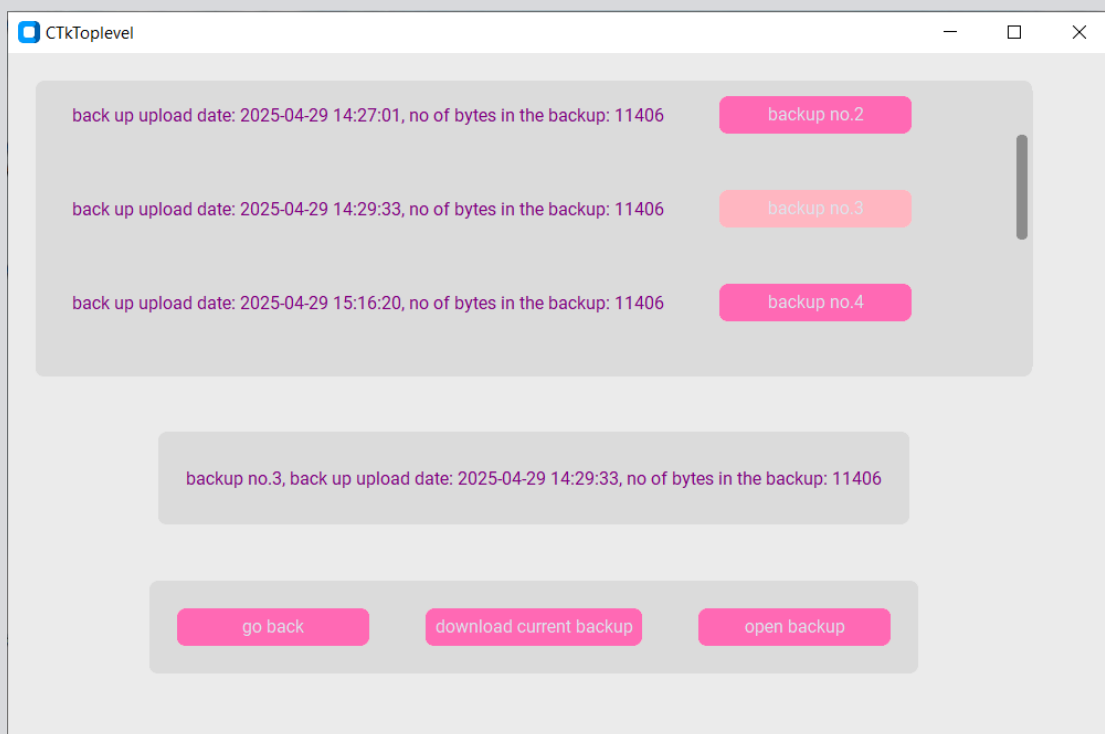
אם העלאת הגיבוי למשתמש הצליחה המשתמש יראה את החלון הבא:



אם המשתמש ילחץ על כפתור 'view bucket' יפתח החלון הבא:



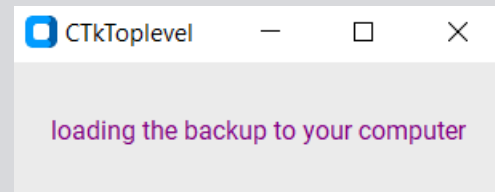
בחלון זה המשתמש יראה את רשימת הגיבויים שלו (המלבן העליון), את הגיבוי שהוא בחר (המלבן האמצעי, מיד עם פתיחת החלון הזה מלבן זה ריק מכיוון שהמשתמש לא בחר גיבוי מסוים עדיין), כדי לבחור גיבוי על המשתמש ללחוץ על הכפתור הורוד בו כתוב את מספר הגיבוי שהוא רוצה. למשל, אם הייתי רוצה לבחור את גיבוי מספר שלוש הייתי לוחצת על הכפתור 'backup no. 3'



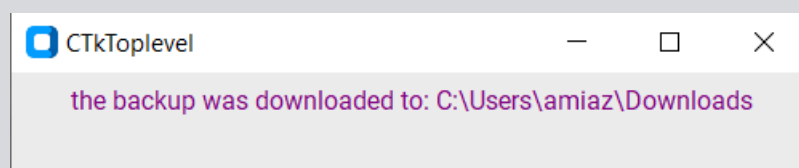
אחרי שלחצתי על כפתור הגיבוי שרציתי, המידע שלו נכתב במלבן האמצעי.

אם המשתמש לא מעוניין יותר במה שמציע חלון זה, הוא יכול ללחוץ על הכפתור השמאלי 'go back' שיחזיר אותו למסך הבית.

אחרי שהמשתמש בחר גיבוי הוא יכול לעשות מספר דברים איתו. הוא יכול ללחוץ על הכפתור האמצעי 'download current backup' שיוריד את הגיבוי הזה למחשבו ויפתח את החלון הבא:



אם הורדת הגיבוי למחשב המשתמש הצליחה המשתמש יראה את החלון הבא:

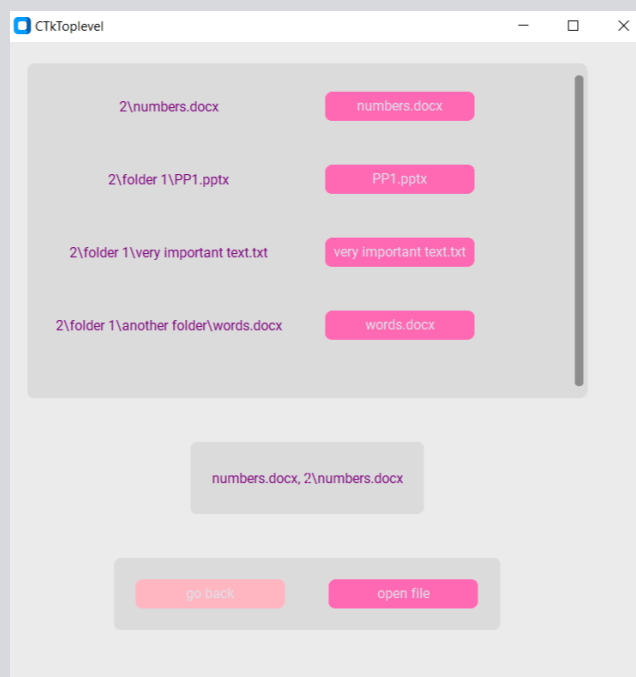


בו כתוב את הנתיב אליו התוכנית הורידה את הגיבוי.

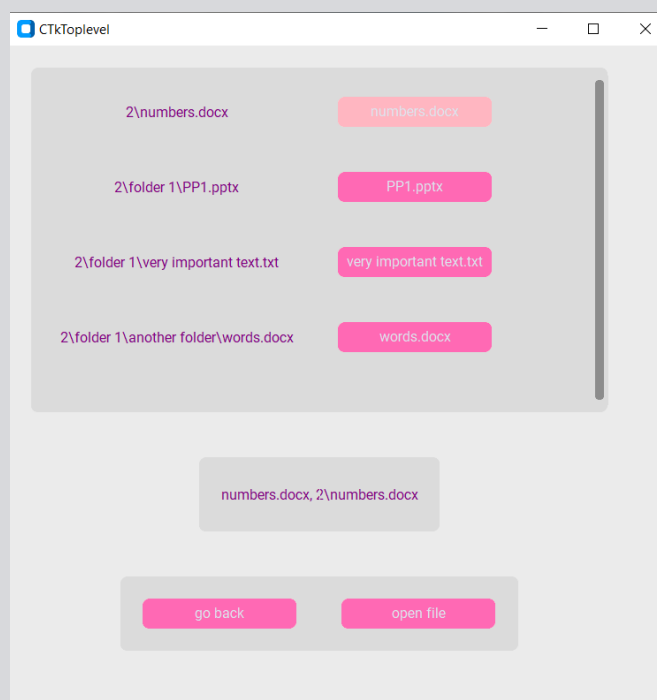
הוא גם יכול ללחוץ על הכפתור הימני 'open backup' שיפתח את החלון הבא:



בחלון זה, ניתן לראות את רשימת הקבצים הנמצאים בתוך הגיבוי שנבחר (מלבן עליון). את הקובץ שבחר המשתמש מתוך הקבצים הללו (מלבן אמצעי, ריק מכיוון שכשהחלון הזה נפתח אין קובץ בחור), ושני כפתורים. הכפתור השמאלי 'go back' יחזיר את המשתמש למסך הבית.

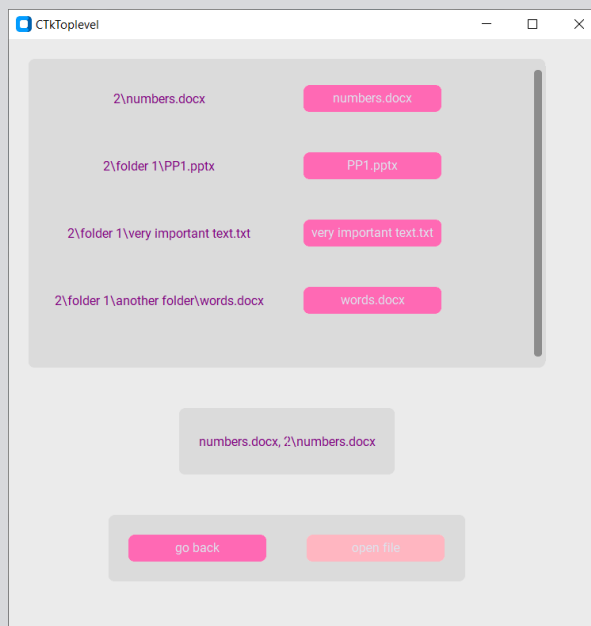


כדי לבחור קובץ, יש על המשתמש ללחוץ על הכפתור הורוד בו כתוב שם הקובץ. לדוגמא, אם הייתי רוצה לבחור את הקובץ 'numbers.docx' הייתי לוחצת על הכפתור הורוד ואז :

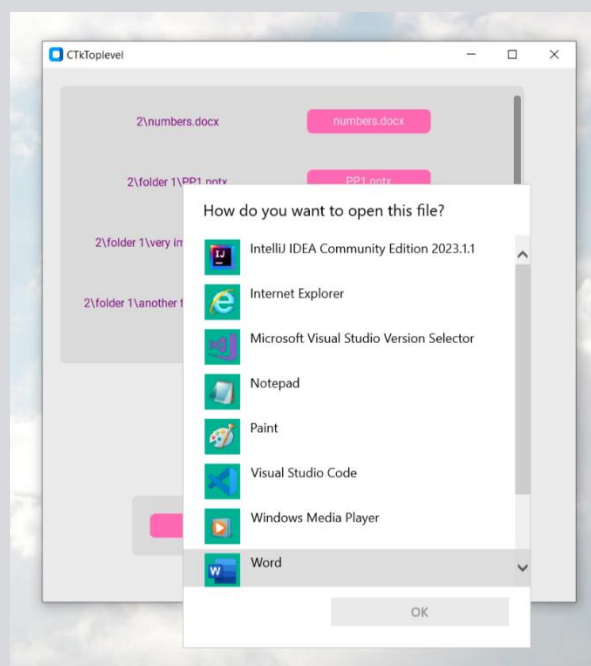


ניתן לראות שעכשיו המידע של הקובץ נמצא במלבן האמצעי.

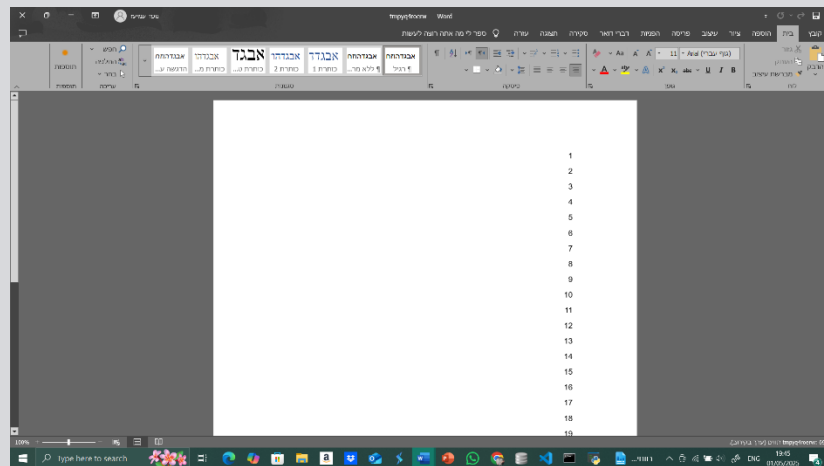
אם בחירת הקובץ מתוך הגיבוי המשתמש יכול לבחור להסתכל על התוכן שלו. בשביל זה, הוא ילחץ על כפתור 'open file'



עם לחיצה על כפתור 'open file', המשתמש יבחר איך לפתוח את הקובץ:



במקרה זה אני אפתח את הקובץ עם Word מכיוון שהקובץ הוא מסוג דוקס. אחרי שבחרתי את תוכנת פתיחת הקובץ ולחצתי על כפתור OK הקובץ יפתח

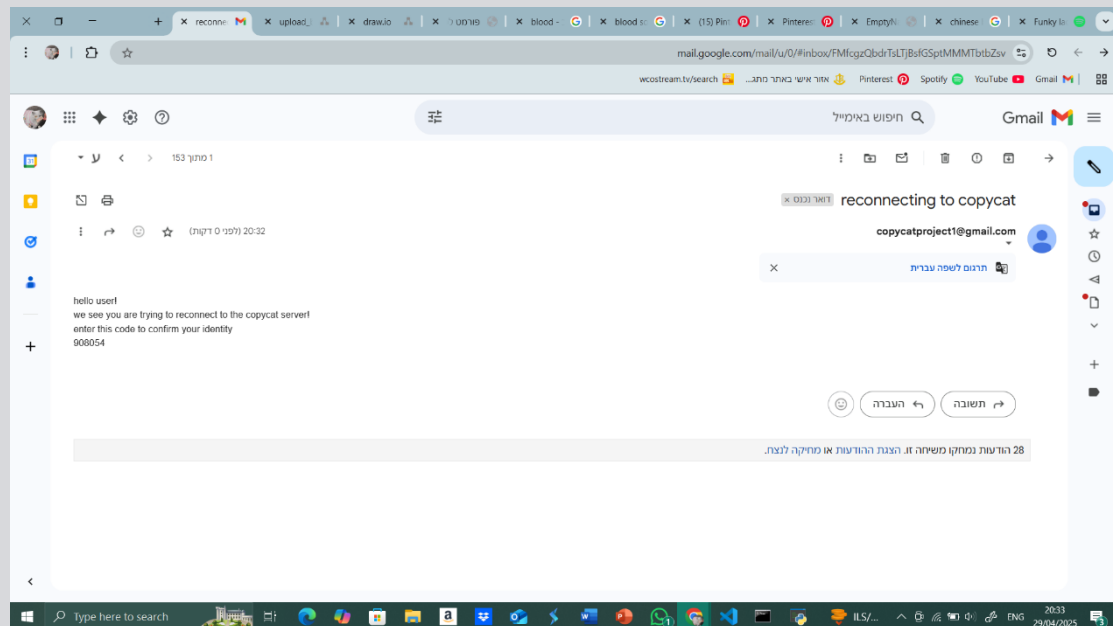


במקרה זה, בקובץ יש רשימה של מספרים.

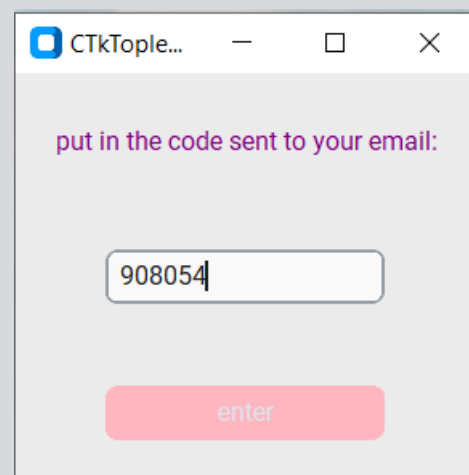
יכול להיות מקרה בו פרטי המשתמש (ספציפית העוגייה) בתוך הקובץ clientInfo.txt ייהרסו, במקרה זה, אם התחלת התוכנה כול כפתור שיילחץ לא יפתח את החלון היעודי שלו ובמקום זה יפתח החלון הבא:



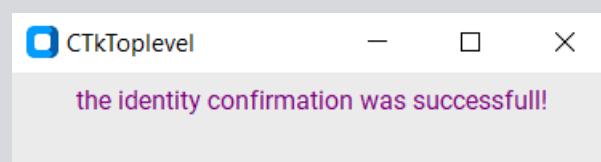
בו תופיע אחת משאלות הקונפומרציה שהמשתמש הכניס עם תחילת התוכנית, אזור הכנסה ריק (בצילום הזה אני כבר הכנסתי את התשובה) וכפתור 'enter'. על המשתמש להכניס את התשובה וללחוץ על הכפתור. עם היפתחות חלון זה גם תשלח הודעת אימייל לכתובת האימייל אותה הכניס המשתמש עם כניסתו הראשונה לתוכנה



האימייל הזה יכול קוד. במקרה הזה, הקוד הוא 908054, אחרי שהמשתמש הכניס את תשובתו ולחץ על 'enter', יופיע החלון הבא בו הוא יצטרך להכניס את הקוד שנשלח אליו :



אחרי שהמשתמש לחץ על 'enter' בחלון זה, אם הוא הכניס את התשובה והקוד הנכונים יפתח החלון הבא :



והמשתמש יוכל לחזור להשתמש בתוכנת copycat כרגיל.

רפלקציה

הפרויקט הזה התחיל מכך שניסיתי לחשוב על הלמידה העצמית. ניסיתי למצוא משהו בתחום הסייבר להתעסק בו והחלטתי שכתובת קוד שמערבת את הענף היה הדבר שהכי עניין אותי. משם התפתח הרעיון של הפרויקט בו המשתמשים מעלים את המידע שלהם לענף.

האתגר הראשון שלי היה עם סיפריות הענף שמצאתי באינטרנט, הסתכלתי על הרבה קורות והסברים אבל לא הצלחתי להבין אותם, לבסוף דיברתי עם המורה שלי וזה פתר את הבעיה בגלל שהוא פשוט אמר לי לא להשתמש בהן. במקום זה המחשב שלי שימש כהענף שעליו המשתמשים ישמרו את המידע שלהם. אחרי ששמעתי את זה התחלתי לעסוק בשמירת קבצים וכתובת ספרים בקבצים ושם היה רוב הקושי של העבודה.

האתגר השני הקשה בעבודה היה התקשורת, לקח הרבה זמן ובדיקות עד שהתקשורת עבדה כמצופה והתחלתי לפתור בעיות בשני קצוות הקוד של הלקוח והשרת.

כתיבת הפרויקט הזה לימדה אותי הרבה דברים על כתיבה במגוון נושאים הבולטים ביניהם הם : כתיבה למסד נתונים, תקשורת, עבודה עם שטח אחסון UI. שלא עבדתי איתם קודם.

אם היה לי עוד זמן אני חושבת שהייתי מוסיפה פיצ'רים, דברים שונים ומתקנת קצת את הקוד. לדוגמא הייתי רוצה להוסיף משהו שהיה מאפשר למשתמש לבחור איזה קבצים לגבות מUI. הייתי רוצה לעבוד במערכת של גיבוי השינויים ולא רק לגבות את כול המידע מחדש כול פעם, הייתי מוסיפה עוד אמצעי אבטחה הצפנה והגנת מידע וגם הייתי מתקנת את הUI לרוץ יותר חלק.

אני רוצה להודות למורי המגמת סייבר שעזרו לי בכתיבת הפרויקט ובמיוחד לליאת סגל ועידן פלדברג. אני גם רוצה להודות לאבא שלי ואחותי שיעצו לי ונתנו תמיכה טכנית ומורלית.

ביבליוגרפיה

- websiteplanet "10 שירותי הגיבוי הענני הטובים ביותר ב-2025 (בטוחים ואמינים)" - [/https://www.websiteplanet.com/he/cloud-storage](https://www.websiteplanet.com/he/cloud-storage)
- <https://cloud.google.com/apis/docs/client-libraries-explained>
- the GCS's "natural language guides - <https://cloud.google.com/natural-language/docs/#guides>
- the playlist for the basics with google storage - <https://www.youtube.com/watch?v=8DMOJ6Lgm7s&list=PLIivdWyY5sqJcBvDh5dfPoblLGhG1R1-O>
- How to Compress Files in Linux | Tar Command - <https://www.geeksforgeeks.org/tar-command-linux-examples>
- Time-based one-time password - https://en.wikipedia.org/wiki/Time-based_one-time_password

נספחים

הקובץ copyCatClient.py

```
'''
```

```
so there are three files of client (and constants).
```

```
client_comm - has all the socket stuff. its main class  
has two traits. incoming_msg and outgoing_msg.
```

```
this file will have a loop, if there is anything in  
outgoing it will send it and wait for a response.
```

```
this will happen in a thread.
```

```
on another thread, the gui will run. the GUI file will  
have the BOOP classes. it will have access to the
```

```
incoming_msg and outgoing_msg paremeters. it will put  
stuff in the outgoing according to the users actions.
```

```
so it will be the GUI that uses the BOOP classes to  
construct the messages.
```

```
this file will also call the other two. its the main  
client.
```

```
'''
```

```
import sys
```

```
import threading
```

```
import time
```

```
import copyCatClient_comm as comms
```

```
import copyCatGUI as gui
```

```
#this function will cause the comm file to send the
message in the outgoing_msg through the socket

def send_outgoing_msg(client_comms):

    if gui_thread.is_alive():

        client_comms.send()

client_lock = threading.Lock()

client_comms = comms.client_comm(client_lock)

gui_thread = threading.Thread(target = gui.start_gui,
args=(client_comms,)) #creates the GUI thread and starts
running it

gui_thread.start()

while gui_thread.is_alive():

    if client_comms.outgoing_msg:

        threading.Thread(target=send_outgoing_msg,args=(c
lient_comms,)).start() #as long as the user hasnt left
the GUI everytime there will be a message in outgoing_msg
there will be a thread created to send it

        time.sleep(0.5)

client_comms.disconnect()

sys.exit(0)
```

הקובץ copyCatClient_comm.py

```
import socket

import clientSideConstants as const

import ssl

class client_comm:

    def __init__(self,lock):

        self.incoming_msg = ""

        self.outgoing_msg = ""

        self.lock = lock

        self.client_socket = socket.socket() #creates a
socket

        context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)

        context.check_hostname = False

        context.verify_mode = ssl.CERT_NONE

        #wraps the socket in a protective SSL layer

        self.wrappedClient =
context.wrap_socket(self.client_socket,
server_side=False, server_hostname=const.server_ip)

        self.wrappedClient.connect((const.server_ip,
const.server_port))

#this function will send the message in outgoing_msg in
the socket and then receive the response from the server.
once the message was sent this function sets outgoing_msg
to be empty
```



```

def send(self):

    self.wrappedClient.send(self.outgoing_msg.encode(
))

    self.lock.acquire()

    self.outgoing_msg = ""

    self.lock.release()

    response = self.wrappedClient.recv(2048).decode()

    msg_header = BOOP_recieve(response)

    msg_start =
msg_header.get_everything_but_content_len()

    data_content_len = len(response)-msg_start

    while msg_header.get_content_length() >
data_content_len: #a loop that ensures that the client
catches all of the server's response

        response +=
self.wrappedClient.recv(msg_header.get_content_length() -
data_content_len).decode()

        data_content_len = len(response)-msg_start

    self.lock.acquire()

    self.incoming_msg = response

    self.lock.release()

#put a specific value in this parameter

def set_outgoing_msg(self, message):

    self.lock.acquire()

    self.outgoing_msg = message

```

```
        self.lock.acquire()

def get_outgoing_msg(self):
    return self.outgoing_msg

def reset_incoming(self):
    self.lock.acquire()
    self.incoming_msg = ""
    self.lock.acquire()

def get_incoming_msg(self):
    return self.incoming_msg

def disconnect(self):
    self.wrappedClient.shutdown(socket.SHUT_RDWR)
    self.wrappedClient.close()

def recieve(self):
    self.lock.acquire()
    self.incoming_msg =
self.wrappedClient.recv(1024).decode()
    self.lock.release()

#the boop receive class to get the value in content-
length. similar to the one in GUI with a few changes
```

```
class BOOP_recieve:

    def __init__(self,msg):

        self.message_len = len(msg)

        msg = msg.split("\r\n")

        self._start, self._header, self._content = msg


    def get_everything_but_content_len(self):

        return

    (self.get_header_length()+self.get_start_length()+len("\r\n\r\n"))


    def get_client_request_type(self):

        _,request_type = tuple(self._start.split())

        return request_type


    def get_content(self):

        return self._content


    def get_header_length(self):

        return len(self._header)


    def get_start_length(self):

        return len(self._start)


    def split_header(self):
```

```
        return self._header.split(";")

        #order: content type, content length, username,
        bucket id
```

```
def get_content_length(self):
    _,content_length = self.split_header()
    length = content_length.split(": ")
    _,content_length = length
    content_length = int(content_length)
    return content_length
```

הקובץ copyCatGUI.py

```
#library imports

from urllib import response

import customtkinter as tk

import threading

import time

import tempfile

import os

import zlib

import json

import base64

import pathlib

from PIL import Image

#project files import

from copyCatClient_comm import client_comm as comm

import clientSideConstants as const

from request_types import client_requests


class handle_backups:

    def __init__(self):

        pass


    #this func gets the following arguments: itself,
    backup data - the data from the server returns as very
    messy.
```

```
# there will be a function before this one that will
turn the contents of the server's response to a
dictionary,

# the key will be the file name (the name will be the
full path of the file) and the value will be the file
contents.

#path - will be where the program will download the
backup too.

def download_backup_to_user(self, backup_data):

    for key,filecontent in backup_data.items():

        filepath = os.path.join(const.goto_path,key)
#this will be the full path in the user's computer

        compressed_data =
base64.b64decode(filecontent.encode())

        file_data = zlib.decompress(compressed_data)

        try:

            pathlib.Path(os.path.dirname(filepath)).m
kdir(parents=True, exist_ok=True)

            with open(filepath,'wb') as f:

                f.write(file_data)

        except OSError as error:

            print("at except in download backup to
user")

            print(error)

#this will make a list of all the files in the
directory. all the files will be represented as their
full path
```

```
def make_file_list(self):  
    all_dir_list = []  
  
    for dir in const.top_dirs:  
        in_list =  
self.make_file_list2(dir,all_dir_list)  
  
        return all_dir_list  
  
  
def make_file_list2(self,current_path,file_list):  
    dir_tuple = next(os.walk(current_path))  
  
    _,folders,files = dir_tuple  
  
    for file in files:  
        file_path = os.path.join(current_path,file)  
  
        file_list.append(file_path)  
  
    for folder in folders:  
        new_path = os.path.join(current_path,folder)  
  
        self.make_file_list2(new_path,file_list)  
  
    return file_list  
  
  
#this function will create the dictionary from the  
computer data that will be sent to the server  
  
def compress_the_computer_data(self):  
    file_list = self.make_file_list()  
  
    computer_data = {}  
  
    for file in file_list:
```

```

        with open(file, 'rb') as f:

            content = f.read()

            compressed_data = zlib.compress(content)

            encoded_data =
base64.b64encode(compressed_data).decode()

            file_name = file

            for dir in const.top_dirs:

                if dir in file:

                    file_name = file[len(dir)+1:]

                    computer_data[file_name] = encoded_data

        return computer_data

```

#this class extracts the user's information from the
clientInfo file

```

class client_info:

    def __init__(self):

        info_list = []

        with open("clientInfo.txt", 'r') as f:

            for line in f:

                line_parts = line.split("=")

                _, value = line_parts

                if value[1:len(value)-1] == "0":

                    info_list.append(None)

            else:

```



```

        info_list.append(value[1:len(value)-
1])

        self.username,self.bucket_id,self.cookie =
info_list

def update_info(self):
    info_list = []
    with open("clientInfo.txt",'r') as f:
        for line in f:
            _,value = line.split("=")
            info_list.append(value[1::])
        self.username,self.bucket_id,self.cookie =
info_list

def set_cookie(self, cookie):
    with open("clientInfo.txt",'w') as f:
        f.write(f"username =
{self.username}\nbucket_id = {self.bucket_id}\ncookie =
{cookie}")

#this class will interpret the responses of the server
class BOOP_recieve:
    def __init__(self,message):
        message_parts = message.split("\r\n")
        self._start, self._header, self._content =
message_parts

```

```
def get_start(self):  
    return self._start  
  
def get_request_outcome(self):  
    start_parts = self._start.split(" ")  
    _, outcome = start_parts  
    return outcome  
  
def get_header(self):  
    return self._header  
  
def get_content(self):  
    return self._content  
  
def split_header(self):  
    return self._header.split(";")  
  
def split_content(self, key):  
    return self._content.split(key)  
  
def get_content_type(self):  
    content_type, _ = self.split_header()  
    return content_type
```

```
def get_content_length(self):  
    _,content_length = self.split_header()  
    return content_length  
  
class receive_signup_response(BOOP_recieve):  
    def __init__(self, message):  
        super().__init__(message)  
        self.info_dict = {}  
        if self.get_request_outcome() == "200 OK":  
            self.info_dict = json.loads(self._content)  
  
    def get_bucket_id(self):  
        return self.info_dict["bucket_id"]  
  
    def get_userName(self):  
        return self.info_dict["username"]  
  
    def get_cookie(self):  
        return self.info_dict["cookie"]  
  
    def check_if_failed(self):  
        outcome = self.get_request_outcome()  
        match outcome:
```

```

        case "200 OK": #if the server accepted the
sign up request the content of the server response will
contain the bucket_id and the username

```

```

        return
(self.get_bucket_id(),self.get_userName(),self.get_cookie
())

```

```

        case "401 Unauthorized":

```

```

            return False

```

```

        case "400 Bad Request":

```

```

            return False

```

```

class receive_access_response(BOOP_recieve):

```

```

    def __init__(self, message):

```

```

        super().__init__(message)

```

```

    #this function will extract the identity conformation
question from the server's response.

```

```

    def get_con_question(self):

```

```

        question,_ = self.split_content(";")

```

```

        question_parts = question.split(": ")

```

```

        _,question = question_parts

```

```

        return question

```

```

class receive_access_ans_response(BOOP_recieve):

```

```

    def __init__(self, message):

```

```

        super().__init__(message)

```

```

def get_con_answer_result(self):
    answer,_ = self.split_content(".")
    answer_parts = answer.split(":")
    _,answer = answer_parts
    return answer

def get_code_result(self):
    _,code = self.split_content(".")
    code_parts = code.split(":")
    _,code = code_parts
    return code

def get_cookie(self):
    if self.get_request_outcome() == "200 OK":
        cookie,_ = self.split_content(";")
        cookie_parts = cookie.split(": ")
        _,cookie = cookie_parts
        return cookie
    return "no cookie :("

class receive_download_response(BOOP_recieve):
    def __init__(self, message):
        super().__init__(message)

```

```
def get_file_dict(self):
    return json.loads(self._content)

class receive_getBucket_response(BOOP_recieve):
    def __init__(self, message):
        super().__init__(message)

    def get_backup_dict(self):
        return json.loads(self._content)

class receive_getBackup_response(BOOP_recieve):
    def __init__(self, message):
        super().__init__(message)

    def get_files_dict(self):
        file_dict = json.loads(self._content)
        return file_dict

class receive_getFile_response(BOOP_recieve):
    def __init__(self, message):
        super().__init__(message)

    def get_file_directory(self):
        directory,_ = self.split_content(": ")
        return directory
```

```
def get_file_data(self):

    content_parts = self._content.split(":", ",
maxsplit=1)

    _,data = content_parts

    data = json.loads(data)

    decoded_data = base64.b64decode(data.encode())

    decompressed_data = zlib.decompress(decoded_data)

    return decompressed_data


#this class will create requests to send to the server.
class BOOP_send:

    def __init__(self,request_enum):

        self.start = f"BOOP {request_enum.name}"

        self.header = ""

        self.content = ""


    def set_header(self):

        info = client_info()

        info.update_info()

        self.header = f"content-type: text; content-
length: {len(self.content)}; cookie: {info.cookie};
username: {info.username}; bucket-id: {info.bucket_id}"


    def send(self):
```

```

        return
self.start+"\r\n"+self.header+"\r\n"+self.content

class send_signup_request(BOOP_send):

    def __init__(self, content): #content here will be a
list of the values from the entries in the
regimessageation window

        super().__init__(client_requests.SIGNUP)

        self.start = f"BOOP {client_requests.SIGNUP}"

        content_headers = "username: Email: conQuestion1:
conAnswer1: conQuestion2: conAnswer2: conQuestion3:
conAnswer3:"

        for i,entry in enumerate(content):

            self.content += content_headers.split()[i]+"
"+entry+"; "

        self.header = f'content-type: text; content-
length: {len(self.content)}; cookie: {""} username:
{None}; bucket-id: {None};'

class send_getBucket_request(BOOP_send):

    def __init__(self):

        self.start = f"BOOP {client_requests.GETBUCKET}"

        self.content = "requests the data on the backups
within the bucket"

        self.set_header()

```



```
class send_getBackup_request(BOOP_send):

    def __init__(self, backup_num):

        self.start = f"BOOP {client_requests.GETBACKUP}"

        self.content = f'requests backup: {backup_num}'

        self.set_header()


class send_getFile_request(BOOP_send):

    def __init__(self, file_directory):

        self.start = f"BOOP {client_requests.GETFILE}"

        self.content = f"{file_directory}"

        self.set_header()


class send_upload_request(BOOP_send):

    def __init__(self, data):

        self.start = f"BOOP {client_requests.UPLOAD}"

        self.content = data

        self.set_header()


class send_download_request(BOOP_send):

    def __init__(self, backup_id):

        self.start = f"BOOP {client_requests.DOWNLOAD}"

        self.content = f'requests backup: {backup_id}'
#so that the client will ask for the download of a
specific backup

        self.set_header()
```

```

class send_access_request(BOOP_send):

    def __init__(self):

        self.start = f"BOOP {client_requests.ACCESS}"

        self.content = "requests user identity
conformation"

        self.set_header()

class send_access_ans_request(BOOP_send):

    def __init__(self, user_entry): #user entry will be a
tuple of: (con question answer, code)

        self.start = f"BOOP {client_requests.ACCESS_ANS}"

        answer,code = user_entry

        self.content = f"user answer:{answer};user
code:{code}"

        self.set_header()

# a screen that shows the user the registration was
successfull

class registration_successfull(tk.CTkToplevel):

    def __init__(self, master = None, *args, **kwargs):

        super().__init__(master, *args, **kwargs)

        self.geometry(const.success)

        self.root = master

        self.label = tk.CTkLabel(self, text="user
registration was successfull!", text_color="purple")

```

```
self.label.pack()

#the registration window will recieve the user's
information and send them to the server

class registration_window(tk.CTkToplevel):

    def __init__(self, comms, master=None,*args,
**kwargs):

        super().__init__(master, *args, **kwargs)

        self.geometry(const.registrate)

        #this window will open the homescreen after
finishing the registration

        self.home_page = None

        self.success_window = None

        self.root = master

        #labels

        self.label1 = tk.CTkLabel(self, text="Hello user!
\rto register to COPYCAT you will need to fill the
following fields:", text_color="purple")

        self.label2 = tk.CTkLabel(self, text="username:
", text_color="purple")

        self.label3 = tk.CTkLabel(self, text="email
address: ", text_color="purple")

        self.label4 = tk.CTkLabel(self,
text="conformation question 1", text_color="purple")
```

```
        self.label5 = tk.CTkLabel(self,
text="conformation answer 1", text_color="purple")

        self.label6 = tk.CTkLabel(self,
text="conformation question 2", text_color="purple")

        self.label7 = tk.CTkLabel(self,
text="conformation answer 2", text_color="purple")

        self.label8 = tk.CTkLabel(self,
text="conformation question 3", text_color="purple")

        self.label9 = tk.CTkLabel(self,
text="conformation answer 3", text_color="purple")


#entries

self.entry1 = tk.CTkEntry(self) #username
self.entry2 = tk.CTkEntry(self) #phone number
self.entry3 = tk.CTkEntry(self) #con question 1
self.entry4 = tk.CTkEntry(self) #con answer 1
self.entry5 = tk.CTkEntry(self) #con question 2
self.entry6 = tk.CTkEntry(self) #con answer 2
self.entry7 = tk.CTkEntry(self) #con question 3
self.entry8 = tk.CTkEntry(self) #con answer 3


#button

self.enter_button = tk.CTkButton(self,text="sign
up", command=lambda: self.send_to_server(comms),
fg_color="hot pink", hover_color="light pink")


#placements
```

```
self.label1.pack(padx=20,pady=20) #init message
self.label2.pack() #username:
self.entry1.pack() #enter username
self.label3.pack() #phone number:
self.entry2.pack() #enter phone number
self.label4.pack() #con question 1
self.entry3.pack() #con question 1
self.label5.pack() #con answer 1
self.entry4.pack() #con answer 1
self.label6.pack() #con question 2
self.entry5.pack() #con question 2
self.label7.pack() #con answer 2
self.entry6.pack() #con answer 2
self.label8.pack() #con question 3
self.entry7.pack() #con question 3
self.label9.pack() #con answer 3
self.entry8.pack() #con answer 3
self.enter_button.pack(padx=20,pady=20)
```

```
def destroy_window(self):
    self.quit()
```

#this function extracts the user's information from the entries and puts them in a request that will be sent to the server

```

def send_to_server(self, comms):

    user_info =
[self.entry1.get(),self.entry2.get(),self.entry3.get(),se
lf.entry4.get(),self.entry5.get(),self.entry6.get(),self.
entry7.get(),self.entry8.get()]

    comms.reset_incoming()

    comms.set_outgoing_msg(send_signup_request(user_i
nfo).send())

    while comms.get_incoming_msg() == "":

        time.sleep(0.1)

    response =
receive_signup_response(comms.get_incoming_msg())

    if response.check_if_failed() == False:

        self.label2.configure(text = "the username
you entered is already taken, please enter another: ")

    else:

        bucket_id, username, cookie =
response.check_if_failed()

        with open("clientInfo.txt",'w') as f:

            f.write(f"username =
{username}\nbucket_id = {bucket_id}\ncookie = {cookie}")

        comms.reset_incoming()

        if self.success_window is None or not
self.success_window.winfo_exists():

            self.success_window =
registration_successfull(self.root) # create window if
its None or destroyed

```

```

        else:

            self.success_window.focus()

            self.iconify()

            #this bit will open the home screen and close
the registration screen

            self.destroy() #closes the current window
after opening another

class user_conformation_successfull(tk.CTkToplevel):

    def __init__(self, master = None, *args, **kwargs):

        super().__init__(master, *args, **kwargs)

        self.geometry(const.confirmation_success)

        self.root = master

        self.label = tk.CTkLabel(self, text="the identity
confirmation was successfull!", text_color="purple")

        self.label.pack()

#the identity conformation window! - the question from
the data base

class user_conformationQ(tk.CTkToplevel):

    def __init__(self, comms, master = None, *args,
**kwargs):

        super().__init__(master, *args, **kwargs)

        self.root = master

        self.geometry(const.confirmation)

        comms.reset_incoming()

```

```
comms.set_outgoing_msg(send_access_request().send
())

while comms.get_incoming_msg() == "":

    time.sleep(0.1)

    question =
receive_access_response(comms.get_incoming_msg()).get_con
_question()

#the window this will open

self.user_conformationEmail = None

#labels

self.label1 = tk.CTkLabel(self, text="answer the
following question:", text_color="purple")

self.label2 = tk.CTkLabel(self, text=question,
text_color="purple")

#entry

self.entry1 = tk.CTkEntry(self)

#button

self.button1 = tk.CTkButton(self, text="enter",
command=lambda: self.enter(comms), fg_color="hot pink",
hover_color="light pink")

#grid

self.label1.grid(row=0, column=0, padx=20,
pady=20)

self.label2.grid(row=1, column=0, padx=20)

self.entry1.grid(row=2, column=0, padx=20,
pady=20)
```



```
        self.button1.grid(row=3, column=0, padx=20,
pady=20)

#will pass the info in the entry forward to the email
confirmation screen

    def enter(self, comms):

        if self.user_conformationEmail is None or not
self.user_conformationEmail.winfo_exists():

            self.user_conformationEmail =
user_conformationEmail(comms, self.entry1.get(),
self.master) # create window if its None or destroyed

        else:

            self.user_conformationEmail.focus()

            self.iconify()

            self.destroy() #closes the current window after
opening another

#the identity conformation window! - the code sent to the
email address

class user_conformationEmail(tk.CTkToplevel):

    def __init__(self, comms, q_answer, master = None,
*args, **kwargs):

        super().__init__(master, *args, **kwargs)

        self.geometry(const.confirmation)

        self.root = master

        self.q_answer = q_answer
```

```
#the window this will open

self.home_page = None

self.confirmationQ = None

self.success_window = None

#labels

self.label1 = tk.CTkLabel(self, text="put in the
code sent to your email:", text_color="purple")

#entry

self.entry1 = tk.CTkEntry(self)

#button

self.button1 = tk.CTkButton(self, text="enter",
command=lambda: self.enter(comms), fg_color="hot pink",
hover_color="light pink")

#grid

self.label1.grid(row=0, column=0, padx=20,
pady=20)

self.entry1.grid(row=2, column=0, padx=20,
pady=20)

self.button1.grid(row=3, column=0, padx=20,
pady=20)

#puts the users entry from this window and the preivious
one and puts it in a message to send to the server

def enter(self, comms):

    answers = (self.q_answer, self.entry1.get())

    comms.reset_incoming()
```

```

        comms.set_outgoing_msg(send_access_ans_request(answers).send())

        while comms.get_incoming_msg() == "":

            time.sleep(0.1)

            response =
receive_access_ans_response(comms.get_incoming_msg())

            if response.get_request_outcome() == "200 OK":

                info = client_info()

                info.set_cookie(response.get_cookie())

                if self.success_window is None or not
self.success_window.winfo_exists():

                    self.success_window =
user_conformation_successfull(self.root) # create window
if its None or destroyed

                else:

                    self.confirmationQ.focus()

                    self.iconify()

                #will return the user to the home page

                self.destroy() #closes the current window
after opening another

            else:

                user_answer_result =
response.get_con_answer_result()

                user_code_result = response.get_code_result()

                if user_answer_result and not
user_code_result:

```

```

        self.label1.configure(text = "the code
youve entered is wrong. please try reentering it")

        if not user_answer_result:

            if self.confirmationQ is None or not
self.confirmationQ.winfo_exists():

                self.confirmationQ =
user_conformationQ(comms, self.root) # create window if
its None or destroyed

            else:

                self.confirmationQ.focus()

                self.iconify()

                self.destroy() #closes the current window
after opening another

class upload_successfull(tk.CTkToplevel):

    def __init__(self, master = None, *args, **kwargs):

        super().__init__(master, *args, **kwargs)

        self.geometry(const.success)

        self.root = master

        self.label = tk.CTkLabel(self, text="the upload
was successfull!", text_color="purple")

        self.label.pack()

#the 'uploading data' window

class upload_backup_window(tk.CTkToplevel):

    def __init__(self, comms, master = None, *args,
**kwargs):

```

```
super().__init__(master, *args, **kwargs)

self.geometry(const.upload_backup)

self.h_upload = handle_backups()

self.root = master

#windows

self.conformation_window = None

self.home_page = None

self.registration_window = None

self.success_window = None


        self.label = tk.CTkLabel(self, text="uploading
your computer's data to the cloud...",
text_color="purple")

        self.label.pack(padx=20, pady=20)


        compress_dict =
self.h_upload.compress_the_computer_data()

        comms.reset_incoming()

        comms.set_outgoing_msg(send_upload_request(json.d
umps(compress_dict)).send())

        while comms.get_incoming_msg() == "":

            time.sleep(0.1)


        #todo: add error handeling and close this window
once its done
```

```
server_response =
BOOP_recieve(comms.get_incoming_msg())

    if server_response.get_request_outcome() == "401
Unautherized":

        self.go_to_con(comms)

        time.sleep(0.5)

    elif server_response.get_request_outcome() ==
"400 Bad Request":

        comms.reset_incoming()

        if self.registration_window is None or not
self.registration_window.winfo_exists():

            self.registration_window =
registration_window(comms, self.root) # create window if
its None or destroyed

        else:

            self.registration_window.focus()

            self.iconify()

    if server_response.get_request_outcome() == "200
OK":

        comms.reset_incoming()

        if self.success_window is None or not
self.success_window.winfo_exists():

            self.success_window =
upload_successfull(self.root) # create window if its
None or destroyed

        else:

            self.success_window.focus()
```

```
        self.iconify()

        self.go_back()

    def
call_compress_computer_data(self, compressed_dict):

        compressed_dict =
self.h_upload.compress_the_computer_data()

    def go_back(self):

        self.destroy() #closes the current window after
opening another

    def go_to_con(self, comms):

        #will return the user to the home page

        if self.conformation_window is None or not
self.conformation_window.wininfo_exists():

            self.conformation_window =
user_conformationQ(comms, self.root) # create window if
its None or destroyed

        else:

            self.conformation_window.focus()

            self.iconify()

            self.destroy() #closes the current window after
opening another

class download_successfull(tk.CTkToplevel):

    def __init__(self, master = None, *args, **kwargs):
```

```
super().__init__(master, *args, **kwargs)

self.geometry(const.download_success)

self.root = master

self.label = tk.CTkLabel(self, text=f"the backup
was downloaded to: {const.goto_path}",
text_color="purple")

self.label.pack()

#the 'downloading data' window

class download_backup_window(tk.CTkToplevel):

    def __init__(self, backup_id, comms, master = None,
*args, **kwargs):

        super().__init__(master, *args, **kwargs)

        self.geometry(const.upload_backup)

        self.h_backups = handle_backups()

        self.root = master

        #windows

        self.conformation_window = None

        self.home_page = None

        self.success_window = None

        self.label = tk.CTkLabel(self, text="loading the
backup to your computer", text_color="purple")

        self.label.pack(padx=20, pady=20)
```



```
comms.reset_incoming()

comms.set_outgoing_msg(send_download_request(back
up_id).send())

while comms.get_incoming_msg() == "":

    time.sleep(0.1)

    self.response =
receive_download_response(comms.get_incoming_msg())

    if self.response.get_request_outcome() == "200
OK":

        self.data = self.response.get_file_dict()

        self.call_download_backup()

        comms.reset_incoming()

        if self.success_window is None or not
self.success_window.wininfo_exists():

            self.success_window =
download_successfull(self.root) # create window if its
None or destroyed

        else:

            self.success_window.focus()

            self.iconify()

            self.go_back()

def call_download_backup(self):

    self.h_backups.download_backup_to_user(self.data)
```

```
def go_back(self):

    self.destroy() #closes the current window after
opening another

def go_to_con(self, comms):

    #will return the user to the home page

    if self.conformation_window is None or not
self.conformation_window.winfo_exists():

        self.conformation_window =
user_conformationQ(comms, self) # create window if its
None or destroyed

    else:

        self.conformation_window.focus()

        self.iconify()

        self.destroy() #closes the current window after
opening another

#open bucket window - allows the user to see the backups
he has saved

class open_bucket_then_backup(tk.CTkToplevel):

    def __init__(self, comms, master=None, *args,
**kwargs):

        super().__init__(master=master, *args, **kwargs)

        self.geometry(const.open_backup)

        self.root = master
```

```
#the windows this window might open

self.open_file_window = None

self.download_backup_window = None

self.home_page = None

self.conformation_window = None

self.registration_window = None


comms.set_outgoing_msg(send_getBucket_request().s
end())

while comms.get_incoming_msg() == "":

    time.sleep(0.1)

    response =
receive_getBucket_response(comms.get_incoming_msg())

    if response.get_request_outcome() == "401
Unautherized":

        self.go_to_con(comms)

        time.sleep(0.5)

    elif response.get_request_outcome() == "400 Bad
Request":

        if self.registration_window is None or not
self.registration_window.wininfo_exists():

            self.registration_window =
registration_window(comms, self.root) # create window if
its None or destroyed

        else:

            self.registration_window.focus()
```

```
        self.iconify()

    else:

        #frames

        self.bucket_data_frame =
tk.CTkScrollableFrame(master=self,
width=const.bucket_data_rep[0],
height=const.bucket_data_rep[1])

        self.current_backup_frame =
tk.CTkFrame(master=self, width=const.chosen_backup[0],
height=const.chosen_backup[1]) #the current backup the
user has clicked on

        self.buttons = tk.CTkFrame(master=self,
width=const.OB_buttons[0], height=const.OB_buttons[1])

        #buttons

        self.button1 =
tk.CTkButton(self.buttons,text="go back", command=lambda:
self.go_back(comms), fg_color="hot pink",
hover_color="light pink")

        self.button2 =
tk.CTkButton(self.buttons,text="download current backup",
command= lambda: self.download_current_backup(comms),
fg_color="hot pink", hover_color="light pink")

        self.button3 =
tk.CTkButton(self.buttons,text="open backup",
command=lambda: self.open_backup(comms), fg_color="hot
pink", hover_color="light pink")

        #grid locations - frames
```

```
        self.bucket_data_frame.grid(row=0, column=0,
padx=20, pady=20)

        self.current_backup_frame.grid(row=1,
column=0, padx=20, pady=20)

        self.buttons.grid(row=2, column=0, padx=20,
pady=20)

        #grid locations - buttons

        self.button1.grid(row=0, column=0, padx=20,
pady=20)

        self.button2.grid(row=0, column=1, padx=20,
pady=20)

        self.button3.grid(row=0, column=2, padx=20,
pady=20)

        #secret current_backup label

        self.current_backup =
tk.CTkLabel(self.current_backup_frame, text= f"")

        self.current_backup.grid(row=0, column=0,
rowspan = 4, padx=20, pady=20)

        #the big bad information table

        data_dict = response.get_backup_dict()

        lable_list = []

        num_list = []

        #label and button creation

        for i, (num, list) in
enumerate(data_dict.items()):

            upload_date, bytes_in_backup = list
```

```
        label_text = f"back up upload date:
{upload_date}, no of bytes in the backup:
{bytes_in_backup}"

        label =
tk.CTkLabel(self.bucket_data_frame, text= label_text,
text_color="purple")

        label.grid(row=i, column=0, padx=20,
pady=20)

        lable_list.append(label)

        num_list.append(num)

        button =
self.create_button(num,label.cget("text"))

        button.grid(row=i, column=1,padx=20,
pady=20)


def select_backup(self, serial_num, display_text):

    self.current_backup.configure(text= f"backup
no.{serial_num},"+f" "+display_text, text_color="purple")


def create_button(self,number,text):

    return
tk.CTkButton(self.bucket_data_frame,text=f"backup
no.{number}", command=lambda: self.select_backup(number,
text), fg_color="hot pink", hover_color="light pink")


def go_back(self, comms):

    comms.reset_incoming()
```

```
        self.destroy() #closes the current window after
opening another
```

```
def go_to_con(self, comms):

    comms.reset_incoming()

    #will return the user to the home page

    if self.conformation_window is None or not
self.conformation_window.wininfo_exists():

        self.conformation_window =
user_conformationQ(comms, self.root) # create window if
its None or destroyed

        self.conformation_window.deiconify()

    else:

        self.conformation_window.focus()

        self.iconify()

        self.destroy() #closes the current window after
opening another
```

```
def download_current_backup(self, comms):

    comms.reset_incoming()

    #this will open the 'download backup window'

    try:

        backup_id = self.get_current_backup_id()

        if self.download_backup_window is None or not
self.download_backup_window.wininfo_exists():
```

```
        self.download_backup_window =
download_backup_window(backup_id,comms, self) # create
window if its None or destroyed

        self.download_backup_window.deiconify()

    else:

        self.download_backup_window.focus()

        self.iconify()

        self.destroy() #closes the current window
after opening another

    except:

        return
```

```
def get_current_backup_id(self):

    label_text = self.current_backup.cget("text")

    label_parts = label_text.split(",")

    backup_id,_,_ = label_parts

    backup_id_parts = backup_id.split(".")

    _,backup_id = backup_id_parts

    return backup_id
```

```
def open_backup(self, comms):

    try:

        backup_id = self.get_current_backup_id()

        #this will open the 'open file' window with
the data of the chosen backup.
```



```
        if self.open_file_window is None or not
self.open_file_window.winfo_exists():

            comms.reset_incoming()

            self.open_file_window =
open_file(backup_id, comms, self) # create window if its
None or destroyed

            self.open_file_window.deiconify()

        else:

            self.open_file_window.focus()

            self.iconify()

            self.destroy() #closes the current window
after opening another

    except:

        return

#this window will help the user open a specific file from
a specific backup to look at, there will be no way of
editing said file

class open_file(tk.CTkToplevel):

    def __init__(self, backup_id, comms, master = None,
*args, **kwargs): #backup id is a parameter through which
open_file will figure out which backup to show the user.

        super().__init__(master, *args, **kwargs)

        self.geometry(const.open_file)

        #the windows this window might open

        self.home_page = None
```

```
self.conformation_window = None

#frames

self.backup_data_frame =
tk.CTkScrollableFrame(master=self,
width=const.backup_data_rep[0],
height=const.backup_data_rep[1])

self.current_file = tk.CTkFrame(master=self,
width=const.chosen_file[0], height=const.chosen_file[1])
#the current file the user has clicked on

self.buttons = tk.CTkFrame(master=self,
width=const.OF_buttons[0], height=const.OF_buttons[1])

#buttons

self.button1 = tk.CTkButton(self.buttons, text="go
back", command=lambda: self.go_back(comms), fg_color="hot
pink", hover_color="light pink")

self.button2 =
tk.CTkButton(self.buttons, text="open file",
command=lambda: self.open_file(comms), fg_color="hot
pink", hover_color="light pink")

#grid locations - frames

self.backup_data_frame.grid(row=0, column=0,
padx=20, pady=20)

self.current_file.grid(row=1, column=0, padx=20,
pady=20)

self.buttons.grid(row=2, column=0, padx=20,
pady=20)
```

```

        #grid locations - buttons

        self.button1.grid(row=0, column=0,padx=20,
pady=20)

        self.button2.grid(row=0, column=2,padx=20,
pady=20)

        #the information table

        comms.set_outgoing_msg(send_getBackup_request(bac
kup_id).send())

        while comms.get_incoming_msg() == "":

            time.sleep(0.1)

        response =
receive_getBackup_response(comms.get_incoming_msg())

        if response.get_request_outcome == "401
Unauthorized":

            self.go_to_con(comms)

            data_dict = response.get_files_dict()

            self.buttons_list = []

            self.label_list = []

            #label and button creation

            self.current_file_label =
tk.CTkLabel(self.current_file, text= f"")

            self.current_file_label.grid(row=0, column=0,
rowspan = 4, padx=20, pady=20)

            self.file_showing_result =
tk.CTkLabel(self.current_file, text= f"")

```

```
        self.file_showing_result.grid(row=1, column=0,
rowspan = 4, padx=20, pady=20)

        self.k = 0

        first_dir = next(iter(data_dict.keys()))

        self.create_data_frame(data_dict, first_dir)

        comms.reset_incoming()

        #this func will take the chosen file and open it for
the user to view

        def open_file(self, comms):

            comms.reset_incoming()

            try:

                file_info =
self.current_file_label.cget("text")

                file_info_parts = file_info.split(", ")

                _, file_directory = file_info_parts

                request =
send_getFile_request(file_directory)

                comms.set_outgoing_msg(request.send())

                while comms.get_incoming_msg() == "":

                    time.sleep(0.1)

                response =
receive_getFile_response(comms.get_incoming_msg())

                file_data = response.get_file_data()

                with tempfile.NamedTemporaryFile(mode="wb",
delete=False) as f:
```

```
f.write(file_data)

f.close()

try:

    os.startfile(f.name)

    time.sleep(10)

except:

    self.file_showing_result.configure(text="the program couldnt open the file, we will download it to your computer for you to open manually")

    _,file_name =
os.path.split(file_directory)

    new_dir =
os.path.join("C:\\*userprofile*\\Downloads",file_name)

    try:

        os.makedirs(new_dir,
exist_ok=True)

        with open(new_dir,'w') as f:

            f.write(file_data)

            self.file_showing_result.configure(text="the download succeeded, you can open the file from your downloads.")

    except OSError as error:

        self.file_showing_result.configure(text=f"there has been an error in the download process: \r{error}")

except:

    return
```

#creates the labels and buttons that show the user's
the diffirent files in the backups

```
def create_data_frame(self, data_dict, directory1):

    for directory, list in data_dict.items():

        for file in list:

            if isinstance(file,dict):

                new_dir =
os.path.join(directory1,next(iter(file.keys())))

                self.create_data_frame(file, new_dir)

            else:

                label_text = f"{directory1}\\{file}"

                label =
tk.CTkLabel(self.backup_data_frame, text= label_text,
text_color="purple")

                label.grid(row=self.k, column=0,
padx=20, pady=20)

                self.label_list.append(label)

                button =
self.create_button(file,label_text)

                button.grid(row=self.k,
column=1,padx=20, pady=20)

                self.buttons_list.append(button)

                self.k += 1

        return

def create_button(self,file,label_text):
```

```

        return

tk.CTkButton(self.backup_data_frame, text=f"{file}",
command=lambda: self.select_file(file, label_text),
fg_color="hot pink", hover_color="light pink")

#the function that puts the information of the file
the user clicked on in the current file frame

def select_file(self, file_name, display_text):

    self.current_file_label.configure(text=
f"{file_name}, "+f" "+display_text, text_color="purple")

def go_to_con(self, comms):

    comms.reset_incoming()

    #will return the user to the home page

    if self.conformation_window is None or not
self.conformation_window.winfo_exists():

        self.conformation_window =
user_conformationQ(comms, self) # create window if its
None or destroyed

    else:

        self.conformation_window.focus()

        self.iconify()

        self.destroy() #closes the current window after
opening another

#this func will go back to the 'open backup window'

def go_back(self, comms):

```

```
        comms.reset_incoming()

        self.destroy() #closes the current window after
opening another

#the homepage

# three buttons and the logo

class homePage(tk.CTk):

    def __init__(self, comms):

        comms.reset_incoming()

        super().__init__()

        self.geometry(const.homepage)

        self.title("COPYCAT mainpage")

        tk.set_appearance_mode("light")

        #these are the windows the homepage can direct
too

        self.create_backup_window = None

        self.view_bucket_window = None

        self.registration_window = None

        self.info = client_info()

        self.protocol('WM_DELETE_WINDOW',
self.destroy_window) # root is your root window

        # frames

        self.logo = tk.CTkFrame(master=self,
width=const.HM_logo[0], height=const.HM_logo[1]) #i COULD
```


change this to be just a photo, this doesnt have to be a frame

```
        self.buttons = tk.CTkFrame(master=self,
width=const.HM_buttons[0], height=const.HM_buttons[1])

        self.logo.grid(row=0, column=1, padx=20, pady=20)

        self.buttons.grid(row=1, column=1, padx=20,
pady=20)

        #buttons

        self.button =
tk.CTkButton(self.buttons,text="backup current data",
command=lambda: self.upload_current_data(comms),
fg_color="hot pink", hover_color  ="light pink")

        self.button.grid(row=0, column=0, padx=20,
pady=10)

        self.button =
tk.CTkButton(self.buttons,text="view bucket",
command=lambda: self.view_bucket(comms), fg_color="hot
pink", hover_color  ="light pink")

        self.button.grid(row=2, column=0, padx=20,
pady=10)

        #image

        logo_image =
tk.CTkImage(light_image=Image.open("copyCatLogo.png"),siz
e=(200,200))

        self.image_label =
tk.CTkLabel(self.logo,text="",image=logo_image)

        self.image_label.grid(row=0, column=0, padx=20,
pady=20)
```

```
        if self.info.username == None or
self.info.bucket_id == None:

            threading.Thread(target=self.open_registration, args=(comms,)).start()

def destroy_window(self):

    self.quit()

def view_bucket(self, comms):

    #this will open the 'open backup window'

    if self.view_bucket_window is None or not
self.view_bucket_window.wininfo_exists():

        self.view_bucket_window =
open_bucket_then_backup(comms,self) # create window if
its None or destroyed

    else:

        self.view_bucket_window.focus()

        self.iconify() #closes the current window after
opening another

def open_registration(self, comms):

    if self.info.bucket_id == None or
self.info.username == None:

        if self.registration_window is None or not
self.registration_window.wininfo_exists():
```

```
        self.registration_window =
registration_window(comms) # create window if its None
or destroyed

        else:

            self.registration_window.focus()

            self.iconify()

        else:

            return

#to open up the 'uploading data' window

def upload_current_data(self, comms):

    #this will open the loading bar window

    if self.create_backup_window is None or not
self.create_backup_window.wininfo_exists():

        self.create_backup_window =
upload_backup_window(comms, self) # create window if its
None or destroyed

    else:

        self.create_backup_window.focus()

        comms.reset_incoming()

def start_gui(comms):

    root = homePage(comms) #make it an init requeriment

    root.mainloop()
```

הקובץ clientSideConstants.py

```
goto_path = "C:\\Users\\User\\Downloads" #the location
within the user computer that the program will download
stuff to

top_dirs = ["C:\\Users\\amiaz\\Documents\\example data"]
#the directories that will be downloaded and saved in the
server

#gui constants

#home page:

homepage = "300X300"

homepage_measurements = (300,300)

hm_width,hm_height = homepage_measurements

previous_backups = (hm_width/2,hm_height-40) #width,
height

HM_logo = (200,200) #width, height

HM_buttons = (200,200) #width, height

#upload backup & download backup (there is no reason
those two should have diffirent sizes)

upload_backup = "500X500"

upload_measurements = (500,500)

#open backup

open_backup = "800x500"
```

```
open_backup_measurements = (700,400)
```

```
bucket_data_rep = (700,200)
```

```
chosen_backup = (700,200)
```

```
OB_buttons = (700,100)
```

```
#open file
```

```
open_file = "600x600"
```

```
open_file_measurement = (800,600)
```

```
backup_data_rep = (500,300)
```

```
chosen_file = (500,200)
```

```
OF_buttons = (500,100)
```

```
#registration
```

```
registrate = "500x700"
```

```
registrate_measurements = (500,700)
```

```
#confirmation - the two conformation windows will have  
the same measurements
```

```
confirmation = "400X300"
```

```
conformation_measurements = (400,300)
```

```
#success windows - the success windows will also have the  
same measures(where there isnt too much text)
```

```
success = "200x50"
```

```
download_success = "400x50"
```

```
confirmation_success = "300x50"
```

```
#client constants
```

```
#BOOP
```

```
example_server_response = "BOOP 200 OK\r\ncontent-type:  
zip; content-length: Z; server-hash:  
somethingsomething\r\nwhatever is written when you pass a  
zip file."
```

```
#server ip
```

```
server_ip = '10.168.0.95'
```

```
#server port
```

```
server_port = 1729
```

הקובץ request_type.py

```
from enum import Enum
```

```
#enum of all the requests the client can make.
```

```
class client_requests(Enum):
```

```
    ACCESS = 1
```

```
    ACCESS_ANS = 2
```

```
    DOWNLOAD = 3
```

```
    GETBUCKET = 4
```

```
    GETBACKUP = 5
```

```
    GETFILE = 6
```

```
    SIGNUP = 7
```

```
    UPLOAD = 8
```

```
'''
```

little meaning dictionary because i genuinly forget what they mean:

ACCESS - a request to recieve the con question from the server as well as a request for the server to send the email with the con code

ACCESS_ANS - for answering the conformation question.
both the email code and the con question.

DOWNLOAD - for asking the server to send to the client the information of a specific backup so that it could be downloaded to the users computer

GETBUCKET - a request for the list/datarep of the users whole bucket

GETBACKUP - a request for the data rep for a specific backup

GETFILE - a request for the information of a single file for the user to view

SIGNUP - this signifies to the server that a new user is signing up and that the packet will contain their information for it to be stored in the database

UPLOAD - a request from the client to upload a backup.

'''

הקובץ copyCatServer.py

```
import threading

import serverSideConstants as const

import copyCatHandle as handle

import copyCatServer_comm as comm

import time

'''

so what will happen is that the common parameters that
are shared in the three server files are:

incoming_msgs = a list of tuples, (client_socket,
"clients request")

outgoing_msgs = a list of tuples, (client_socket, "server
response")

this is the main server side file. it will run the other
parts and will have two loops. each going over the two
lists,

if the lists arent empty they will send the messages (if
its the outgoing), or call handle_client (if its the
incoming).

the server_comm will run the loop of waiting for new
clients and the select of making a list of them and
waiting

for messages on one thread. the handle_client will run on
the same file as the DB runner - and another thread.

so that the handling of the clients wont slow the
accepting of new ones.
```

```
'''
```

```
def send_outgoing_msg(client_comm,
client_socket,response):

    client_comm.send_message(client_socket,response)

comm_lock = threading.Lock()

client_comm = comm.server_comm(comm_lock)

comm_thread =
threading.Thread(target=client_comm.comms).start()

while True:

    if client_comm.incoming_msgs:

        for client_socket, message in
client_comm.incoming_msgs:

            threading.Thread(target=handle.handle_client,
args=(client_socket,message,client_comm)).start()

            client_comm.remove_from_incoming(client_socket, message)

        if client_comm.outgoing_msgs:

            for client_socket,response in
client_comm.outgoing_msgs:

                threading.Thread(target=send_outgoing_msg,args=
(client_comm,client_socket,response)).start()

            time.sleep(0.5)
```

הקובץ copyCatServer_comm.py

```
import select

import socket

import threading

import serverSideConstants as const

import uuid

import time

import ssl

import time


class server_comm:

    def __init__(self, lock):

        self.incoming_msgs = []

        self.outgoing_msgs = []

        self.lock = lock

        self.server = socket.socket()

        #the creation of an ssl layer to wrap the socket

        context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)

        context.load_cert_chain('C:\\Users\\amiaz\\cert.pem', 'C:\\Users\\amiaz\\key.pem')

        self.wrappedSocket =

context.wrap_socket(self.server, server_side=True)

        self.wrappedSocket.bind(('0.0.0.0', const.server_port))

        self.wrappedSocket.listen()

        self.clients = []
```

```
self.messages = []

def comms(self):
    while True:
        try:
            rlist,wlist,xlist =
select.select([self.wrappedSocket]+self.clients,self.clie
nts,[])

            except ValueError as e:
                print(f"\n{e}\n")

            for client in rlist:
                if client is self.wrappedSocket:
                    c,address = client.accept()
                    self.clients.append(c)
                else:
                    data = ''
                    data =
client.recv(const.max_msg_length).decode()

                    if data == False or data == None:
                        self.clients.remove(client)
                        client.close()

                    time.sleep(0.01)

                try:
                    msg_header = BOOP_recieve(data)

                    if msg_header != None:
```

```
        msg_start =
msg_header.get_everything_but_content_len()

        data_content_len = len(data) -
msg_start

        while
msg_header.get_content_length() > data_content_len:

            data +=
client.recv(msg_header.get_content_length() -
data_content_len).decode()

            data_content_len =
len(data) - msg_start

        except:

            print("failed at taking apart
user's message")

            if data == "":

                self.clients.remove(client)

                client.close()

            else:

                self.messages.append((client,
data))

        for message in self.messages:

            current_socket, data = message

            if current_socket in wlist:
```

```
#this is where the function
handle_client (which handles the different client request
and has the switch case) is called.
```

```
        self.lock.acquire()

        self.incoming_msgs.append((current_socket,data))

        self.lock.release()

        self.messages.remove(message)

        time.sleep(0.5)
```

```
#deletes a specific request from incoming_msgs and
adds to outgoing_msgs a response
```

```
def add_response(self, client_socket, request,
response):

    self.lock.acquire()

    if (client_socket,request) in self.incoming_msgs:
        self.incoming_msgs.remove((client_socket,request))

    self.outgoing_msgs.append((client_socket,response))

    self.lock.release()
```

```
#sends a specific message and deletes it from
outgoing_msgs
```

```
def send_message(self,client_socket,response):

    if (client_socket,response) in
self.outgoing_msgs:
```

```
        client_socket.send(response.encode())

        self.lock.acquire()

        self.outgoing_msgs.remove((client_socket, response))

        self.lock.release()
```

```
#sends all the messages in outgoing_msgs
```

```
    def send_messages(self):

        for client_socket, response in self.outgoing_msgs:

            client_socket.send(response.encode())

            self.lock.acquire()

            self.outgoing_msgs.remove((client_socket, response))

            self.lock.release()
```

```
#removes a specific request from incoming_msgs
```

```
    def remove_from_incoming(self, client_socket, request):

        if (client_socket, request) in self.incoming_msgs:

            self.lock.acquire()

            self.incoming_msgs.remove((client_socket, request))

            self.lock.release()
```

```
#the boop receive class, similar to the one in copyCatHandle but has a few differences, the use of this
```

class here is to extract the value of the content-length of the client's requests

```
class BOOP_recieve:

    def __init__(self,msg):

        self.message_len = len(msg)

        if self.message_len > 0:

            msg = msg.split("\r\n")

            self._start, self._header, self._content =
msg

            return None

        def get_everything_but_content_len(self):

            return
            (self.get_header_length()+self.get_start_length()+len("\r\n\r\n"))

        def get_client_request_type(self):

            _,request_type = tuple(self._start.split())

            return request_type

        def get_content(self):

            return self._content

        def get_header_length(self):

            return len(self._header)
```



```
def get_start_length(self):  
    return len(self._start)  
  
def split_header(self):  
    return self._header.split(";")  
  
#order: content type, content length, username,  
bucket id  
  
def get_content_type(self):  
    content_type,_,_,_ = self.split_header()  
    type = content_type.split(": ")  
    _,content_type = type  
    return content_type  
  
def get_content_length(self):  
    _,content_length,_,_,_ = self.split_header()  
    length = content_length.split(": ")  
    _,content_length = length  
    content_length = int(content_length)  
    return content_length  
  
def get_cookie(self):  
    if self.get_client_request_type() ==  
"client_requests.SIGNUP" or  
self.get_client_request_type() ==  
"client_requests.ACCESS_ANS":
```

```
        return None

    _,_,cookie,_,_ = self.split_header()

    try:

        cookie_parts = cookie.split(": ")

        _,cookie = cookie_parts

        cookie = uuid.UUID(cookie)

    except ValueError as e:

        print(e)

        return None

    return cookie


def get_username(self):

    if self.get_client_request_type() ==
"client_requests.SIGNUP":

        return None

    _,_,_,username,_ = self.split_header()

    username_parts = username.split(": ")

    _,username = username_parts

    username = username.replace("\n","")

    return username


def get_bucket_id(self):

    if self.get_client_request_type() ==
"client_requests.SIGNUP":

        return None
```

```
_,_,_,_,bucketid = self.split_header()
bucket_id_parts = bucketid.split(": ")
_,bucketid = bucket_id_parts
return bucketid
```

הקובץ copyCatHandle.py

```
#outside libraries

import sqlite3

import os

import sys

import random

import uuid

import json

import zlib

import smtplib

import base64

import pathlib

from Crypto.Cipher import AES

import Crypto.Random

#project file

import copyCatServer_comm

import serverSideConstants as const

from request_types import client_requests

#creates the dictionary with the information that will be
displayed to the user in the open backup window

class bucket_data_rep:

    def __init__(self, bucket_id):

        self.backup_dict =

self.get_backup_dict(bucket_id)
```

```
#will return a dictionary where the key will be the
serial number of the backup in the bucket and the value
will be a list containing the information about the
backup.
```

```
# [0] = the date it was made, [1] = its size in bytes
```

```
def get_backup_dict(self,bucket_id):

    backup_info =
data_base.get_backup_info(bucket_id)

    backup_info_dict = {}

    for row in backup_info:

        serial_num, upload_date, bytes_in_backup =
row

        backup_info_dict[serial_num] =
[upload_date,bytes_in_backup]

    return backup_info_dict
```

```
class backup_data_rep:
```

```
def __init__(self,directory): #the directory is the
location of the backup within the storage
```

```
#this dictionary will have a folder name as the
key and the value as a list of its content.
```

```
os_walk_result = next(os.walk(directory))
```

```
_,folder_list,file_list = os_walk_result
```

```
self.file_dict = {}
```

```
dir_parts = os.path.split(directory)
```

```
        first_key = dir_parts[1]

        self.file_dict[first_key] = file_list

        folder_dict = {}

        self.file_dict[first_key].append(self.folder_dicts(
            folder_list,directory, folder_dict))

#this function will create the dictionary by going over
all the files in the backup tree style

    def folder_dicts(self, folders,
        directory,folder_dict):

        for folder in folders:

            current_path = os.path.join(directory,folder)

            os_walk_result = next(os.walk(current_path))

            _,inner_folders,file_list = os_walk_result

            folder_dict[folder] = file_list

            folder_dict[folder].append(self.folder_dicts(
                inner_folders,current_path,{}))

        return folder_dict

    def get_file_dict(self):

        return self.file_dict

#this is the main fucntion that will call all other parts
to fullfill the client's request

def handle_client(client_socket,message, comms):

    request_str = message
```

```
request = BOOP_recieve(message)

request_type = request.get_client_request_type()

cookie = request.get_cookie()

username = request.get_username()

bucket_id = request.get_bucket_id()


match request_type:

    #this will send one of the cients confirmation
    answers back and also get an email message sent for the
    two factor authentication

    case "client_requests.ACCESS":

        if not data_base.DB_exists() or not
        storage.storage_exists(): #before doing anything each one
        of the cases will make sure the database and storage
        space both exist

            response = send_con_question("500
Internal Server Error")

            if not
data_base.check_username_bucketID(username,bucket_id):

                response = send_con_question("400 Bad
Request")

            else:

                response = send_con_question("200 OK",
username)

                comms.add_response(client_socket,
request_str, response.send())
```

```
case "client_requests.ACCESS_ANS":

    #important, in arguments the users answer to
the con question will always come before the code

    #confirms the users identity

    if data_base.DB_exists() and
storage.storage_exists():

        user_message =
recieve_con_answers(message)

        question_answer =
user_message.get_user_answer()

        code_answer =
user_message.get_user_code()

        result =
security.check_con_answers(username,question_answer,code_
answer)

        if result == (True,True):

            #in the case where the identity is
confirmed the server will send the clients new cookie

            user_cookie =
security.create_new_cookie(username)

            response = send_answer_result("200
OK", user_cookie)

        else:

            user_answer,user_code = result

            response = send_answer_result("401
Unauthorized",None,user_answer,user_code)

        else:
```



```
        response = send_answer_result("500
Internal Server Error")

        comms.add_response(client_socket,
request_str, response.send())

    case "client_requests.DOWNLOAD":

        #will send the client the data of the backup
the user wanted to download

        if data_base.DB_exists() and
storage.storage_exists():

            if not
security.check_cookie(username,cookie): #cases will also
check the user's cookie and if its incorrect the server
will refuse to fullfill the user's request

                response = send_to_download("401
Unautherized")

            else:

                request =
recieve_download_request(message)

                backup_num = request.get_backup_id()

                backup =
storage.get_backup(bucket_id, username, backup_num)

                response = send_to_download("200
OK",backup)

            else:

                response = send_to_download("500 Internal
Server Error")
```

```
        comms.add_response(client_socket,
request_str, response.send())

        case "client_requests.GETBACKUP":

            #sends the user the dictionary of the files in
the backup of the user's choice

            if data_base.DB_exists() and
storage.storage_exists():

                if not
security.check_cookie(username,cookie):

                    response = send_backup_data_rep("401
Unautherized")

                else:

                    request =
recieve_getBackup_request(message)

                    backup_num = request.get_backup_id()

                    backup_directory =
os.path.join(const.base_dir, str(bucket_id),
str(backup_num))

                    response = send_backup_data_rep("200
OK", backup_directory)

                else:

                    response = send_backup_data_rep("500
Internal Server Error")

                    comms.add_response(client_socket,
request_str, response.send())

        case "client_requests.GETBUCKET":
```

```
#sends a dictionary with the information of
the backups in the bucket

    if data_base.DB_exists() and
storage.storage_exists():

        if not
data_base.check_username_bucketID(username,bucket_id):

            response = send_bucket_data_rep("400
Bad Request")

        elif not
security.check_cookie(username,cookie):

            response = send_bucket_data_rep("401
Unautherized")

        else:

            response = send_bucket_data_rep("200
OK",bucket_id)

        else:

            response = send_bucket_data_rep("500
Internal Server Error")

        comms.add_response(client_socket,
request_str, response.send())

case "client_requests.GETFILE":

    #will send the user the unencrypted data of
the file they chose for it to be displayed.

    if data_base.DB_exists() and
storage.storage_exists():

        user_message =
recieve_getfile_request(message)
```

```
        file_directory =
user_message.get_file_directory()

        if os.path.exists(file_directory):

            response = send_file_info("200
OK",username,file_directory)

        else:

            response = send_file_info("404 Not
Found")

    else:

        response = send_file_info("500 Internal
Server Error")

    comms.add_response(client_socket,
request_str, response.send())

    case "client_requests.SIGNUP":

        if data_base.DB_exists() and
storage.storage_exists():

            user_message = recieve_signup(message)

            username = user_message.get_username()

            user_cookie =
security.create_new_cookie(username)

            #this function will check the data base
and see if the username the user put in is already taken,

            # if it is the server will send back a
message to the client that they need to put in a
diffirent username

            #if the usernames fine then it will put
the user's information into the data_base
```

```
        bucket_id =
data_base.sign_up_client(user_message.turn_content_to_list())

        if bucket_id == 0:

            response = send_signup_response("400
Bad Request")

        else:

            handle_storage().create_user_bucket(bucket_id)

            #if the sign up was successful the
server will send the user back their bucket_id

            user_info =
[user_cookie,username,bucket_id]

            response = send_signup_response("200
OK",user_info)

        else:

            response = send_signup_response("500
Internal Server Error")

        comms.add_response(client_socket,
request_str, response.send())

    case "client_requests.UPLOAD":

        #will receive an upload data the client sent
and save it in the storage space

        user_detail_check =
data_base.check_username_bucketID(username,bucket_id)

        if data_base.DB_exists() and
storage.storage_exists():
```

```
        if not user_detail_check:

            response = send_upload_response("400
Bad Request")

        elif not security.check_cookie(username,
cookie):

            response = send_upload_response("401
Unautherized")

        else:

            try:

                user_data =
recieve_upload_request(message).get_d_dictionary()

                handle_storage().create_new_backu
p(bucket_id, username, user_data)

                response =
send_upload_response("200 OK")

            except Exception as e:

                print(f'{e!r}')

                response =
send_upload_response("500 Internal Server Error")

        else:

            response = send_upload_response("500
Internal Server Error")

            comms.add_response(client_socket,
request_str, response.send())

class handle_security:

    def __init__(self):
```

```
self.client_cookies = {}

#reads from the clientcookies text file and adds
the info there to the client_cookie attribute

with open("clientsCookies.txt",'r') as f:

    try:

        file_content = f.read()

        self.client_cookies =
json.loads(file_content)

    except json.decoder.JSONDecodeError:

        self.client_cookies = {}

self.user_codes = {}


#creates the user's encryption key and nonce that
will be used to encrypt their files in the storage space

def create_user_key(self):

    key = Crypto.Random.get_random_bytes(16)

    cipher = AES.new(key,AES.MODE_EAX)

    nonce = cipher.nonce

    return (key,nonce)


#gets the key and the nonce that are saved in the
data base and then uses them to encrypt the data

def encrypt_user_data(self,username,data):

    user_key = data_base.get_user_key(username)

    user_nonce = data_base.get_user_nonce(username)
```

```
        cipher = AES.new(user_key,AES.MODE_EAX,  
nonce=user_nonce)
```

```
        encrypted_data = cipher.encrypt(data)
```

```
        return encrypted_data
```

#gets the key and the nonce that are saved in the data base and then uses them to decrypt the data

```
def decrypt_user_data(self,username,encrypted_data):
```

```
    user_key = data_base.get_user_key(username)
```

```
    user_nonce = data_base.get_user_nonce(username)
```

```
    cipher = AES.new(user_key,AES.MODE_EAX,  
nonce=user_nonce)
```

```
    data = cipher.decrypt(encrypted_data)
```

```
    return data
```

#gets the user's email address and creates an SMTP server and sends through it an email message to the user. the user's code will be sent in the email and saved in the user_codes attribute

```
def send_email(self,username):
```

```
    destination = data_base.get_user_email(username)
```

```
    if destination != None:
```

```
        s = smtplib.SMTP('smtp.gmail.com', 587)
```

```
        s.starttls()
```

```
        s.login(const.server_email_adress,  
const.email_app_password)
```

```
        code = random.randint(100000, 999999)
```



```
        self.user_codes[username] = code

        message = f"subject: reconnecting to
copycat\rhello user {username}!\rwe see you are trying to
reconnect to the copycat server!\renter this code to
confirm your identity\r{code}"

        try:

            s.sendmail(const.server_email_adress,
destination, message)

        finally:

            s.quit()


#creates a new cookie and replaces the old one with
it in the client_cookies attribute

def create_new_cookie(self,username):

    cookie = uuid.uuid4()

    #the while loop will run until the cookie value
will be something that doesnt exist.

    while str(cookie) in
self.client_cookies.values():

        cookie = uuid.uuid4()

    self.client_cookies[username] = str(cookie)

    with open("clientsCookies.txt",'w') as f:

        cookies_dict =
json.dumps(self.client_cookies)

        f.write(cookies_dict)

    return cookie
```

#checks if the con question answer is correct against the data base and checks the code against the user_code attribute

```
def check_con_answers(self,username, user_answer,
user_code):

    if
data_base.check_user_answer_in_DB(username,user_answer)
and user_code == str(self.user_codes[username]):

        return (True,True)

    elif not
data_base.check_user_answer_in_DB(username,user_answer)
and user_code == str(self.user_codes[username]):

        return (False,True)

    elif
data_base.check_user_answer_in_DB(username,user_answer)
and not user_code == str(self.user_codes[username]):

        return (True,False)

    return (False,False)
```

#checks if a cookie is the correct one for the user with the client_cookies attribute

```
def check_cookie(self,username,cookie):

    if self.client_cookies[username] == cookie:

        return True

    return False
```

```
class handle_DB:
```

```

def __init__(self):

    #data base stuff - creating the tables

    self.conn =
sqlite3.connect("COPYCATDB.db",check_same_thread=False)

    #columns: username, bucket_id, email_address,
user_key, user_nonce, conformation_question1,
conformation_question2, conformation_question3

    user_info = self.conn.executescript("CREATE TABLE
IF NOT EXISTS user_info (username TEXT NOT NULL,
bucket_id TEXT NOT NULL, email_address TEXT NOT NULL,
user_key TEXT NOT NULL, user_nonce TEXT NOT NULL,
conformation_question1 TEXT NOT NULL,
conformation_question2 TEXT NOT NULL,
conformation_question3 TEXT NOT NULL, PRIMARY
KEY(username), FOREIGN KEY (bucket_id) REFERENCES
bucket_info(bucket_id), FOREIGN KEY (username) REFERENCES
answers_to_questions(username), FOREIGN KEY
(conformation_question1) REFERENCES
answers_to_questions(conformation_question1), FOREIGN KEY
(conformation_question2) REFERENCES
answers_to_questions(conformation_question2), FOREIGN KEY
(conformation_question3) REFERENCES
answers_to_questions(conformation_question3))")

    #columns: bucket_id, last_update, backup_num,
bytes_in_bucket

    bucket_info = self.conn.executescript("CREATE
TABLE IF NOT EXISTS bucket_info (bucket_id TEXT NOT NULL,
last_update DATE NOT NULL, backup_num INTEGER NOT NULL,
bytes_in_bucket INTEGER NOT NULL, PRIMARY KEY(bucket_id),
FOREIGN KEY (bucket_id) REFERENCES
user_info(bucket_id))")

```

```

        #columns: bucket_id, serial_num, upload_date,
bytes_in_backup

        backup_info = self.conn.executescript("CREATE
TABLE IF NOT EXISTS backup_info (bucket_id TEXT NOT NULL,
serial_num INTEGER NOT NULL, upload_date DATE NOT NULL,
bytes_in_backup IMTEGER NOT NULL, PRIMARY KEY (bucket_id,
serial_num), FOREIGN KEY (bucket_id) REFERENCES
bucket_info(bucket_id))")

        #columns: username, conformation_question1,
conformation_question2, conformation_question3,
conformation_answer1, conformation_answer2,
conformation_answer3

        answers_to_questions =
self.conn.executescript("CREATE TABLE IF NOT EXISTS
answers_to_questions (username TEXT NOT NULL,
conformation_question1 TEXT NOT NULL,
conformation_question2 TEXT NOT NULL,
conformation_question3 TEXT NOT NULL,
conformation_answer1 TEXT NOT NULL, conformation_answer2
TEXT NOT NULL, conformation_answer3 TEXT NOT NULL,
PRIMARY KEY(username), FOREIGN KEY (username) REFERENCES
user_info(username), FOREIGN KEY (conformation_question1)
REFERENCES user_info(conformation_question1), FOREIGN KEY
(conformation_question2) REFERENCES
user_info(conformation_question2), FOREIGN KEY
(conformation_question3) REFERENCES
user_info(conformation_question3))")

        #checks if the data base file exists

        def DB_exists(self):

```

```
        fileExist
= os.path.isfile("C:\\Users\\amiaz\\Documents\\nufar\\DP
project\\COPYCATDB.db")

        return fileExist

    #this function will reiceve the user information from
the registration and put the data in the DB and assign
the user

    # a 'backupID' - which should be called a bucket id
now.

    #user_info = username, bucket_id, email_address,
user_key, conformation_question1, conformation_question2,
conformation_question3.

    def sign_up_client(self, user_info):

        username, email_address, con_question1,
con_answer1, con_question2, con_answer2, con_question3,
con_answer3 = user_info

        bucket_id = random.randint(1000,10000)

        key,nonce = security.create_user_key()

        user_nonce = base64.b64encode(nonce).decode()

        user_key = base64.b64encode(key).decode()

        user_info_params = (username, bucket_id,
email_address, user_key, user_nonce, con_question1,
con_question2, con_question3)

        answers_2_questions_params = (username,
con_question1,con_question2, con_question3,con_answer1,
con_answer2, con_answer3)
```

```

        insert_to_user_info = self.conn.execute(f"INSERT
INTO user_info VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
user_info_params)

        answers_to_questions = self.conn.execute(f"INSERT
INTO answers_to_questions VALUES (?, ?, ?, ?, ?, ?, ?)",
answers_2_questions_params)

        self.conn.execute("commit")

        #ive heard that SQL will only enter values in the
Pk if they arent already there so thats convinient

        handle_storage().create_user_bucket(bucket_id)

        try:

            bucket_info_params = (bucket_id,
storage.get_bytes_in_bucket(bucket_id))

            bucket_info = self.conn.execute(f"INSERT INTO
bucket_info VALUES (?,CURRENT_TIMESTAMP, {0},?)",
bucket_info_params)

            self.conn.execute("commit")

        except sqlite3.Error as er:

            print(er+"\r\nerror in putting the users
information in bucket_info")

            return 0

        return bucket_id

    #checks if the user's bucket id username combo exists
in the data base.

    def check_username_bucketID(self, username,
bucket_id):

        user_info_params = (username, bucket_id)

```

```
        user_info = self.conn.execute(f"SELECT
username,bucket_id FROM user_info WHERE EXISTS (SELECT
username,bucket_id FROM user_info WHERE username == ? AND
bucket_id == ?)", user_info_params)
```

```
        result = user_info.fetchone()
```

```
        if result != None:
```

```
            return True
```

```
        return False
```

#checks if the user's answer of the question is one of his answers saved in the database

```
def
```

```
check_user_answer_in_DB(self,username,user_answer):
```

```
    answer_params = (username,)
```

```
    answer = self.conn.execute(f"SELECT
conformation_answer1, conformation_answer2,
conformation_answer3 FROM answers_to_questions WHERE
username == ?", answer_params)
```

```
    answer_str = answer.fetchall()
```

```
    for answer in answer_str[0]:
```

```
        if answer == user_answer:
```

```
            return True
```

```
    return False
```

#backup num meaning the number of backups in a specific bucket, not backup id

#this function will need to get the correct num from the DB

```
def get_user_backup_num(self, bucket_id):  
    backup_info_params = (bucket_id,)   
  
    grab_backup_num = self.conn.execute(f"SELECT  
backup_num FROM bucket_info WHERE bucket_id = ?;",  
backup_info_params)  
  
    try:  
        backup_num = grab_backup_num.fetchone()[0]  
  
        return backup_num  
  
    except TypeError:  
  
        return 0
```

#this function will get the user's email address

```
def get_user_email(self, username):  
    user_info_params = (username,)   
  
    grab_email_address = self.conn.execute(f"SELECT  
email_address FROM user_info WHERE username = ?",  
user_info_params)  
  
    try:  
        email_address =  
grab_email_address.fetchone()[0]  
  
        return email_address  
  
    except:  
  
        return None
```



```
#this function gets the user's nonce from the
database
```

```
def get_user_nonce(self, username):

    user_info_params = (username,)

    grab_user_nonce = self.conn.execute(f"SELECT
user_nonce FROM user_info WHERE username = ?",
user_info_params)

    encoded_nonce = grab_user_nonce.fetchone()[0]

    user_nonce = base64.b64decode(encoded_nonce)

    return user_nonce
```

```
#this function gets the user;s key from the database
```

```
def get_user_key(self, username):

    user_info_params = (username,)

    grab_user_key = self.conn.execute(f"SELECT
user_key FROM user_info WHERE username = ?",
user_info_params)

    encoded_key = grab_user_key.fetchone()[0]

    user_key = base64.b64decode(encoded_key)

    return user_key
```

```
#this function gets one of the three confirmation
questions saved in the database. the number this function
recieves determines which one
```

```
def get_a_users_conQuestion(self,username,
question_num):

    user_info_params = (username,)
```

```
        match question_num:

            case 1:

                grab_question =

self.conn.execute(f"SELECT conformation_question1 FROM
answers_to_questions WHERE username = ?",
user_info_params)

                question = grab_question.fetchone()[0]

            case 2:

                grab_question =

self.conn.execute(f"SELECT conformation_question2 FROM
answers_to_questions WHERE username = ?",
user_info_params)

                question = grab_question.fetchone()[0]

            case 3:

                grab_question =

self.conn.execute(f"SELECT conformation_question3 FROM
answers_to_questions WHERE username = ?",
user_info_params)

                question = grab_question.fetchone()[0]

        return question
```

#gets all the information about all the backups in the bucket this function recieves the id of.

```
def get_backup_info(self, bucket_id):

    backup_info_params = (bucket_id,)

    return self.conn.execute(f"SELECT serial_num,
upload_date, bytes_in_backup FROM backup_info WHERE
bucket_id == ?", backup_info_params)
```

```
#update the number of the backups in the bucket

def set_user_backup_num(self, bucket_id, backup_num):

    bucket_info_params = (backup_num, bucket_id)

    bucket_info = self.conn.execute(f"UPDATE
bucket_info SET backup_num = ? WHERE bucket_id = ?",
bucket_info_params)

    self.conn.execute("commit")


#updates the last_update value of a specific bucket

def set_latest_update(self, bucket_id):

    bucket_info_params = (bucket_id,)

    bucket_info = self.conn.execute(f"UPDATE
bucket_info SET last_update = CURRENT_TIMESTAMP WHERE
bucket_id = ?", bucket_info_params)

    self.conn.execute("commit")


#update the bytes_in_bucket value of a specific
bucket

def set_bytes_in_bucket(self, bucket_id):

    bucket_info_params =
(storage.get_bytes_in_bucket(bucket_id), bucket_id)

    bucket_info = self.conn.execute(f"UPDATE
bucket_info SET bytes_in_bucket = ? WHERE bucket_id =
?", bucket_info_params)

    self.conn.execute("commit")
```

#sets the bytes_in_backup value of a specific backup
in a specific bucket

```
def set_bytes_in_backup(self, bucket_id, backup_num):

    backup_info_params =
(storage.get_bytes_in_backup(bucket_id, backup_num),
bucket_id, backup_num)

    backup_info = self.conn.execute(f"UPDATE
backup_info SET bytes_in_backup = ? WHERE bucket_id = ?
AND serial_num == ?", backup_info_params)

    self.conn.execute("commit")
```

#adds a new backup to the database

```
def set_new_backup(self, bucket_id, backup_num):

    #columns: bucket_id, serial_num, upload_date,
bytes_in_backup

    backup_info_params = (bucket_id, backup_num,
storage.get_bytes_in_backup(bucket_id, backup_num))

    backup_info = self.conn.execute(f"INSERT INTO
backup_info VALUES (?, ?, CURRENT_TIMESTAMP, ?)",
backup_info_params)

    self.conn.execute("commit")
```

#calls the other functions that update the
information that will be changed after the creation of a
new backup.

```
def new_backup_update(self, bucket_id, backup_num):

    self.set_user_backup_num(bucket_id, backup_num)

    self.set_latest_update(bucket_id)
```

```
        self.set_bytes_in_bucket(bucket_id)

        self.set_new_backup(bucket_id, backup_num)

class handle_storage:

    def __init__(self):

        pass

        #checks if the storage space exists, returns a
        boolean value

        def storage_exists(self):

            isExist =
os.path.exists("C:\\Users\\amiaz\\Documents\\copyCatStorage")

            return isExist

        #returns a list with all the directories of all the
        files in a specifc backup, using a recursive helper
        function

        def get_backup_file_list(self, bucket_id,
        backup_num):

            file_list = []

            backup_path =
os.path.join(const.base_dir, str(bucket_id), str(backup_num
        ))

            os_walk_result = next(os.walk(backup_path))

            _, folders, files = os_walk_result

            for file in files:
```

```
        file_path = os.path.join(backup_path, file)

        file_list.append(file_path)

    return

self.files_in_folder(folders, backup_path, file_list)

#the recursive helper function

def files_in_folder(self, folders, directory,
file_list):

    for folder in folders:

        current_path = os.path.join(directory, folder)

        os_walk_result = next(os.walk(current_path))

        _, inner_folders, files = os_walk_result

        for file in files:

            file_path = os.path.join(current_path,
file)

            file_list.append(file_path)

        self.files_in_folder(inner_folders, current_pa
th, file_list)

    return file_list

#uses the get_backup_file_list function to get the
list of all the directories of the files in the backup
and creates a dictionary from it where the key is the
directory (directory from the backup folder and not
before that) and the value is the decrypted and
compressed data of the file
```

```

def get_backup(self, bucket_id, username,
backup_num):

    #this function will create an information
dictionary (key = directory, value = data)

    file_list =
self.get_backup_file_list(bucket_id,backup_num)

    backup_data = {}

    for file in file_list:

        with open(file,'rb') as f:

            file_name = file[len(const.base_dir)+6:]

            encrypted_content = f.read()

            content =
security.decrypt_user_data(username,encrypted_content)

            backup_data[file_name] =
base64.b64encode(content).decode()

    return backup_data


#sums the byte num of all the backups in the bucket
and returns the sum of bytes in the bucket

def get_bytes_in_bucket(self, bucket_id):

    #just run a loop on all the backups and run
get_bytes_in_backup

    bucket_path =
os.path.join(const.base_dir,str(bucket_id))

    walk_result = os.walk(bucket_path)

    os_walk_result = next(walk_result)

    _,backups,_ = os_walk_result

```

```
        bytes_in_bucket = 0

        for backup in backups:

            bytes_in_bucket +=
self.get_bytes_in_backup(bucket_id,backup)

        return bytes_in_bucket


#this function starts the process of summing the
number of bytes in the backup.

def get_bytes_in_backup(self, bucket_id, backup_num):

    backup_path =
os.path.join(const.base_dir,str(bucket_id),str(backup_num
))

    os_walk_result = next(os.walk(backup_path))

    _,folders,file_list = os_walk_result

    backup_bytes = 0

    for file in file_list:

        file_path =
os.path.join(backup_path,file)

        backup_bytes +=
self.get_bytes_in_file(file_path)

    bytes_sum = 0

    return backup_bytes +
self.bytes_in_folder(folders, backup_path,bytes_sum)


#this is a helper function to get_bytes_in_backup, it
uses recursive trees to sum the number of bytes in all
the folders of the backup
```



```
def bytes_in_folder(self, folders, directory,
bytes_sum):

    for folder in folders:

        current_path = os.path.join(directory, folder)

        os_walk_result = next(os.walk(current_path))

        _, inner_folders, file_list = os_walk_result

        for file in file_list:

            file_path =
os.path.join(current_path, file)

            bytes_sum +=
self.get_bytes_in_file(file_path)

        return bytes_sum +
self.bytes_in_folder(inner_folders, current_path, bytes_sum
)

    return bytes_sum
```

#this function opens a specified file in a spesified backup and returns the data for sending to the user

```
def get_file(self, username, path):

    with open(path, 'rb') as f:

        file_data = f.read()

        decrypted_data =
security.decrypt_user_data(username, file_data)

        encoded_file_data =
base64.b64encode(decrypted_data)

        return encoded_file_data
```

```
#this function returns the number of bytes in a
specific file
```

```
def get_bytes_in_file(self, path):

    file_info = os.stat(path)

    return file_info.st_size
```

```
#this function will create a storage space for the
user in the cloud, in the argument - will be the id given
to the user in sign_up_client
```

```
def create_user_bucket(self, bucket_id):

    path =
os.path.join(const.base_dir, str(bucket_id))

    try:

        os.makedirs(path, exist_ok=True)

    except OSError as error:

        print(error)
```

```
#this function creates a new backup in the server
storage space, the backup will be based on the user_data
```

```
def create_new_backup(self, bucket_id, username,
user_data): #user data is the big bad dictionary with all
the file paths and file data in it
```

```
#also add the relevant information to the DB
```

```
backup_num =
data_base.get_user_backup_num(bucket_id)

backup_num += 1
```

```

        root =
os.path.join(const.base_dir, str(bucket_id), str(backup_num
)) #const.base_dir = the location within the server where
all the user's buckets will reside

        try:

            os.makedirs(root, exist_ok=True)

            for key,value in user_data.items():

                file_location = key

                newPath =
os.path.join(root, file_location)

                self.create_file_in_backup(username, newPa
th,value)

                data_base.new_backup_update(bucket_id, backup_
num)

        except OSError as error:

            print("in the except in create_new_backup")

            print(error)


        #writes a single file inside the backup

        def create_file_in_backup(self, username,
root,file_data):

            encrypted_data =
security.encrypt_user_data(username, file_data)

            try:

                pathlib.Path(os.path.dirname(root)).mkdir(par
ents=True, exist_ok=True)

                with open(root, 'wb') as f:

```

```
f.write(encrypted_data)

except OSError as error:

    print("failed.")

    print(error)


#this class will handle sending messages to the client
#request outcome - the function handle_client will pass
this code to this class. this class is only for creating
a message to send to the client.

class BOOP_send:

    def __init__(self,request_outcome):

        self.start = f"BOOP {request_outcome}"

        self.header = ""

        self.content = ""

        if request_outcome == "400 Bad Request":

            self.content = "the request couldnt be
completed."

        elif request_outcome == "401 Unautherized":

            self.content = "request couldnt be completed,
confirm identity and try again."

        elif request_outcome == "404 Not Found":

            self.content = "server could not find
requested resource"

        elif request_outcome == "500 Internal Server
Error":

            self.content = "there has been an error
within the server. please try again."
```

```
        self.set_header()

    def set_header(self):

        self.header = f'content-type: text; content-
length: {len(self.content)}'

    def send(self):

        return
f'{self.start}\r\n{self.header}\r\n{self.content}'

class send_con_question(BOOP_send):

    def __init__(self, request_outcome, username=None):

        super().__init__(request_outcome)

        if request_outcome == "200 OK":

            security.send_email(username)

            self.content = f'confirmation-question:
{self.get_con_question(username)};'

            self.set_header()

    def get_con_question(self, username):

        question_num = random.randint(1,3)

        return
data_base.get_a_users_conQuestion(username,question_num)

class send_answer_result(BOOP_send):
```

```

    def __init__(self, request_outcome, cookie=None,
*args):

        super().__init__(request_outcome)

        if request_outcome == "401 Unauthorized":

            self.content = f"the answer to the question
was:{args[0]}. the the code inputed was:{args[1]}"

            if request_outcome == "200 OK":

                self.content = f"cookie: {cookie};
conformation succeded"

                self.set_header()

class send_signup_response(BOOP_send):

    def __init__(self, request_outcome,
user_info_list=None):

        super().__init__(request_outcome)

        if request_outcome == "400 Bad Request":

            self.content += "user is already in server's
data base."

            if request_outcome == "200 OK":

                content_headers = "cookie username bucket_id"
                content_headers = content_headers.split()
                content_dict = {}

                for i,header in enumerate(content_headers):

                    if i == 0:

                        user_info_list[i] =
str(user_info_list[i])

```

```
        content_dict[header] = user_info_list[i]

        self.content = json.dumps(content_dict)

        self.set_header()

class send_upload_response(BOOP_send):

    def __init__(self, request_outcome):

        super().__init__(request_outcome)

        if request_outcome == "200 OK":

            self.content = "the upload was succesful"

            self.set_header()

            return

class send_bucket_data_rep(BOOP_send):

    def __init__(self, request_outcome, bucket_id =
None):

        super().__init__(request_outcome)

        if request_outcome == "200 OK":

            data_rep =
bucket_data_rep(bucket_id).backup_dict

            self.content = json.dumps(data_rep)

            self.set_header()

class send_to_download(BOOP_send):

    def __init__(self, request_outcome, backup = None):

        super().__init__(request_outcome)
```

```
if request_outcome == "200 OK":

    self.content = json.dumps(backup)

    self.set_header()

class send_backup_data_rep(BOOP_send):

    def __init__(self, request_outcome, directory =
None):

        super().__init__(request_outcome)

        if request_outcome == "200 OK":

            files_in_backup =
backup_data_rep(directory).get_file_dict()

            self.content = json.dumps(files_in_backup)

            self.set_header()

class send_file_info(BOOP_send):

    def __init__(self, request_outcome, username=None,
file_directory = None):

        super().__init__(request_outcome)

        if request_outcome == "200 OK":

            file_data =
handle_storage().get_file(username, file_directory)

            directory_to_user =
file_directory[len(const.base_dir)+1:]

            self.content = f"{directory_to_user}:
{json.dumps(file_data.decode())}"

            self.set_header()
```



```
#this class will handle receiving messages to the client.
```

```
class BOOP_recieve:
```

```
    def __init__(self,msg):
```

```
        msg = msg.split("\r\n")
```

```
        self._start, self._header, self._content = msg
```

```
    def get_client_request_type(self):
```

```
        _,request_type = tuple(self._start.split())
```

```
        return request_type
```

```
    def get_content(self):
```

```
        return self._content
```

```
    def get_header_length(self):
```

```
        return len(self._header)
```

```
    def split_header(self):
```

```
        stripped_header = self._header.strip()
```

```
        return stripped_header.split(";")
```

```
        #order: content type, content length, username,  
        bucket id
```

```
    def get_content_type(self):
```

```
        content_type,_,_,_,_ = self.split_header()
```

```
type = content_type.split(": ")

_,content_type = type

return content_type


def get_content_length(self):

    _,content_length,_,_,_ = self.split_header()

    length = content_length.split(": ")

    _,content_length = length

    content_length = int(content_length)

    return content_length


def get_cookie(self):

    if self.get_client_request_type() ==
"client_requests.SIGNUP" or
self.get_client_request_type() ==
"client_requests.ACCESS_ANS":

        return None

    _,_,cookie,_,_ = self.split_header()

    try:

        cookie_parts = cookie.split(": ")

        _,cookie = cookie_parts

    except ValueError as e:

        print(e)

        return None

    return cookie
```

```

def get_username(self):

    if self.get_client_request_type() ==
"client_requests.SIGNUP":

        return None

    _,_,_,username,_ = self.split_header()

    username_parts = username.split(": ")

    _,username = username_parts

    username = username.replace("\n","")

    return username


def get_bucket_id(self):

    if self.get_client_request_type() ==
"client_requests.SIGNUP":

        return None

    _,_,_,_,bucketid = self.split_header()

    bucket_id_parts = bucketid.split(": ")

    _,bucketid = bucket_id_parts

    return bucketid


class recieve_signup(BOOP_recieve):

    def __init__(self, str):

        super().__init__(str)


    def content_split(self):

```

```
        content_list = self._content.split(";")

        content_list.pop()

        return content_list


    #converts the content of the message - the
    information the user puts in - to a list.

    def turn_content_to_list(self):

        user_info_list = []

        for field in self.content_split():

            header_info = field.split(": ")

            _,info = header_info

            user_info_list.append(info)

        return user_info_list


    def get_username(self):

        username_field =
self.content_split()[0].split(":")

        return username_field[1][1::]


class recieve_upload_request(BOOP_recieve):

    def __init__(self, str):

        super().__init__(str)


    def get_client_request_type(self):

        _,request_type = tuple(self._start.split())
```

```
        return request_type

    def get_d_dictionary(self):
        file_dictionary = json.loads(self._content)
        for key,value in file_dictionary.items():
            data_bytes = value.encode()

            compressed_data =
base64.b64decode(data_bytes)

            file_dictionary[key] = compressed_data
        return file_dictionary

class recieve_con_answers(BOOP_recieve):
    def __init__(self, str):
        super().__init__(str)

    def content_split(self):
        return self._content.split(";")

    def get_user_answer(self):
        answer,_ = self.content_split()
        _,answer = answer.split(":")
        return answer

    def get_user_code(self):
        _,code = self.content_split()
```

```
_, code = code.split(":")

return code


class recieve_getfile_request(BOOP_recieve):

    def __init__(self, str):

        super().__init__(str)


    def get_file_directory(self):

        file_path =
os.path.join(const.base_dir, self.get_bucket_id(), self._co
ntent)

        return file_path


class recieve_getBackup_request(BOOP_recieve):

    def __init__(self, str):

        super().__init__(str)


    def get_backup_id(self):

        _, backup_id = self._content.split(":")

        return backup_id[1::]


class recieve_download_request(BOOP_recieve):

    def __init__(self, str):

        super().__init__(str)
```

```
def get_backup_id(self):  
    _,backup_id = self._content.split(":")  
    return backup_id[1::]  
  
#these three are the creation of the three main classes  
of the handle  
  
data_base = handle_DB() #this class contains all the  
functions relating to the database and is the only part  
of the code with access to it  
  
security = handle_security() #this function deals with  
elements relating to the cyber security of the project.  
it handles the cookie and the two factor authentication  
  
storage = handle_storage() #this function handles the  
storage space and is the only one with access to it
```

הקובץ serverSideConstants

```
import copyCatHandle as handle

#server constants
server_ip = '0.0.0.0'
server_port = 1729
email_port = 27060
generic_id = 4422
example_con_code = 42069

#storage space
base_dir = "C:\\Users\\amiaz\\Documents\\copyCatStorage"

#handle
server_email_adress = "copycatproject1@gmail.com"
email_app_password = "piqt pxab esym tzel"

#the max length for a message that the server will accept
intially.

max_msg_length = 1024
```